

Application Delivery Fundamentals 2.0 B:

Introduction to Postgres

Parameswari Ettiappan

consulting | technology | outsourcing



Postgres Goals

- History Of Postgre SQL
- Major Features
- Connecting to Postgre SQL DB using psql
- Connecting to Pgadmin tool
- Writing Simple Queries In Postgre
- Understanding Data types
- Understanding Single row Functions in PostgreSQL
- Specific features of SQL in PostgreSQL as compared to Oracle Database

Postgres Goals

- Installing and Configuring Postgre in Windows OS
- Understanding Data types in Postgre
- New Data type data types of Postgre
- Single row functions of Postgre
- Understanding Group Functions and writing Queries (Postgre Specific)

Postgres Goals

- Understanding DDL's in Postgre
- Creating Tables/Constraints
- Understanding DML's and MVCC in Postgre
- Creating Other Database Objects in Postgre
- Views/Materialized View/Indexes/sequences
- Btree/Partial Indexes In Postgre
- Understanding Query Performance issues in Postgre
- Explain Vs Explain Analyze options and interpretation
- Table Statistics

Postgres Goals

- Moving Data the Copy command
- Querying Metadata Information Postgre
- Compare Dictionaries Vs Meta data tables of Postgre
- Table Partitions In Postgre
- Partition Setup

Postgres Goals

- Introduction to PL/PGSQL
- Control structures Overview
- Loops Overview
- Creating Anonymous blocks in postgres
- Understanding Cursors
- Bound Vs Unbound Cursors
- Creating Subprograms (Stored Procedures/Functions) in PLPGSQL
- Exceptions in Postgres
- Database Triggers in Postgre
- Writing a Function as Trigger body
- Basic Architecture of a Postgres DB from the perspective of a developer

What is Postgre SQL

- PostgreSQL is an object-relational database management system (ORDBMS) based on POSTGRES,
- Version 4.21, developed at the University of California at Berkeley Computer Science Department.
- POSTGRES pioneered many concepts that only became available in some commercial database systems much later.

What is Postgre SQL

- PostgreSQL is an open-source descendant of this original Berkeley code.
- It supports a large part of the SQL standard and offers many modern features:
 - complex queries
 - foreign keys
 - triggers
 - pdatable views
 - transactional integrity
 - multiversion concurrency control

What is Postgre SQL

- PostgreSQL can be extended by the user in many ways, for example by adding new
 - data types
 - functions
 - operators
 - aggregate functions
 - index methods
 - procedural languages

Postgre History

- The object-relational database management system now known as PostgreSQL is derived from the POSTGRES package written at the University of California at Berkeley.
- With over two decades of development behind it, PostgreSQL is now the most advanced open-source database available anywhere.

Postgre History

- The POSTGRES project, led by Professor Michael Stonebraker, was sponsored by the Defense Advanced Research Projects Agency (DARPA), the Army Research Office (ARO), the National Science Foundation (NSF), and ESL, Inc.
- The implementation of **POSTGRES** began in 1986.
- In 1994, Andrew Yu and Jolly Chen added an SQL language interpreter to POSTGRES.
- Under a new name, **Postgres95** was subsequently released to the web to find its own way in the world as an opensource descendant of the original POSTGRES Berkeley code.

Postgre History

- By 1996, it became clear that the name “Postgres95” would not stand the test of time.
- We chose a new name, **PostgreSQL**, to reflect the relationship between the original POSTGRES and the more recent versions with SQL capability.
- At the same time, we set the version numbering to start at 6.0, putting the numbers back into the sequence originally begun by the Berkeley POSTGRES project.

Installation

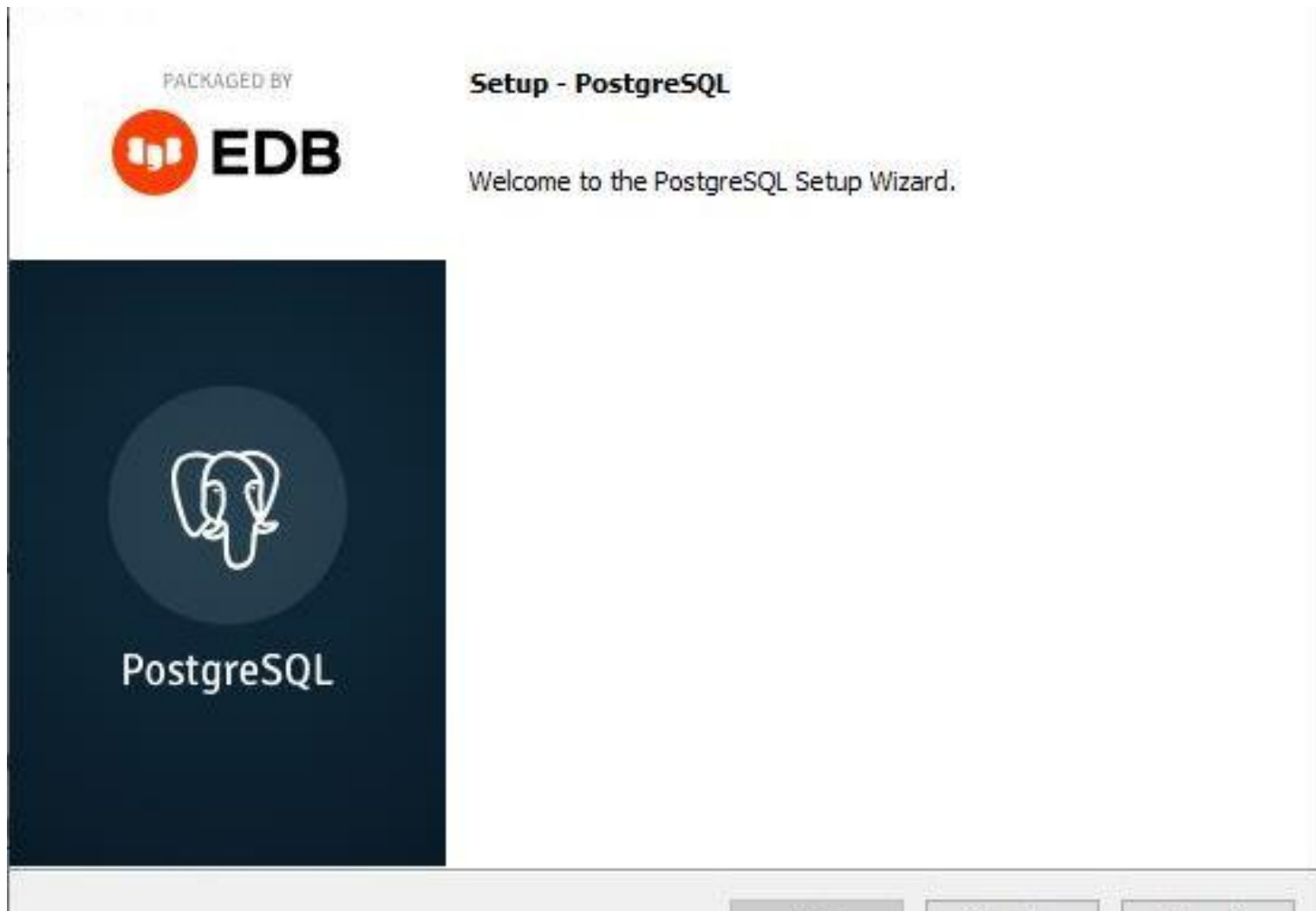
- Download PostgreSQL Installer for Windows
- Install PostgreSQL on Window step by step.
- Verify the Installation

Installation

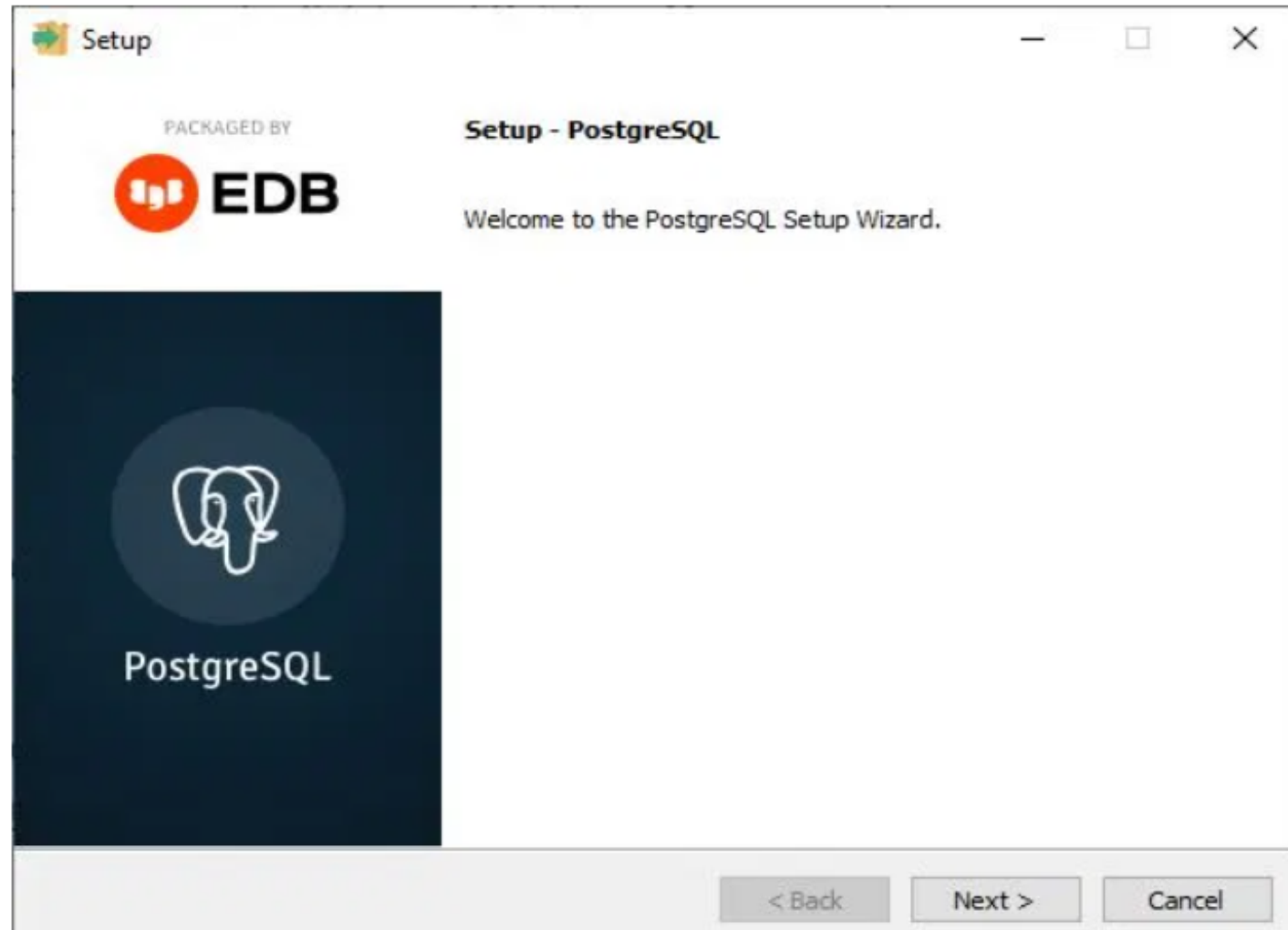
Access the [POSTGRESQL](#) website and download the POSTGRESQL installation package.

Version	Linux x86-64	Linux x86-32	Mac OS X	Windows x86-64	Windows x86-32
13	N/A	N/A	Download	Download	N/A
12.4	N/A	N/A	Download	Download	N/A
11.9	N/A	N/A	Download	Download	N/A
10.14	Download	Download	Download	Download	Download
9.6.19	Download	Download	Download	Download	Download
9.5.23	Download	Download	Download	Download	Download
9.4.26 (Not Supported)	Download	Download	Download	Download	Download
9.3.25 (Not Supported)	Download	Download	Download	Download	Download

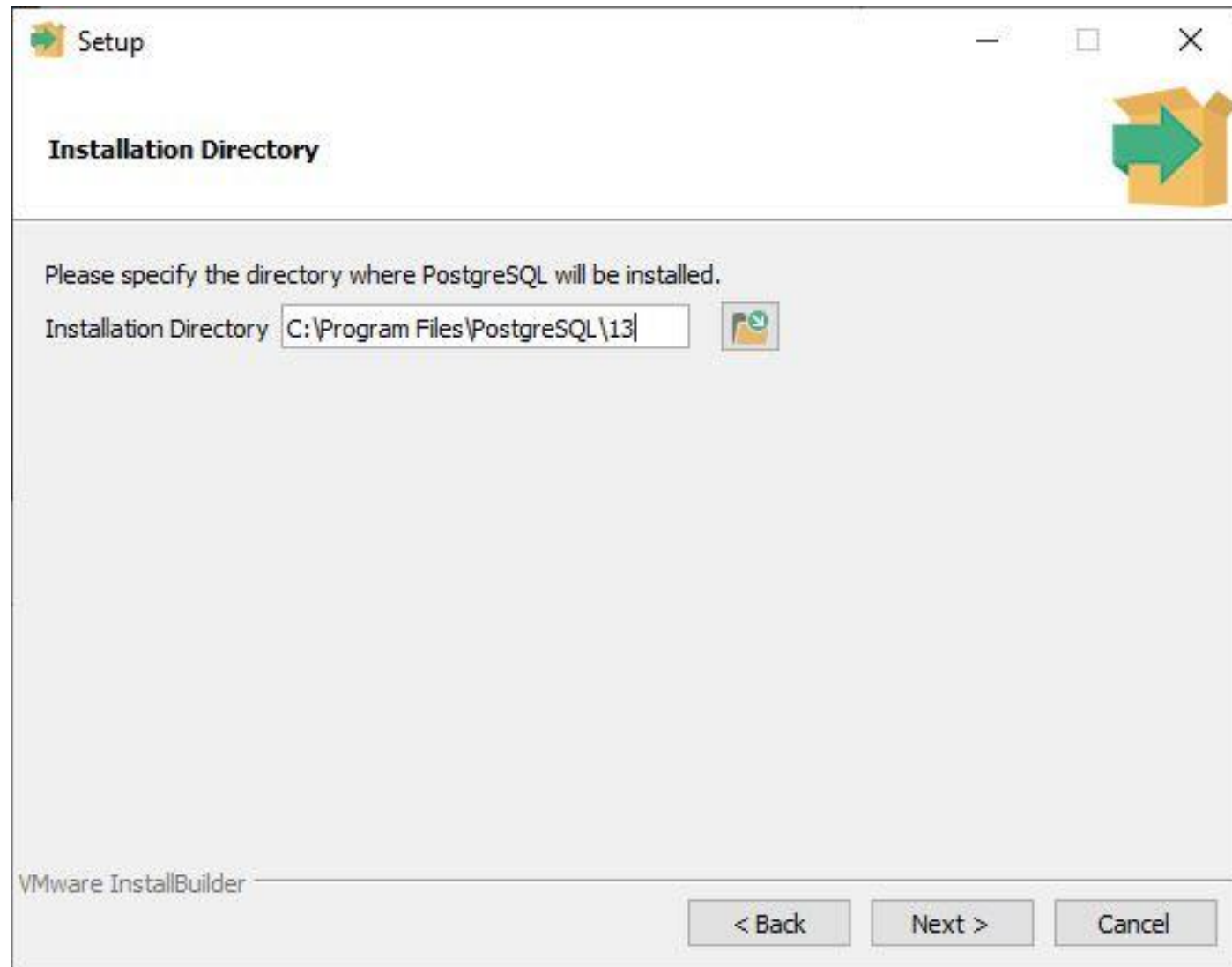
Installation



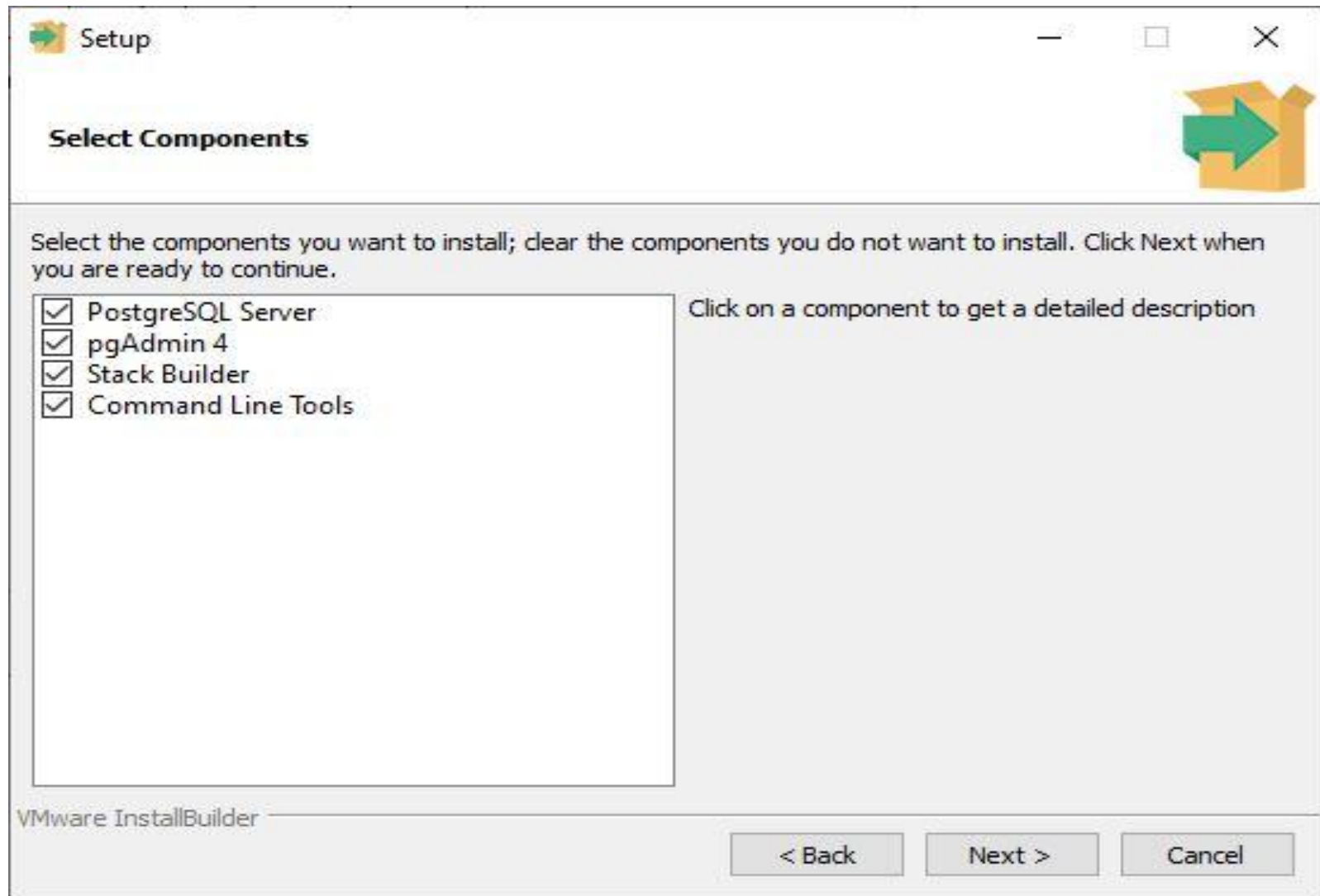
Installation



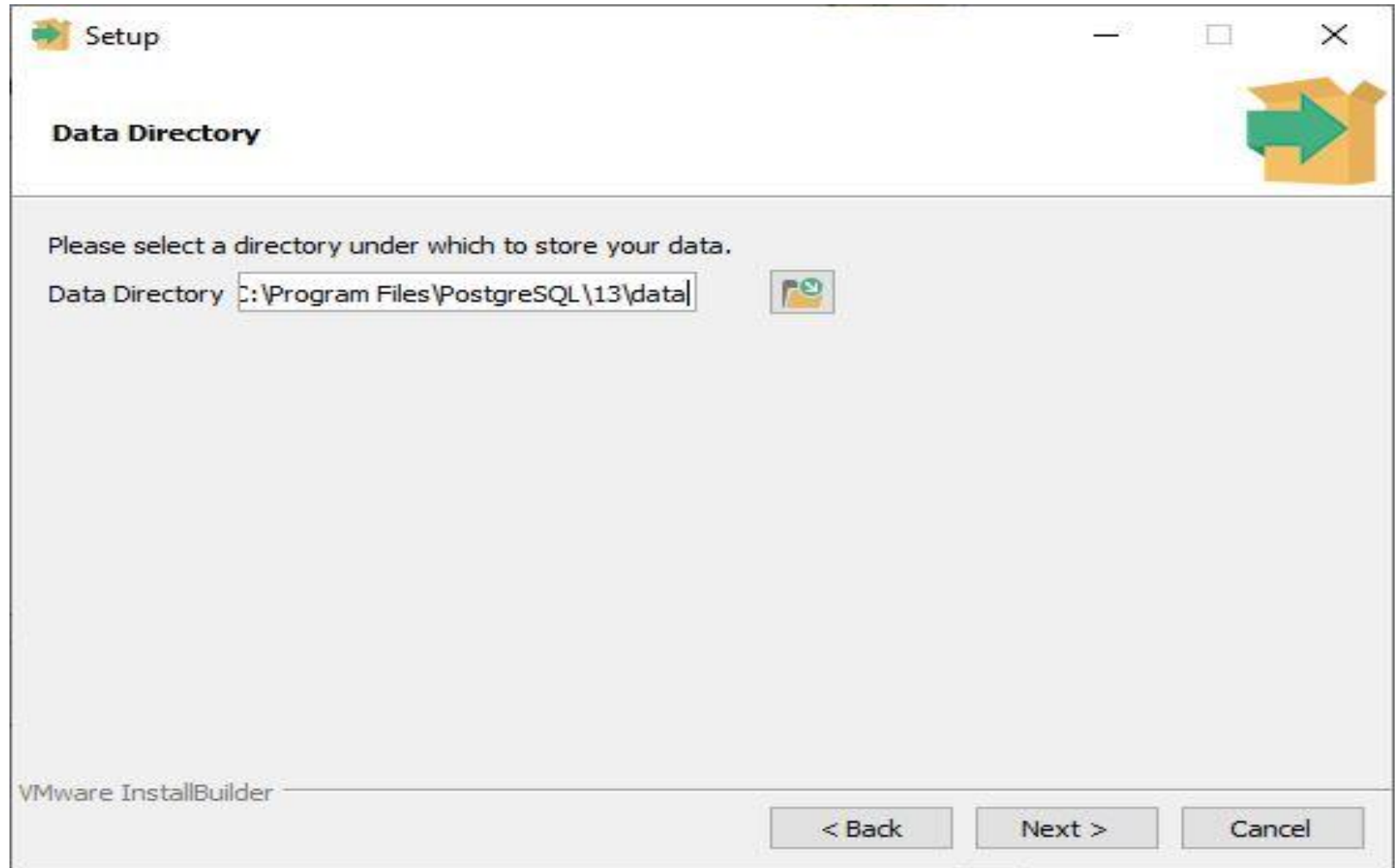
Installation



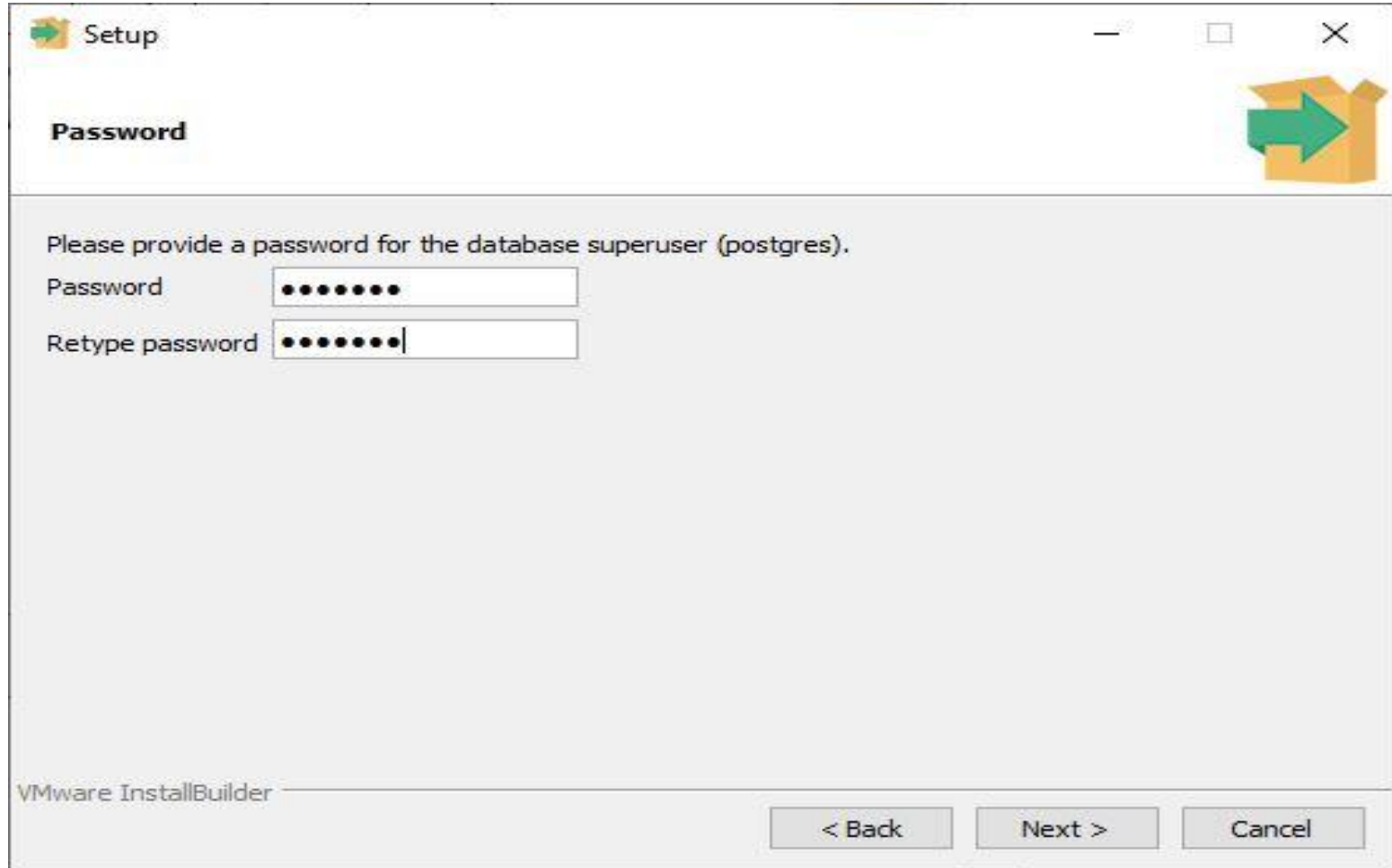
Installation



Installation



Installation



The screenshot shows a window titled "Setup" with a green arrow icon in the top-left corner. The window has standard Windows window controls (minimize, maximize, close) in the top-right corner. Below the title bar, the word "Password" is displayed in bold. In the top-right corner of the main content area, there is a large green arrow pointing right, superimposed on a yellow box icon. The main content area contains the text "Please provide a password for the database superuser (postgres)." followed by two input fields. The first field is labeled "Password" and contains seven black dots. The second field is labeled "Retype password" and contains seven black dots and a vertical cursor line. At the bottom-left of the window, the text "VMware InstallBuilder" is visible. At the bottom-right, there are three buttons: "< Back", "Next >", and "Cancel".

Setup

Password

Please provide a password for the database superuser (postgres).

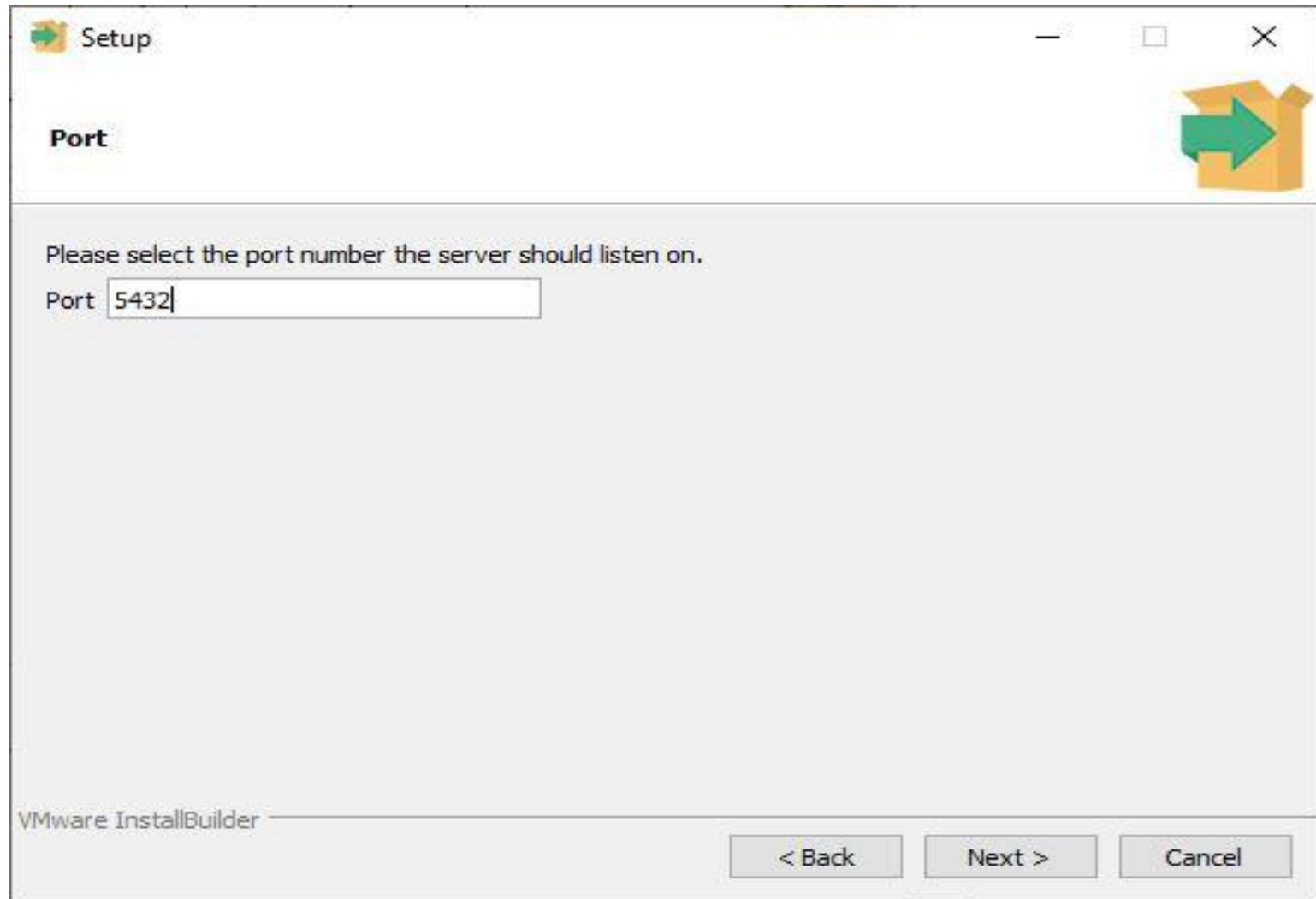
Password

Retype password

VMware InstallBuilder

< Back Next > Cancel

Installation



The screenshot shows a Windows-style window titled "Setup" with a green arrow icon. The window has standard minimize, maximize, and close buttons. A large green arrow icon is also present in the top right corner of the window's content area. The main content area is titled "Port" and contains the instruction "Please select the port number the server should listen on." Below this, there is a text input field labeled "Port" containing the value "5432". At the bottom of the window, the text "VMware InstallBuilder" is displayed on the left, and three buttons labeled "< Back", "Next >", and "Cancel" are on the right.

Setup

Port

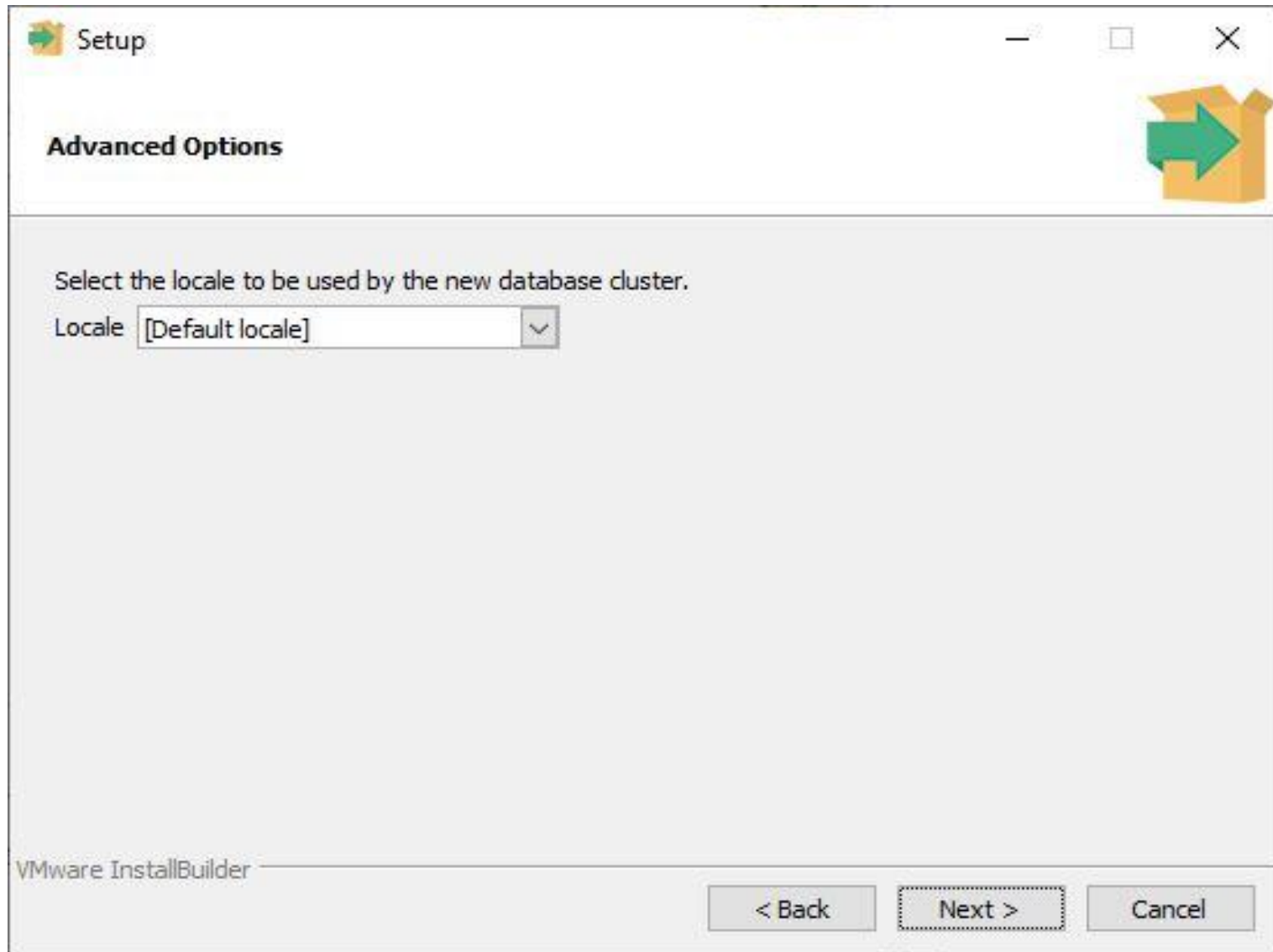
Please select the port number the server should listen on.

Port

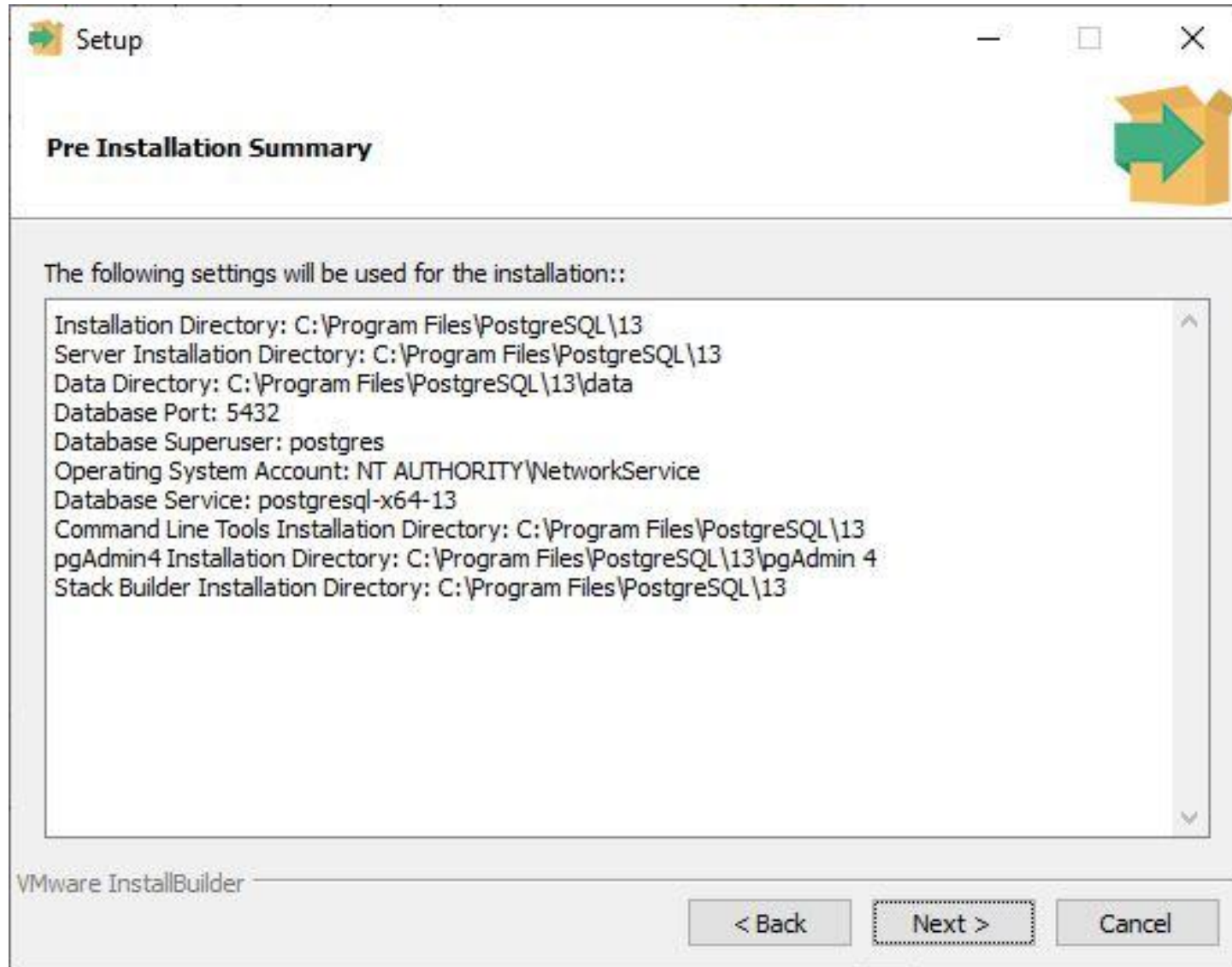
VMware InstallBuilder

< Back Next > Cancel

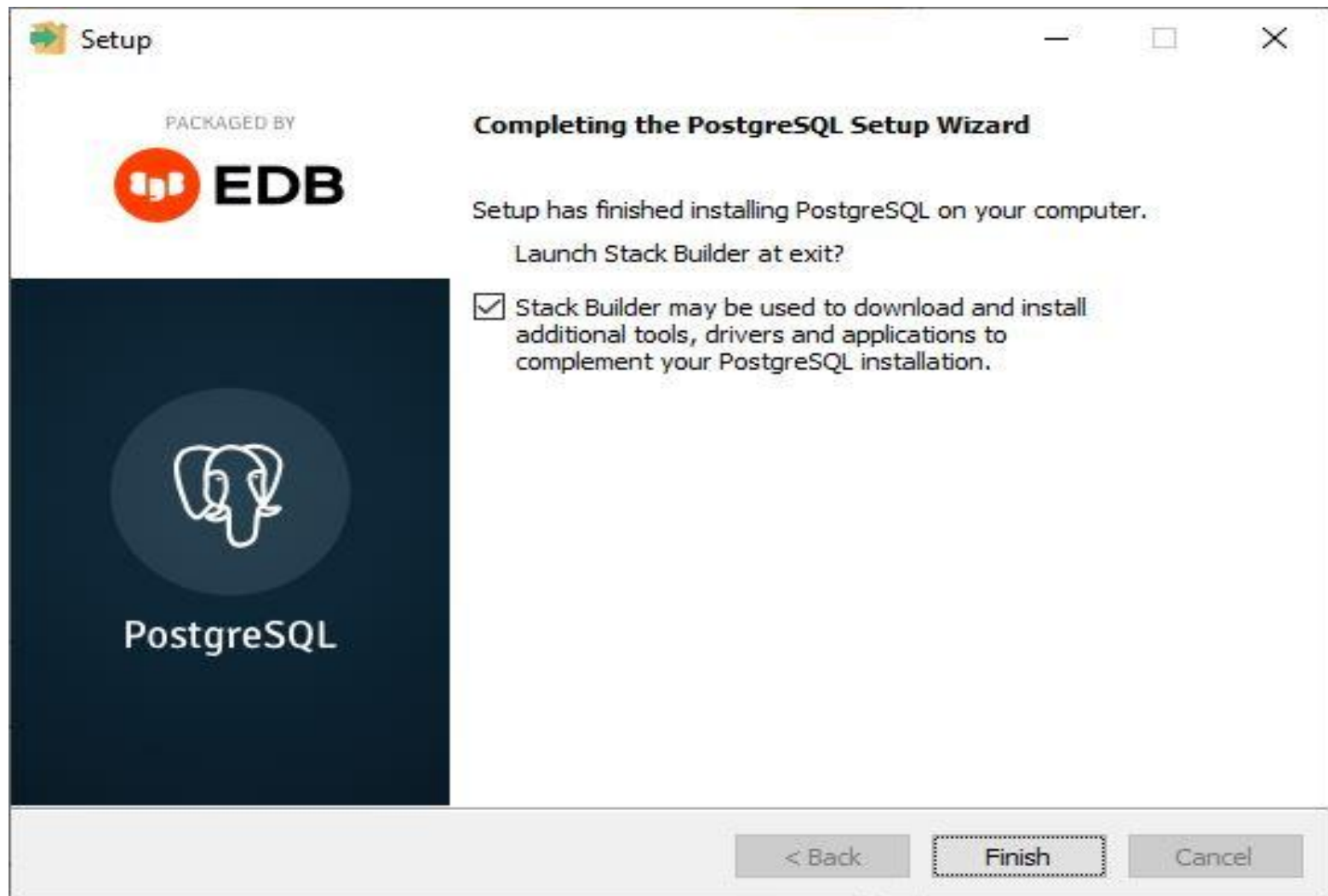
Installation



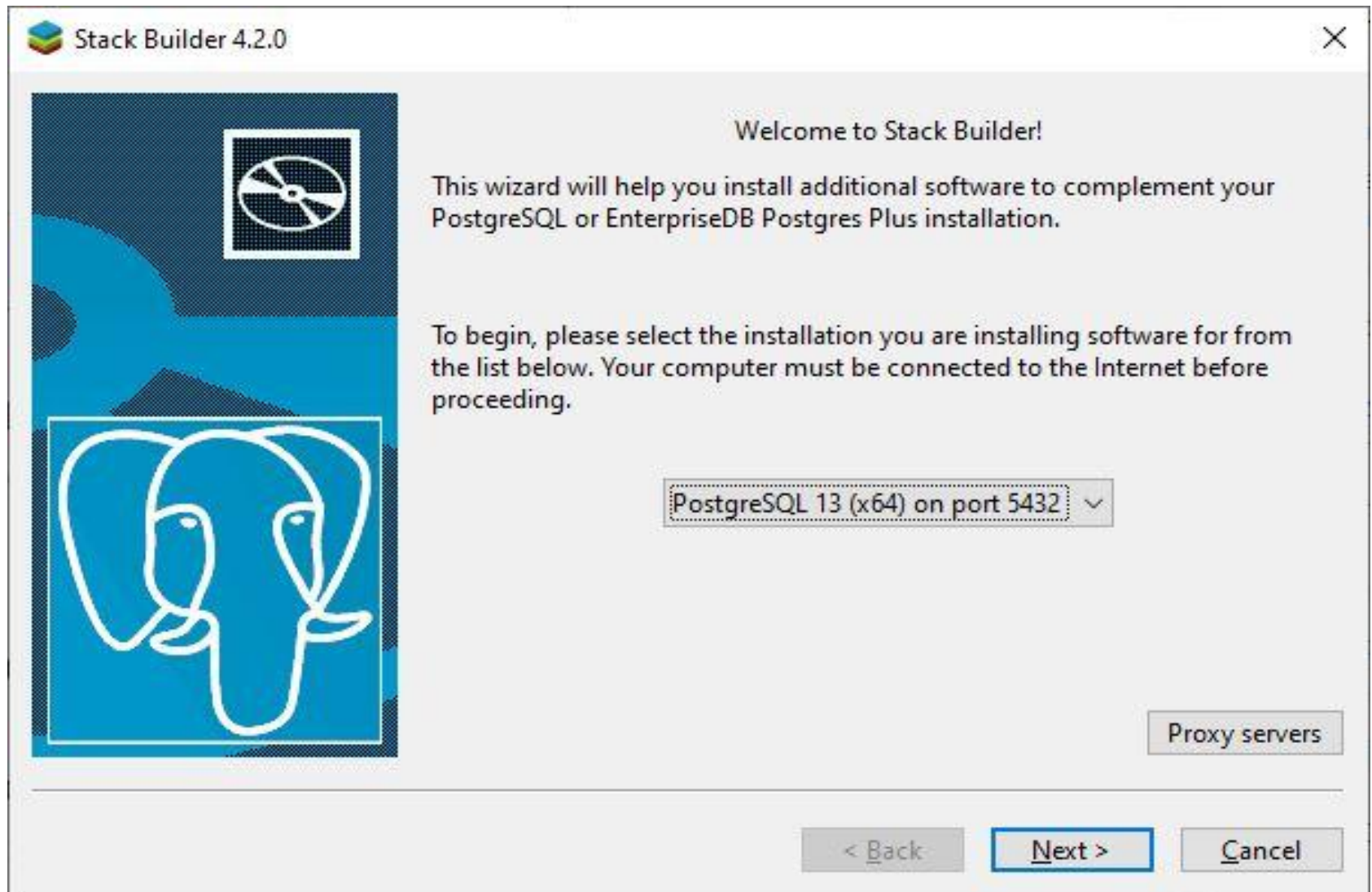
Installation



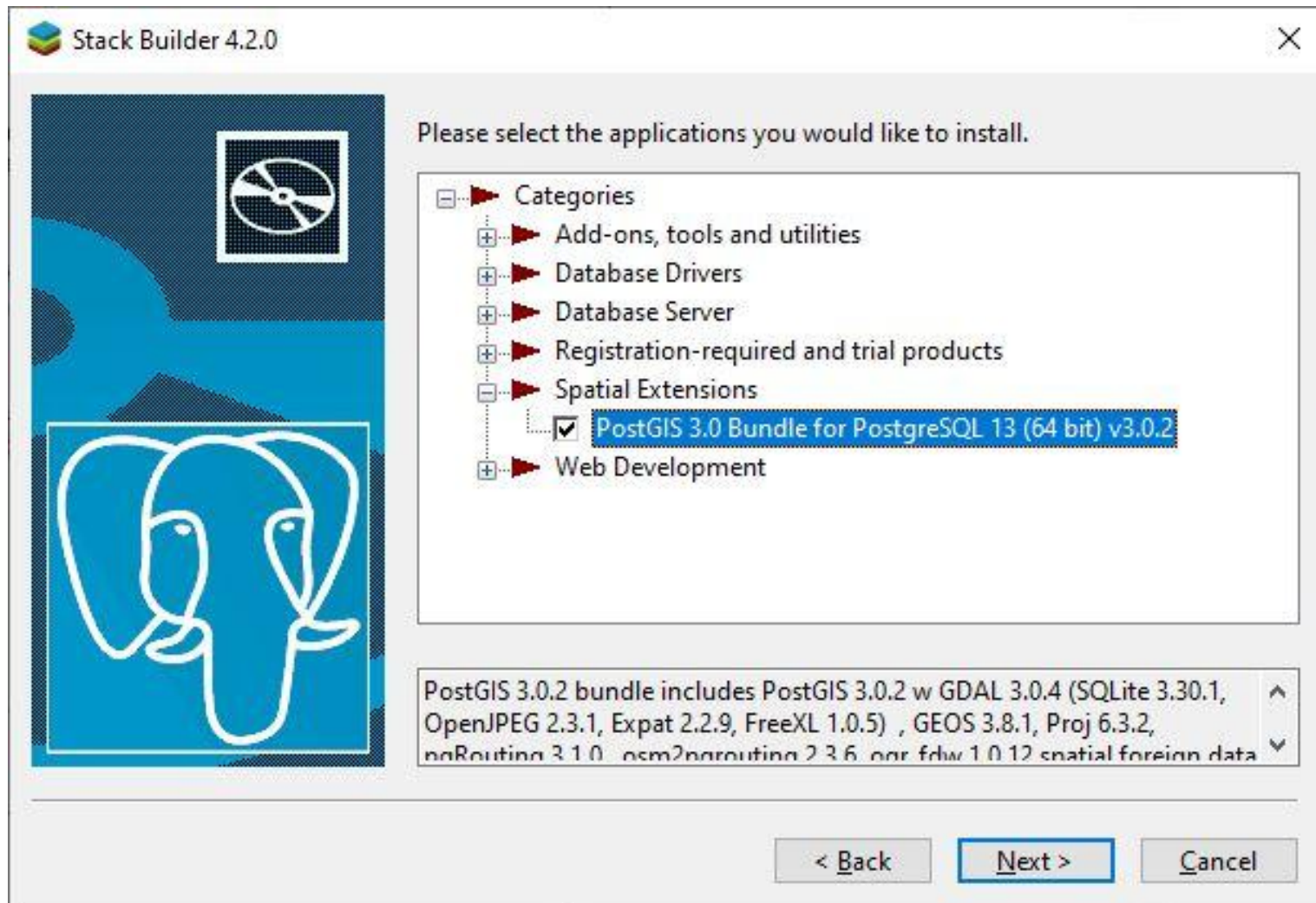
Installation



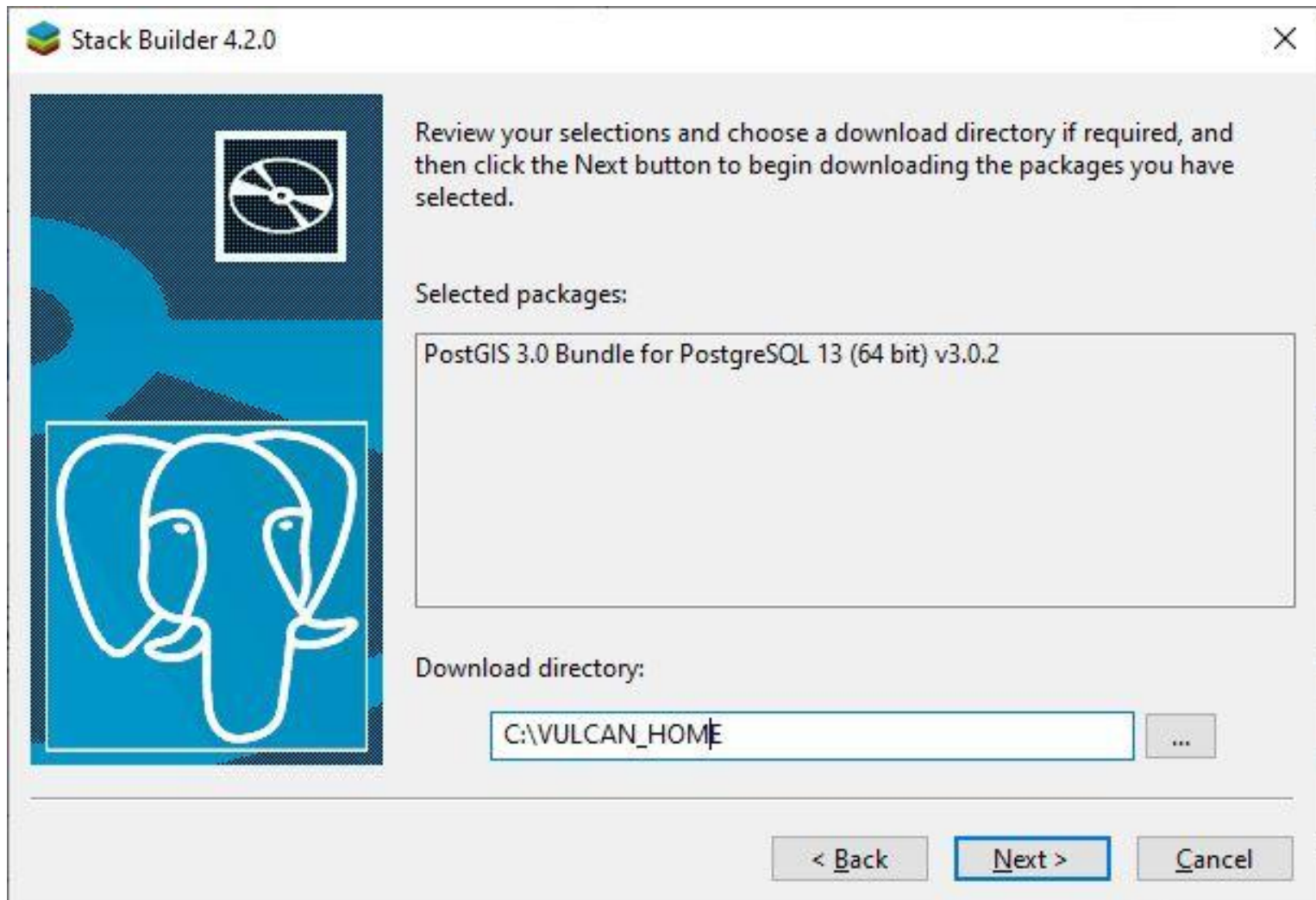
Install PostGIS 3.0 on PostgreSQL 13



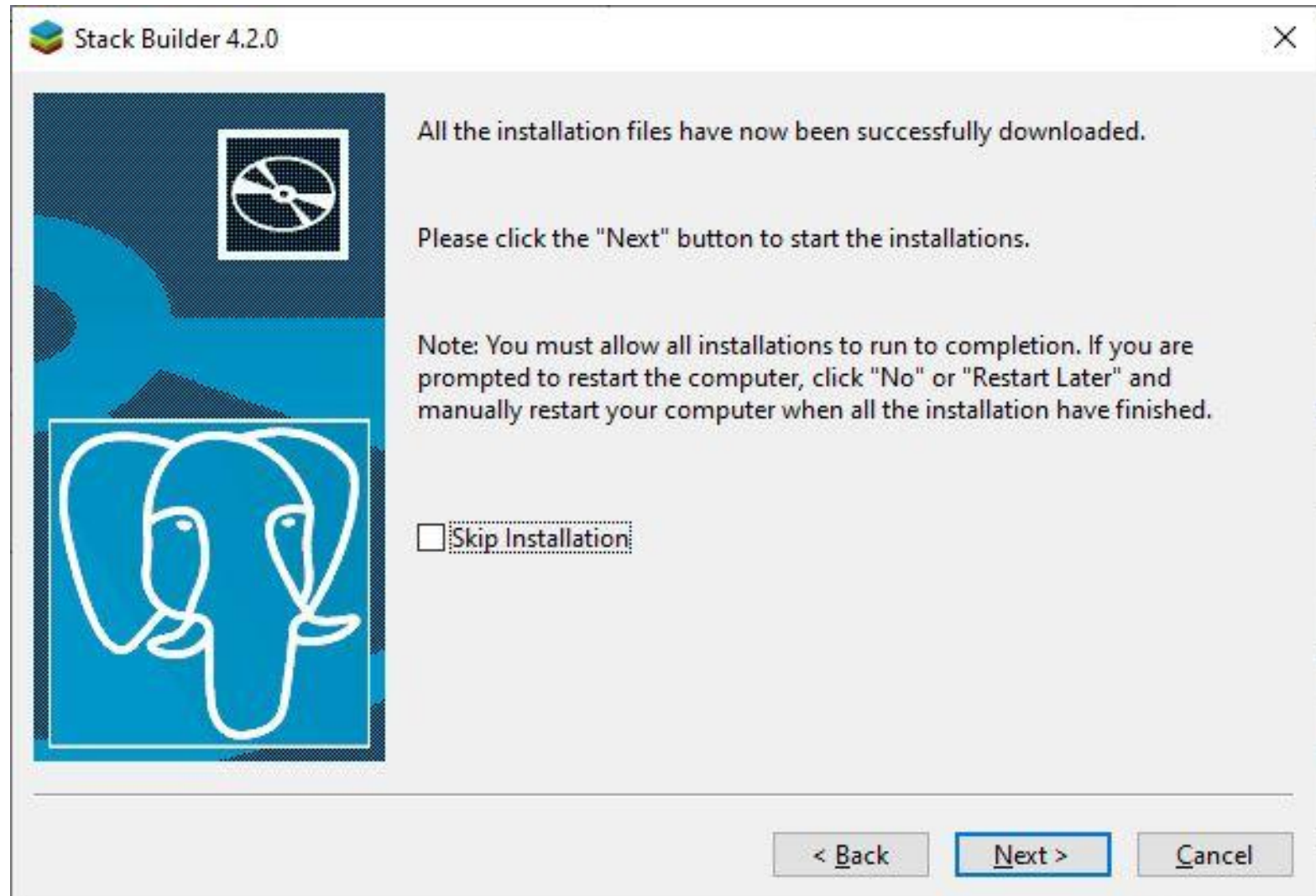
Install PostGIS 3.0 on PostgreSQL 13



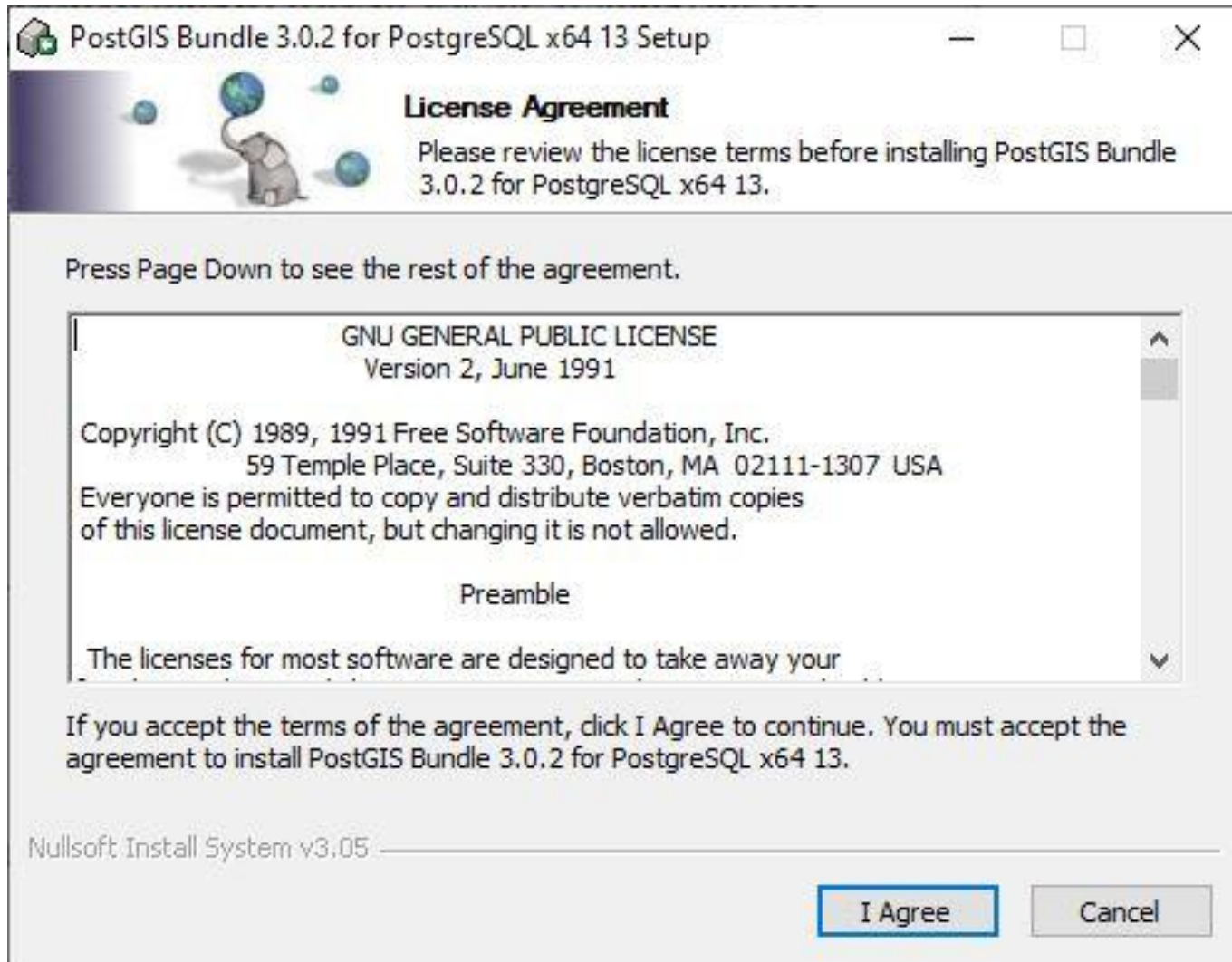
Install PostGIS 3.0 on PostgreSQL 13



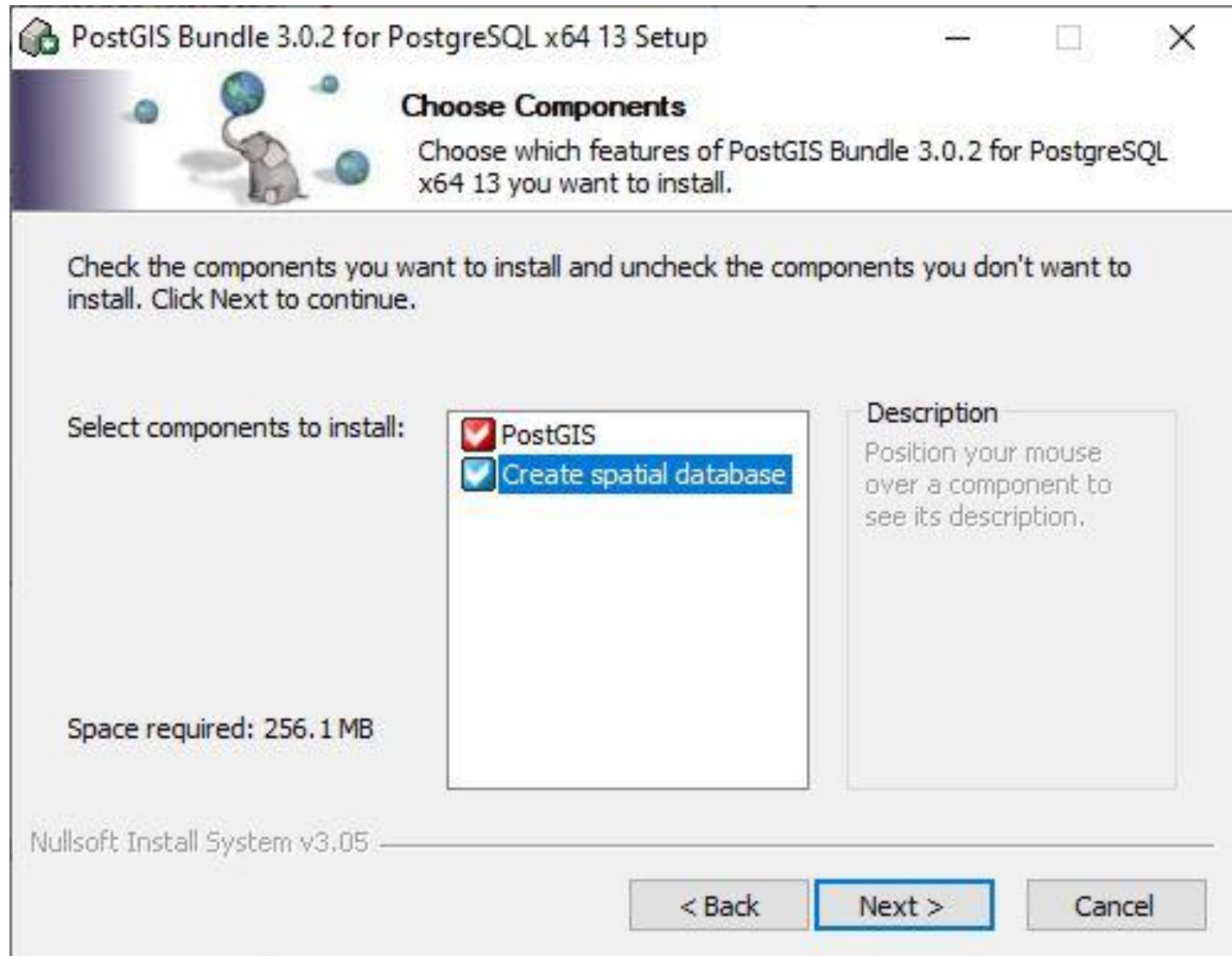
Install PostGIS 3.0 on PostgreSQL 13



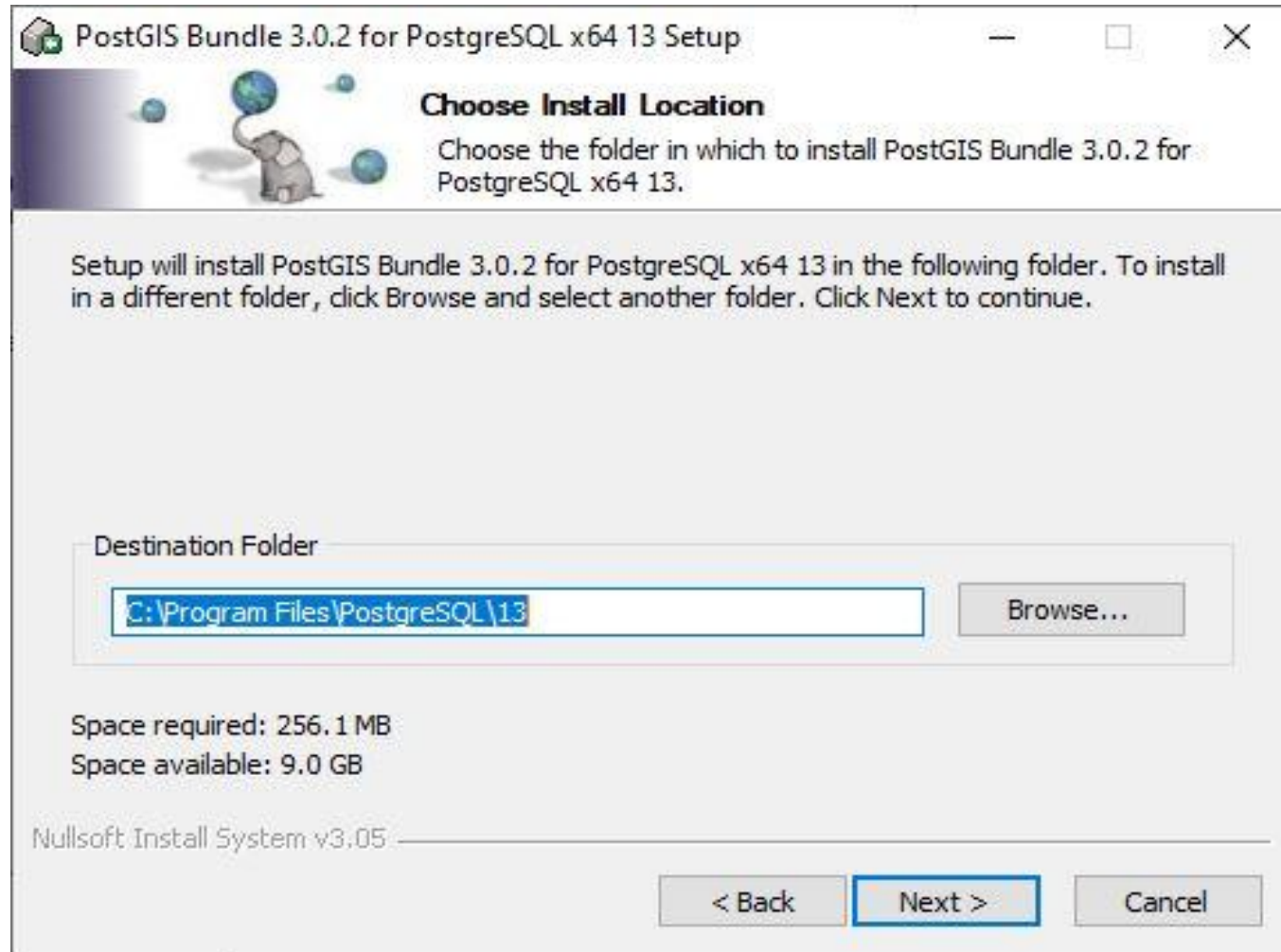
Install PostGIS 3.0 on PostgreSQL 13



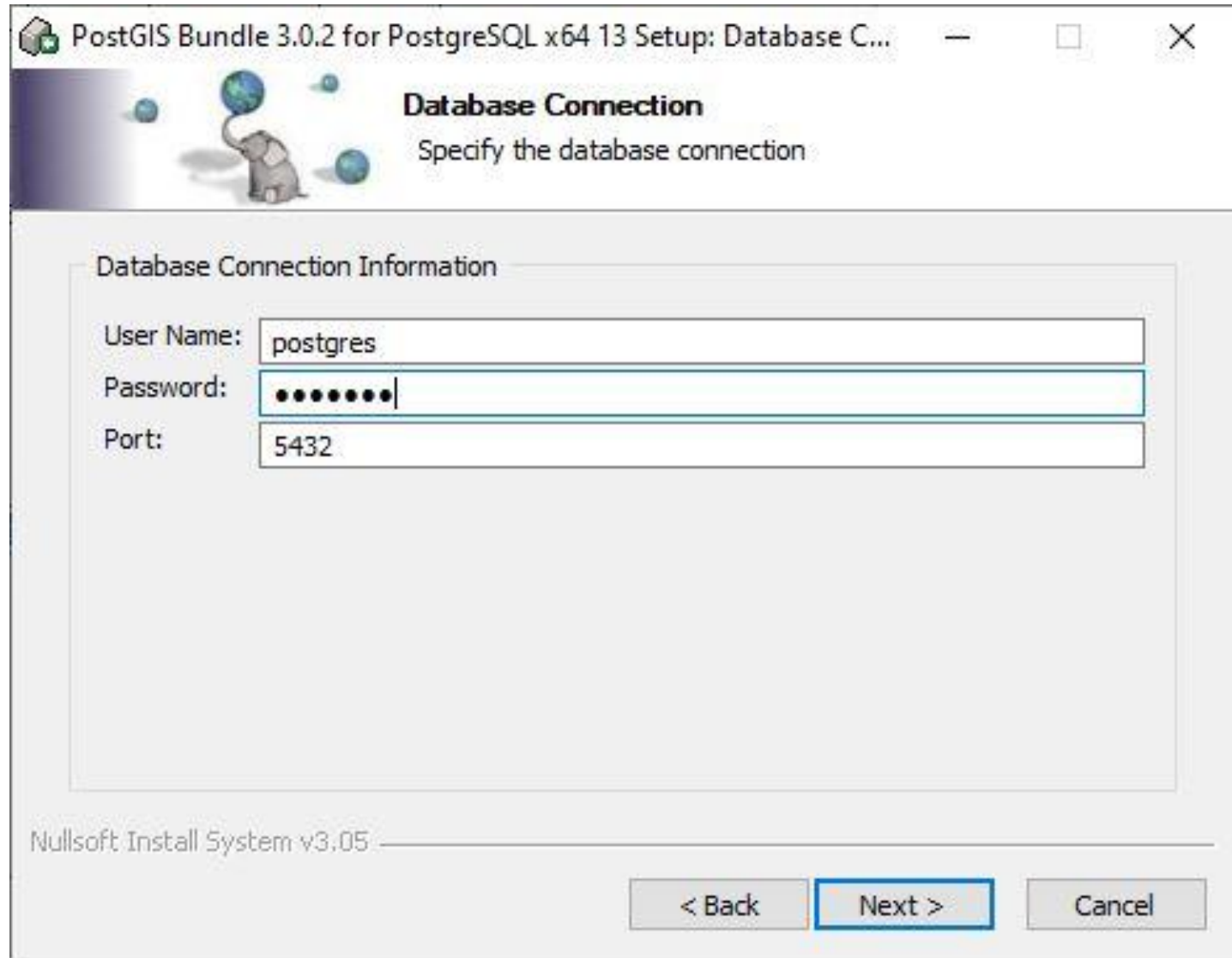
Install PostGIS 3.0 on PostgreSQL 13



Install PostGIS 3.0 on PostgreSQL 13



Install PostGIS 3.0 on PostgreSQL 13



The image shows a Windows installer window titled "PostGIS Bundle 3.0.2 for PostgreSQL x64 13 Setup: Database C...". The window has a standard Windows title bar with minimize, maximize, and close buttons. Below the title bar is a decorative header with a small globe icon and the text "Database Connection" and "Specify the database connection". The main area is titled "Database Connection Information" and contains three input fields: "User Name:" with the value "postgres", "Password:" with a masked password of eight dots, and "Port:" with the value "5432". At the bottom of the window, there is a status bar that says "Nullsoft Install System v3.05" and three buttons: "< Back", "Next >" (which is highlighted with a blue border), and "Cancel".

PostGIS Bundle 3.0.2 for PostgreSQL x64 13 Setup: Database C...

Database Connection
Specify the database connection

Database Connection Information

User Name: postgres

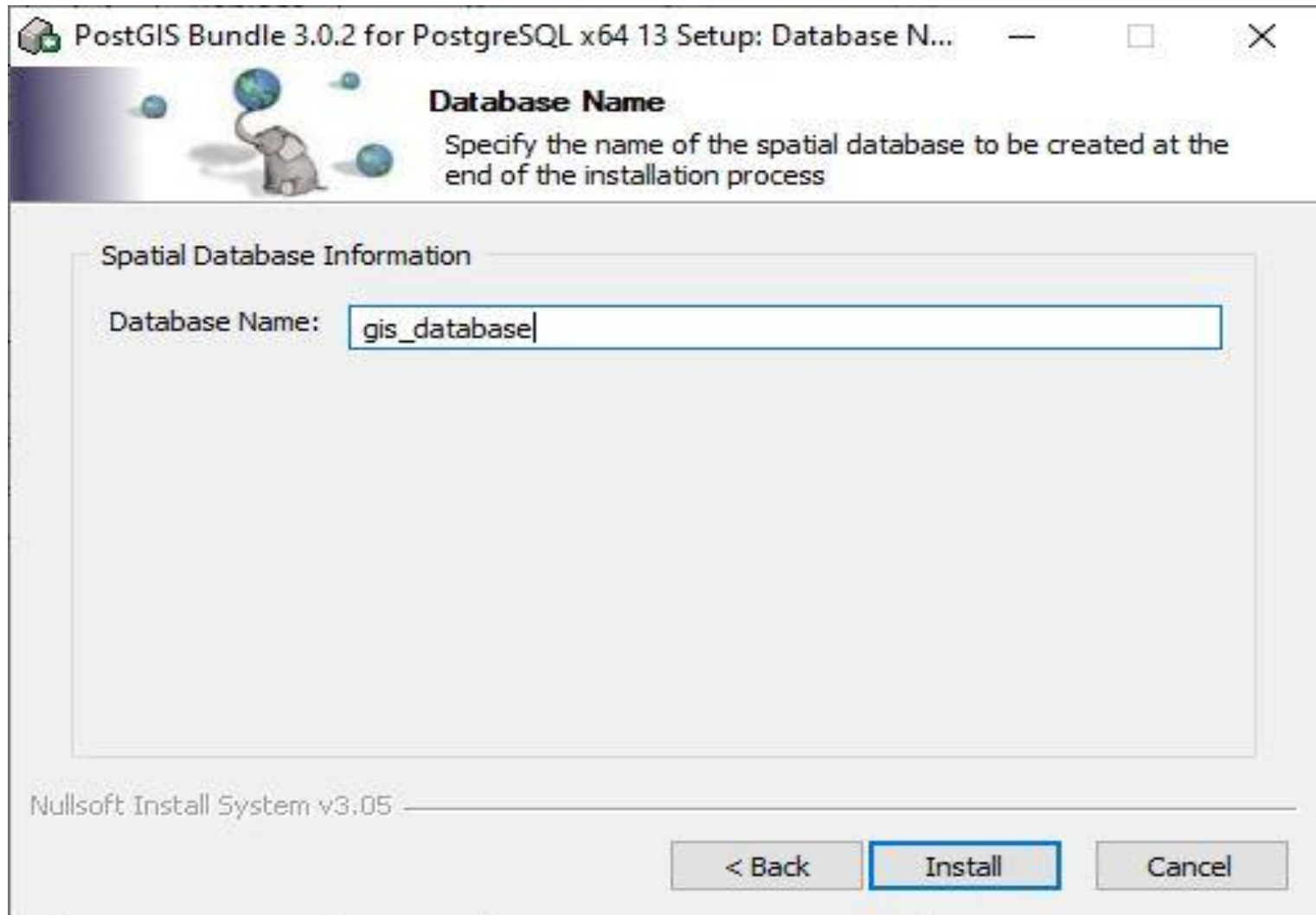
Password: ●●●●●●●●

Port: 5432

Nullsoft Install System v3.05

< Back Next > Cancel

Install PostGIS 3.0 on PostgreSQL 13



Postgre Architecture

- **PostgreSQL: Client-Server Architecture**
 - PostgreSQL follows a **client-server model**
- In the simplest terms, a PostgreSQL service has **2 processes**:
- **Server-side process**
 - The "Postgres" application itself
 - Manages connections, operations, and static & dynamic assets
- **Client-side process (front-end applications)**
 - Applications users use to interact with the database
 - Simple UI acting as the bridge between user and database
 - Communicates typically through APIs

Client Side and Server Side Processes

- **Real-time banking use case: fund transfer**
- A customer transfers ₹5,000 from account ACC1001 to ACC2001.
- Client-side flow
 - The customer enters transfer details in a mobile banking application.
 - The banking API obtains a connection from its connection pool.
 - The API sends a parameterized transaction to PostgreSQL.
 - It waits for success or failure.
 - It displays the transaction status to the customer.

Client Side and Server Side Processes

- **Server-side flow**

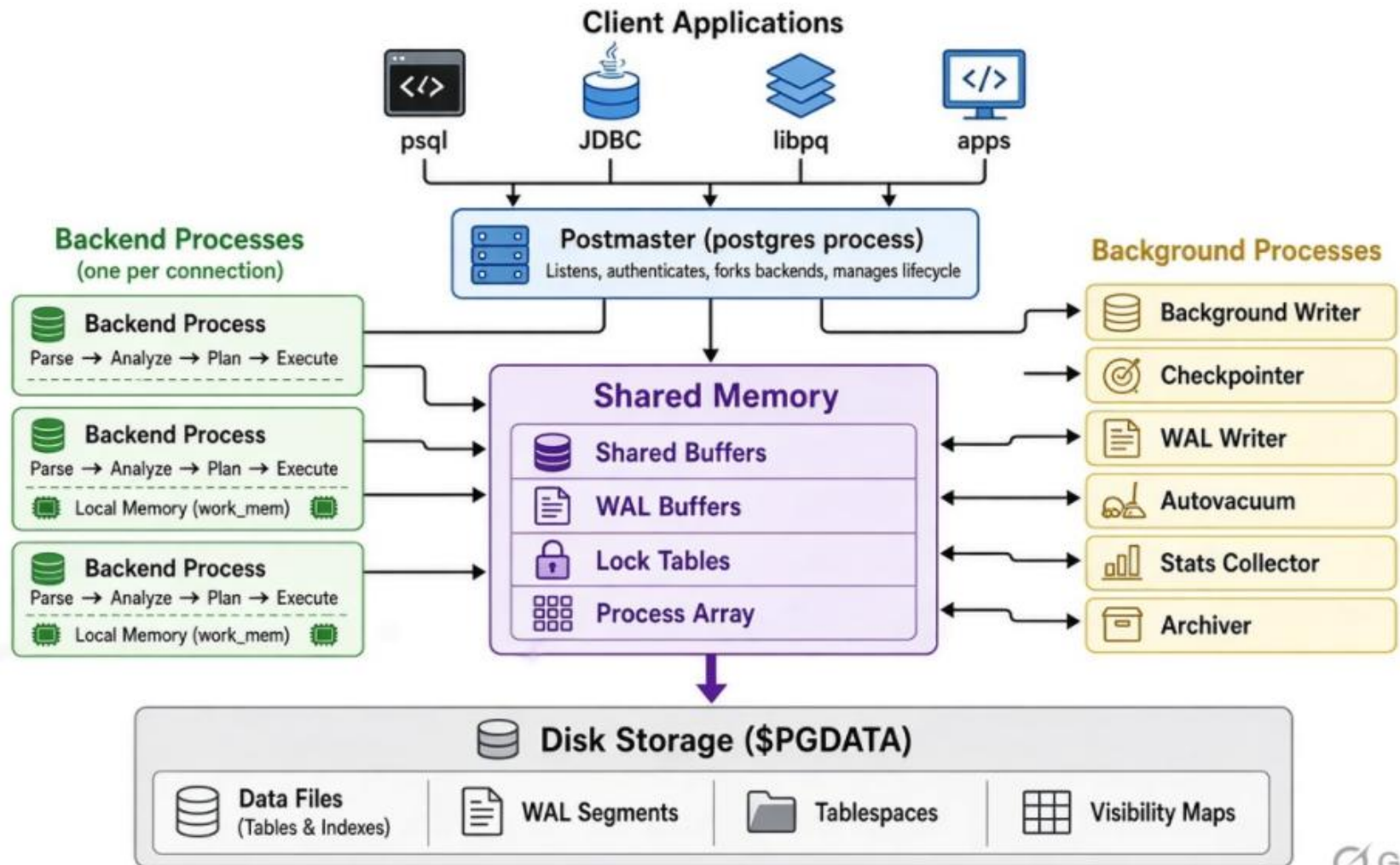
- The postmaster accepts the connection.
- A backend process handles the client session.
- PostgreSQL authenticates the application user.
- The backend begins a transaction.
- It locks the required account rows.
- It verifies the sender's balance.
- It debits the sender.
- It credits the receiver.
- Changes are first recorded in WAL.

Client Side and Server Side Processes

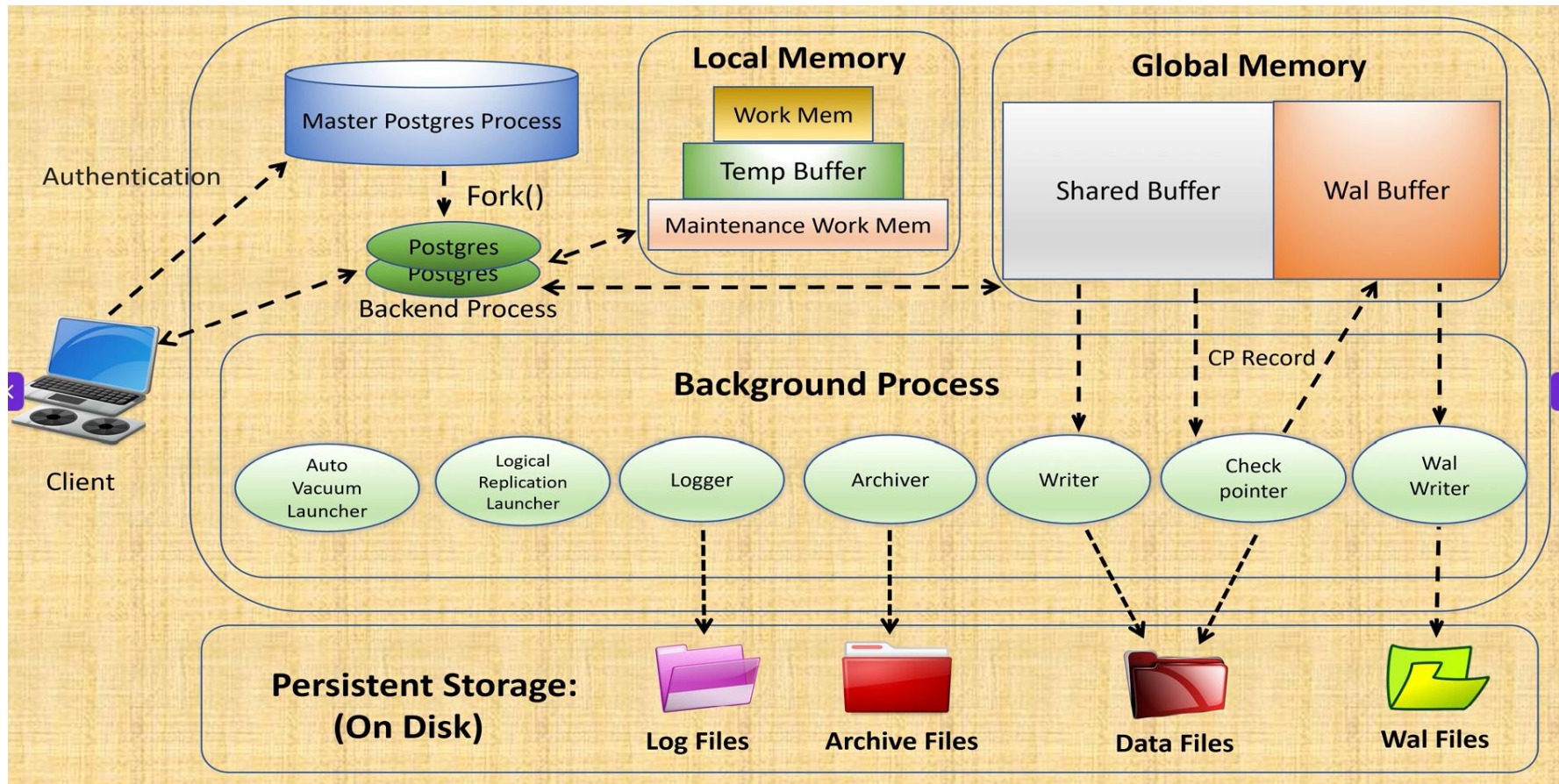
- **Server-side flow**

- PostgreSQL commits the transaction.
- The backend returns success to the banking application.
- Background processes later write modified pages to database files.

Postgre Architecture



Postgre Architecture



Postgre Configuration files

- **PostgreSQL Configuration Files**
- PostgreSQL mainly uses these configuration files:
 - postgresql.conf
 - pg_hba.conf
 - pg_ident.conf
 - postgresql.auto.conf
 - start.conf — Debian/Ubuntu package-managed clusters

Postgre Configuration files

- **Find the active files**
- Always query the running PostgreSQL instance instead of assuming their locations:
 - `sudo -iu postgres psql -d postgres`
- `SHOW config_file;`
- `SHOW hba_file;`
- `SHOW ident_file;`
- `SHOW data_directory;`

Postgre Configuration files

- **Find the active files**
- Display them together:
- SELECT name, setting
- FROM pg_settings
- WHERE name IN (
 - 'config_file',
 - 'hba_file',
 - 'ident_file',
 - 'data_directory'
-);

1. postgresql.conf

- This is the primary PostgreSQL server configuration file.
- It controls:
 - Network connections
 - Memory
 - WAL
 - Replication
 - Logging
 - Autovacuum
 - Query-planner behaviour
 - Resource limits
 - Checkpoints

1. postgresql.conf

- **Important settings**

- listen_addresses = '*'
- port = 5432
- max_connections = 100

- **Memory:**

- shared_buffers = 2GB
- work_mem = 16MB
- maintenance_work_mem = 256MB
- effective_cache_size = 6GB

1. postgresql.conf

- WAL and checkpoints:
 - wal_level = replica
 - max_wal_senders = 10
 - max_replication_slots = 10
 - checkpoint_timeout = 5min
 - max_wal_size = 2GB
 - Logging:
- logging_collector = on
 - log_directory = 'log'
 - log_filename = 'postgresql-%Y-%m-%d_%H%M%S.log'
 - log_connections = on
 - log_disconnections = on
 - log_min_duration_statement = 1000

1. postgresql.conf

- Autovacuum:
 - autovacuum = on
 - autovacuum_max_workers = 3
 - autovacuum_naptime = 1min
- **View a setting**
 - SHOW shared_buffers;
 - SHOW max_connections;
 - SHOW wal_level;

1. postgresql.conf

- **Change a setting**
- Edit the active configuration:
 - `sudoedit /etc/postgresql/18/main/postgresql.conf`
- Reload PostgreSQL:
 - `sudo systemctl reload postgresql`
- Some parameters require a restart:
 - `sudo systemctl restart postgresql`
- Check whether a restart is pending:
 - `SELECT name, setting, pending_restart FROM pg_settings WHERE pending_restart;`

2. pg_hba.conf

- HBA means **Host-Based Authentication**.
- This file controls:
 - Who can connect
 - Which database they can access
 - Where they can connect from
 - Which authentication method is used

2. pg_hba.conf

- **Patroni replication rule**
- Use your actual VPC CIDR or narrower node-specific CIDRs:
- `host replication replicator 172.31.0.0/16 scram-sha-256`
- **Rule order matters**
- PostgreSQL reads pg_hba.conf from top to bottom and applies the **first matching rule**.
- After editing:
 - `SELECT pg_reload_conf();`

2. pg_hba.conf

- Inspect parsed rules and errors:
- SELECT line_number,
 - type,
 - database,
 - user_name,
 - address,
 - auth_method,
 - error
- FROM pg_hba_file_rules;

3. pg_ident.conf

- This file maps operating-system usernames to PostgreSQL roles.
- It is mainly used with authentication methods such as:
 - peer
 - ident
- **Example mapping**
 - # MAPNAME SYSTEM-USERNAME DATABASE-USERNAME
 - admin_map ubuntu postgres
- Reference the map in pg_hba.conf:
 - local all postgres peer map=admin_map
- This permits the Linux user ubuntu to authenticate as the PostgreSQL role postgres, based on the mapping.
- This file is less commonly used than pg_hba.conf.

4. postgresql.auto.conf

- This file is automatically maintained by PostgreSQL when settings are changed with ALTER SYSTEM.
- Example:
 - ALTER SYSTEM SET log_min_duration_statement = '500ms';
 - SELECT pg_reload_conf();
- The setting is written to:
 - \$PGDATA/postgresql.auto.conf
- View it:
 - sudo cat /var/lib/postgresql/18/main/postgresql.auto.conf

4. postgresql.auto.conf

- Reset one setting:
 - `ALTER SYSTEM RESET log_min_duration_statement;`
 - `SELECT pg_reload_conf();`
- Reset all ALTER SYSTEM settings:
 - `ALTER SYSTEM RESET ALL;`
 - `SELECT pg_reload_conf();`
- Do not normally edit postgresql.auto.conf manually.
- Settings in postgresql.auto.conf override matching settings from postgresql.conf.

5. start.conf

- On Debian and Ubuntu package installations, this file controls whether a package-managed PostgreSQL cluster starts automatically.
- Typical location:
 - `/etc/postgresql/18/main/start.conf`
- Possible values include:
 - `auto`
 - `manual`
 - `disabled`
 - `auto`: starts with the PostgreSQL service.
 - `manual`: starts only when explicitly requested.
 - `disabled`: cluster-management commands will not normally start it.
- This file is specific to Debian/Ubuntu cluster-management tools; it is not a standard PostgreSQL core configuration file.

Configuration priority

- When the same parameter is set in multiple places, a higher-priority source wins. A simplified order is:

```
postgresql.conf
    ↓ overridden by
included configuration files
    ↓ overridden by
postgresql.auto.conf
    ↓ overridden by
server command-line options
    ↓ overridden for a database/role/session when applicable
```

Configuration priority

- `SELECT name, setting, source, sourcefile, sourceline, pending_restart FROM pg_settings WHERE name = 'max_connections';`
- **Validate configuration errors**
- Check settings that PostgreSQL could not apply:
- `SELECT sourcefile, sourceline, name, setting, error FROM pg_file_settings WHERE error IS NOT NULL;`
- Reload:
 - `SELECT pg_reload_conf();`

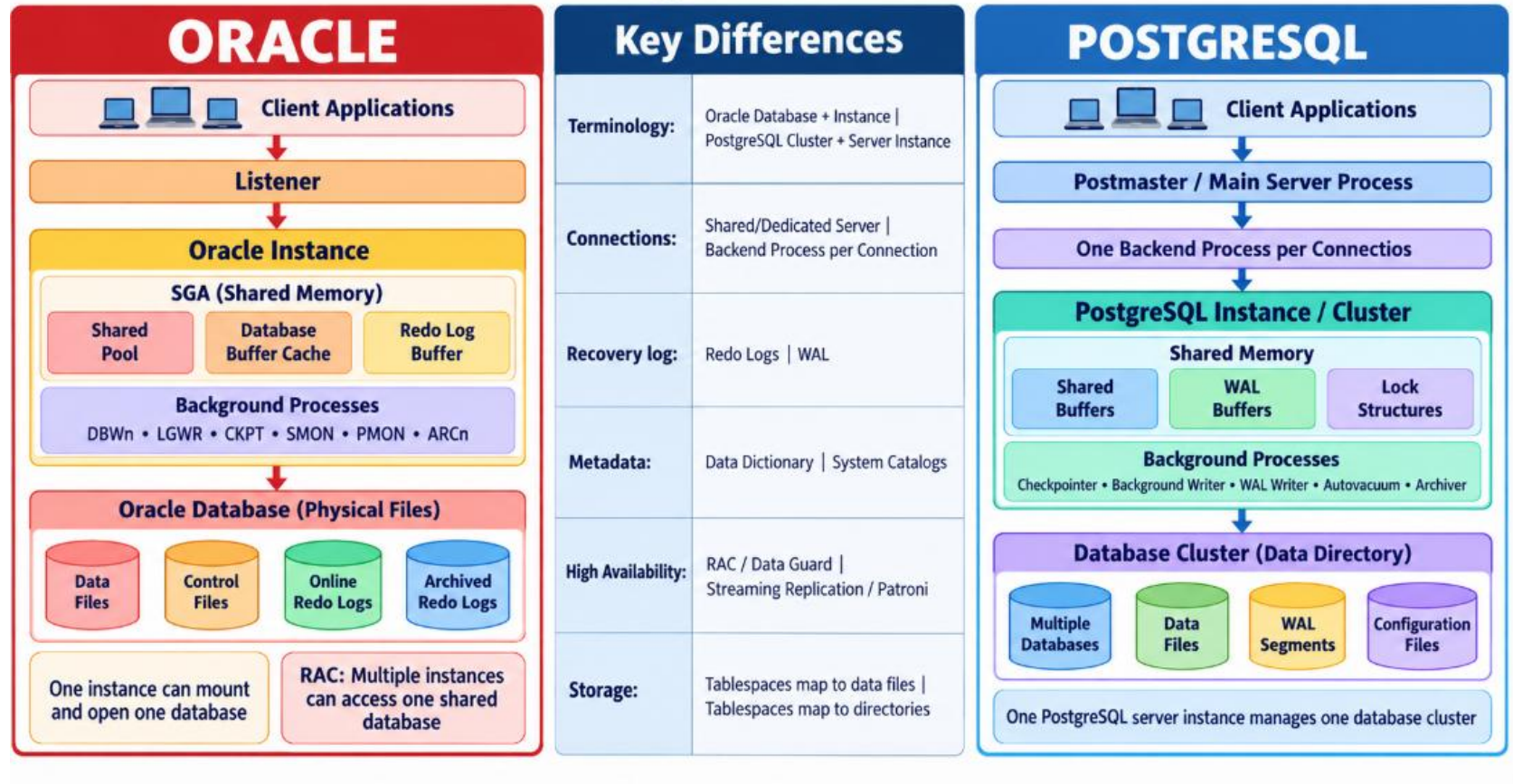
Configuration under Patroni

- For a Patroni-managed cluster, do not independently change important cluster-wide parameters on one node.
- Patroni typically manages settings such as:
- postgresql:
 - parameters:
 - max_connections: 100
 - shared_buffers: 1GB
 - wal_level: replica
 - max_wal_senders: 10
 - max_replication_slots: 10

Configuration under Patroni

- Cluster-wide settings should normally be changed through:
 - `/opt/patroni/bin/patronictl edit-config`
- Patroni then coordinates the configuration across the nodes.
- Use PostgreSQL files primarily for understanding and troubleshooting, but allow Patroni to manage replication and HA-critical parameters consistently.

Oracle vs Postgre Architecture



Oracle vs Postgre Architecture

- **Architecture terminology**

- Oracle: **Database + Instance**
- PostgreSQL: **Database Cluster + Server Instance**

- **Meaning of database**

- Oracle database represents the physical files stored on disk.
- A PostgreSQL database is one logical database inside a database cluster.

- **Multiple databases**

- An Oracle instance normally opens one Oracle database.
- One PostgreSQL instance manages a cluster containing multiple databases.

- **Connection handling**

- Oracle supports dedicated and shared server processes.
- PostgreSQL normally creates one backend process for every client connection.

Oracle vs Postgre Architecture

- **Main memory**

- Oracle uses the **System Global Area (SGA)**.
- PostgreSQL uses shared memory containing shared buffers, WAL buffers and lock structures.

- **Session memory**

- Oracle uses the **Program Global Area (PGA)**.
- PostgreSQL uses backend-local memory such as `work_mem`, `maintenance_work_mem` and `temp_buffers`.

Oracle vs Postgre Architecture

- **Data caching**

- Oracle caches data blocks in the **Database Buffer Cache**.
- PostgreSQL caches pages in **Shared Buffers** and also benefits from the operating-system cache.

- **Transaction logging**

- Oracle records changes in the redo log buffer and online redo logs.
- PostgreSQL records changes in WAL buffers and WAL segment files.

- **Recovery terminology**

- Oracle: **Redo logs**
- PostgreSQL: **Write-Ahead Log (WAL)**

Oracle vs Postgre Architecture

- **Main connection process**

- Oracle clients normally connect through the Oracle Listener.
- PostgreSQL connections are accepted by the postmaster/main server process.

- **Background processes**

- Oracle uses DBWn, LGWR, CKPT, SMON, PMON and ARCn.
- PostgreSQL uses the checkpoint, background writer, WAL writer, autovacuum launcher/workers and archiver.

- **Data writer**

- Oracle DBWn writes dirty buffers to data files.
- PostgreSQL's background writer and backend processes write dirty buffers; the checkpoint ensures required pages are written at checkpoints.

Oracle vs Postgre Architecture

- **Storage structure**

- Oracle uses data files, control files, online redo logs and archived redo logs.
- PostgreSQL uses a data directory containing relation files, WAL, configuration files and internal metadata.

- **Control files**

- Oracle has dedicated control files containing critical database structure and checkpoint information.
- PostgreSQL has no direct equivalent to Oracle control files; metadata is distributed across pg_control, system catalogs and the data directory.

- **Tablespaces**

- Oracle tablespaces are logical storage units backed by one or more data files.
- PostgreSQL tablespaces map logical names to operating-system directories.

Oracle vs Postgre Architecture

- **Metadata**

- Oracle stores metadata in the data dictionary.
- PostgreSQL stores metadata in system catalogs such as `pg_class`, `pg_database` and `pg_roles`.

- **Process recovery**

- Oracle SMON performs instance recovery and system-level cleanup.
- PostgreSQL performs crash recovery by replaying WAL when the server starts.

- **Failed-session cleanup**

- Oracle PMON cleans up failed user processes.
- PostgreSQL's operating system and main server process handle terminated backend cleanup.

- **Vacuum requirement**

- Oracle uses undo information and automatic undo management.
- PostgreSQL uses MVCC row versions and requires VACUUM/autovacuum to reclaim dead tuples.

Oracle vs Postgre Architecture

- **High availability**

- Oracle commonly uses RAC and Data Guard.
- PostgreSQL commonly uses streaming replication with Patroni, repmgr or similar tools.

- **Shared-database clustering**

- Oracle RAC allows multiple instances to access the same shared database.
- Standard PostgreSQL replication uses separate data copies; multiple PostgreSQL instances do not simultaneously modify the same data directory.

- **Built-in HA management**

- Oracle provides integrated enterprise HA technologies.
- PostgreSQL commonly combines PostgreSQL replication with external components such as Patroni, etcd, HAProxy and PgBouncer.

Oracle vs Postgre Architecture

- **Licensing**

- Oracle Database is primarily commercial and many advanced capabilities require additional licenses.
- PostgreSQL is open source under the PostgreSQL License.

- **One-line summary**

- Oracle centres on an instance opening physical database files, while PostgreSQL centres on one server instance managing a database cluster containing multiple logical databases.

Postgre Architecture

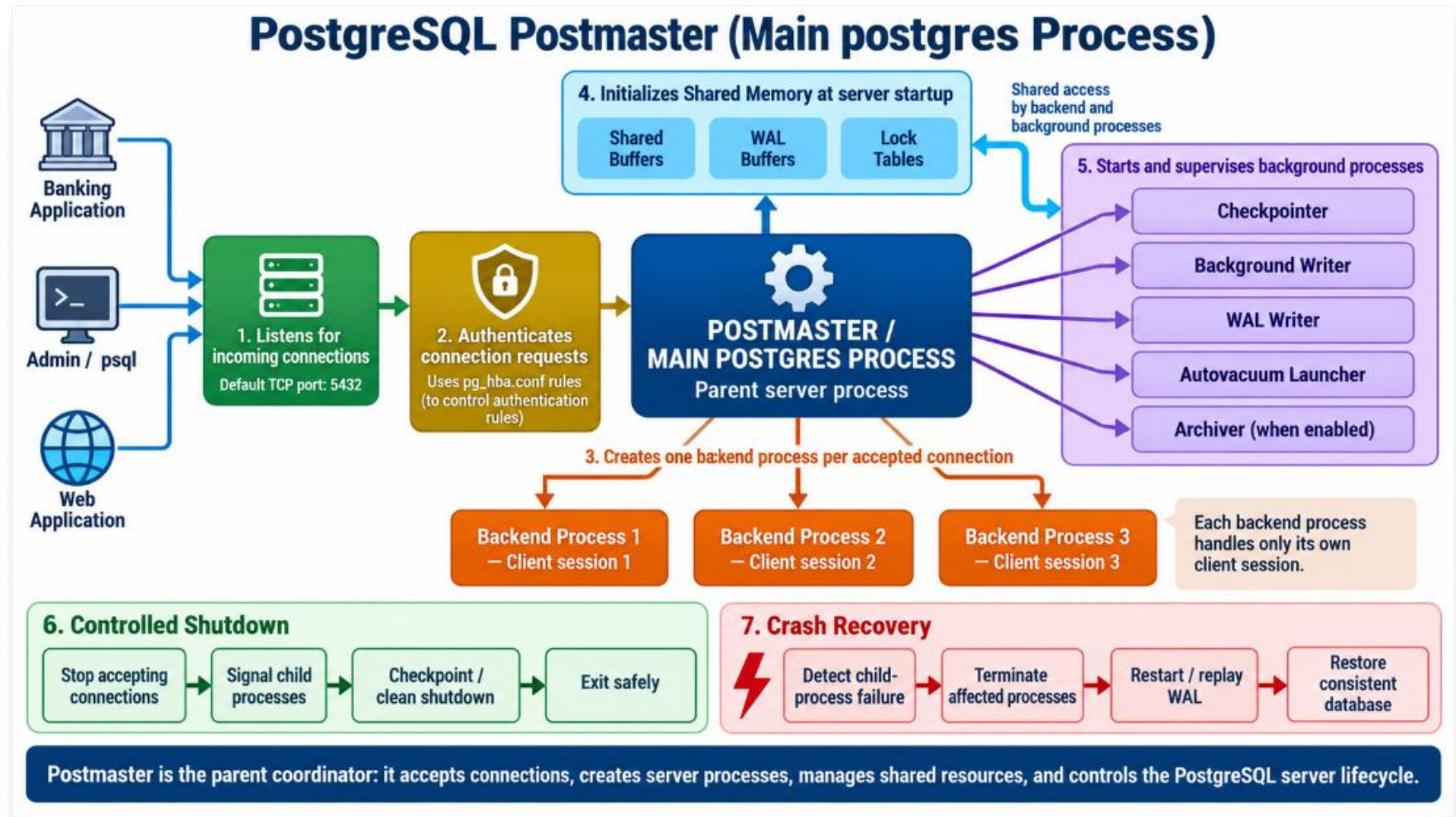
ps -ef | grep '[p]ostgres'

```
ubuntu@ip-172-31-40-213:~$ ps -ef | grep '[p]ostgres'
postgres  931      1  0 01:10 ?        00:00:00 /usr/lib/postgresql/18/bin/postgres -D /var/lib/postgresql/18/training -c
config_file=/etc/postgresql/18/training/postgresql.conf
postgres  932      1  0 01:10 ?        00:00:00 /usr/lib/postgresql/18/bin/postgres -D /var/lib/postgresql/18/main -c conf
ig_file=/etc/postgresql/18/main/postgresql.conf
postgres  982      932  0 01:10 ?        00:00:00 postgres: 18/main: logger
postgres  991      932  0 01:10 ?        00:00:00 postgres: 18/main: io worker 0
postgres  996      932  0 01:10 ?        00:00:00 postgres: 18/main: io worker 1
postgres  998      931  0 01:10 ?        00:00:00 postgres: 18/training: io worker 0
postgres  999      932  0 01:10 ?        00:00:00 postgres: 18/main: io worker 2
postgres 1000      932  0 01:10 ?        00:00:00 postgres: 18/main: checkpointer
postgres 1001      931  0 01:10 ?        00:00:00 postgres: 18/training: io worker 2
postgres 1002      932  0 01:10 ?        00:00:00 postgres: 18/main: background writer
postgres 1003      931  0 01:10 ?        00:00:00 postgres: 18/training: io worker 1
postgres 1004      931  0 01:10 ?        00:00:00 postgres: 18/training: checkpointer
postgres 1005      931  0 01:10 ?        00:00:00 postgres: 18/training: background writer
postgres 1021      932  0 01:10 ?        00:00:00 postgres: 18/main: walwriter
postgres 1023      932  0 01:10 ?        00:00:00 postgres: 18/main: autovacuum launcher
postgres 1024      932  0 01:10 ?        00:00:00 postgres: 18/main: logical replication launcher
postgres 1025      931  0 01:10 ?        00:00:00 postgres: 18/training: walwriter
postgres 1026      931  0 01:10 ?        00:00:00 postgres: 18/training: autovacuum launcher
postgres 1027      931  0 01:10 ?        00:00:00 postgres: 18/training: logical replication launcher
ubuntu@ip-172-31-40-213:~$
```


Postgre Architecture

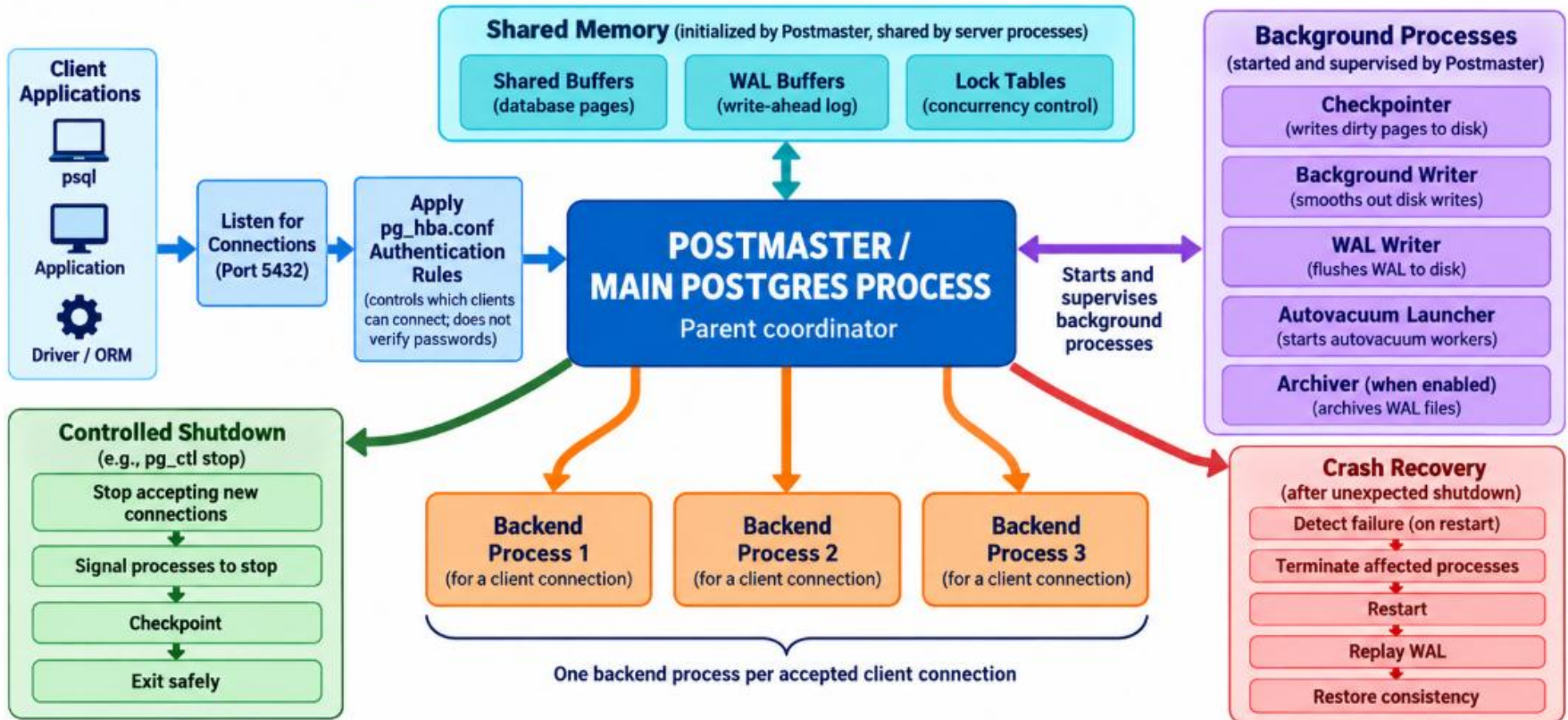
- **1. Client applications**
- Clients connect to PostgreSQL, normally through TCP port 5432.
- psql – command-line client
- JDBC – Java connectivity
- libpq – native PostgreSQL client library
- Applications – Spring Boot, FastAPI, .NET, Node.js, etc.

Postgre Architecture



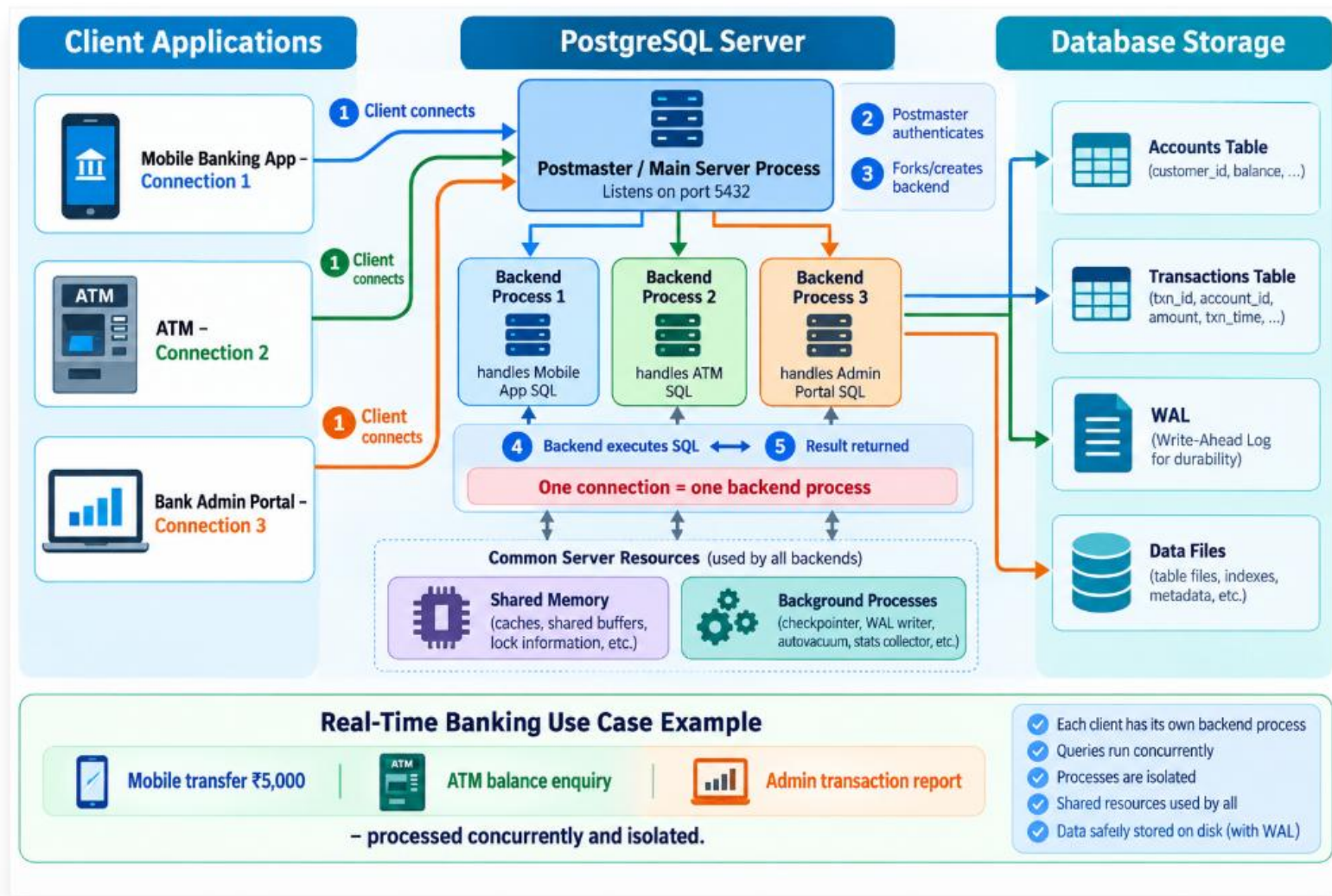
Postgre Architecture

PostgreSQL Postmaster (Main postgres Process)



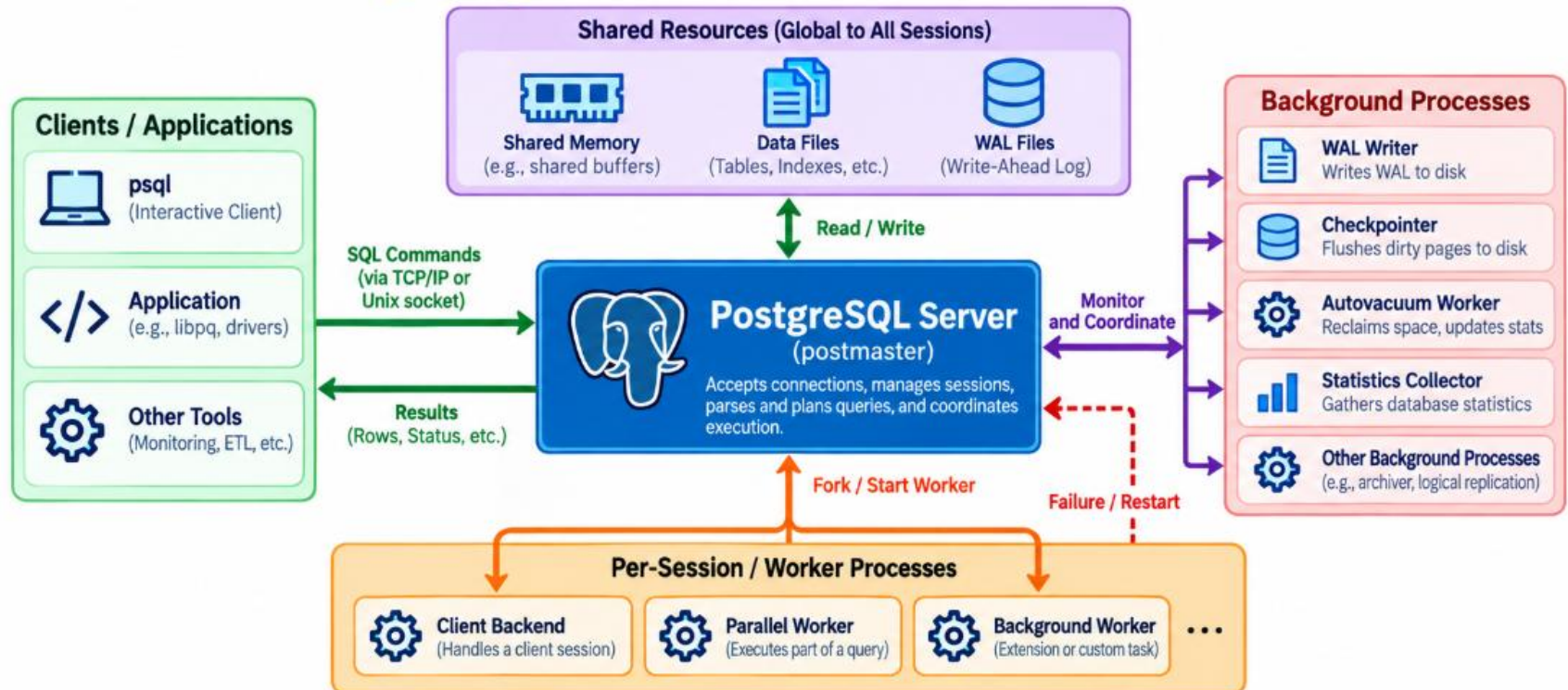
The Postmaster accepts connections, creates backend processes, initializes shared resources, supervises background processes, and controls the server lifecycle.

Backend process for every connection



Postgre Architecture

PostgreSQL Architecture and Process Flow

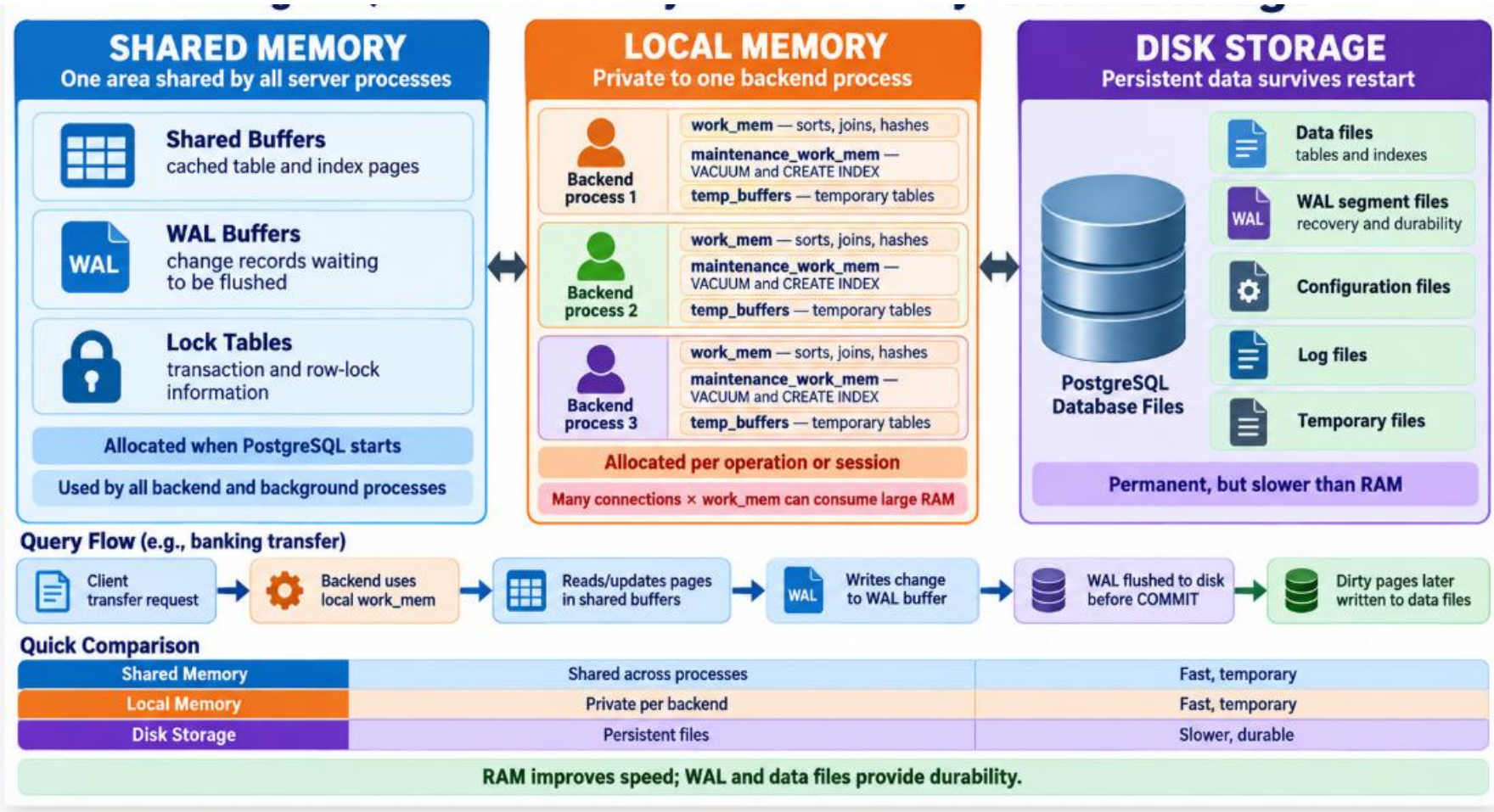


Clients send SQL commands → PostgreSQL server coordinates execution using shared resources → Worker and background processes perform the work → Results are returned to clients.

Shared Memory vs Local Memory vs Disk Storage

Requirement	PostgreSQL area
Data frequently reused by many sessions	Shared memory
Query-specific sorting or hashing	Local memory
Session-specific temporary tables	Local temp_buffers
Permanent tables and indexes	Disk
Transaction durability and recovery	WAL on disk
Large sort that exceeds work_mem	Temporary disk
Shared transaction locks	Shared memory
Large index creation	Local maintenance_work_mem, then disk

Shared Memory vs Local Memory vs Disk Storage



Shared Memory vs Local Memory vs Disk Storage

Connection	Operation	Main resource used
Mobile application	Check balance	Shared buffers
ATM	Withdraw ₹10,000	Shared buffers, WAL buffers and disk
Admin portal	Generate daily report	Local work_mem, possibly temporary disk

Postgre Architecture

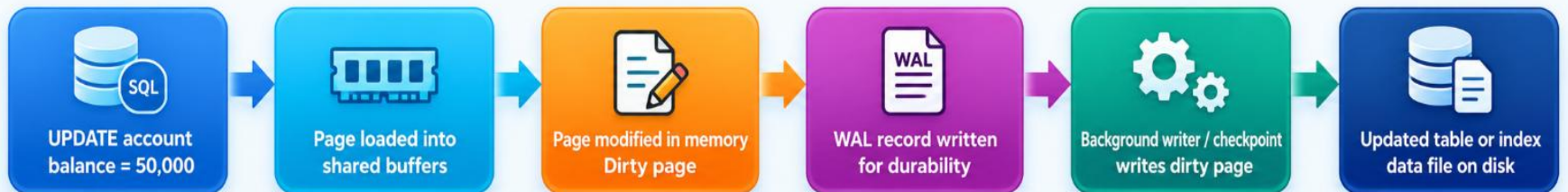
• 4. Shared Memory

Component	Purpose
Shared Buffers	Caches table and index pages
WAL Buffers	Temporarily holds WAL records before disk writes
Lock Tables	Coordinates row, table and transaction locks
Process Array	Tracks active processes, transactions and snapshots

Postgre Architecture

- A **dirty page** is a database page in PostgreSQL's shared memory that has been modified but has not yet been written to the table or index file on disk.

PostgreSQL Dirty Page Flow



Postgre Architecture

- Suppose PostgreSQL executes:
 - UPDATE accounts SET balance = 50000 WHERE account_id = 101;
- PostgreSQL:
 - Loads the required data page from disk into **shared buffers**.
 - Changes the balance inside the memory page.
 - Marks that memory page as **dirty**.
 - Generates a corresponding **WAL record**.

Postgre Architecture

- Flushes the WAL record before confirming the committed transaction.
- Flush means Write pending WAL records from volatile memory to durable storage and synchronize them
- Later writes the dirty page from shared buffers to its table data file.

Postgre Architecture

- **5. Background processes**

Process	Function
Background Writer	Gradually writes dirty shared-buffer pages to data files
Checkpointer	Coordinates checkpoints and ensures required dirty pages reach disk
WAL Writer	Flushes WAL-buffer contents into WAL segment files
Autovacuum	Reclaims reusable space and updates optimizer statistics
Archiver	Copies completed WAL segments to an archive destination
Statistics facilities	Collect activity and performance information

Postgre Architecture

- **5. Background processes**
- In modern PostgreSQL versions, the old dedicated **statistics collector process** was removed.
- Statistics are now maintained through shared-memory-based infrastructure, although the architecture diagram remains useful conceptually.

Postgre Architecture

- **6. Disk storage — \$PGDATA**
 - \$PGDATA is the PostgreSQL database-cluster directory.
- It contains or references:
 - **Data files** – table and index pages
 - **WAL segments** – transaction-change records
 - **Tablespaces** – links to storage locations outside the main data directory
 - **Visibility maps** – identify pages containing tuples visible to all transactions
 - Free-space maps, transaction status data and configuration files

Postgre Architecture

- **7. Write operation flow**
- For an UPDATE, PostgreSQL follows this simplified sequence:
 - The backend receives and executes the SQL.
 - PostgreSQL creates a new row version using MVCC.
 - The changed page becomes dirty in shared buffers.
 - A corresponding record is placed in WAL buffers.
 - WAL is flushed to disk when the transaction commits.
 - The modified data page is written later by the background writer or checkpoint.

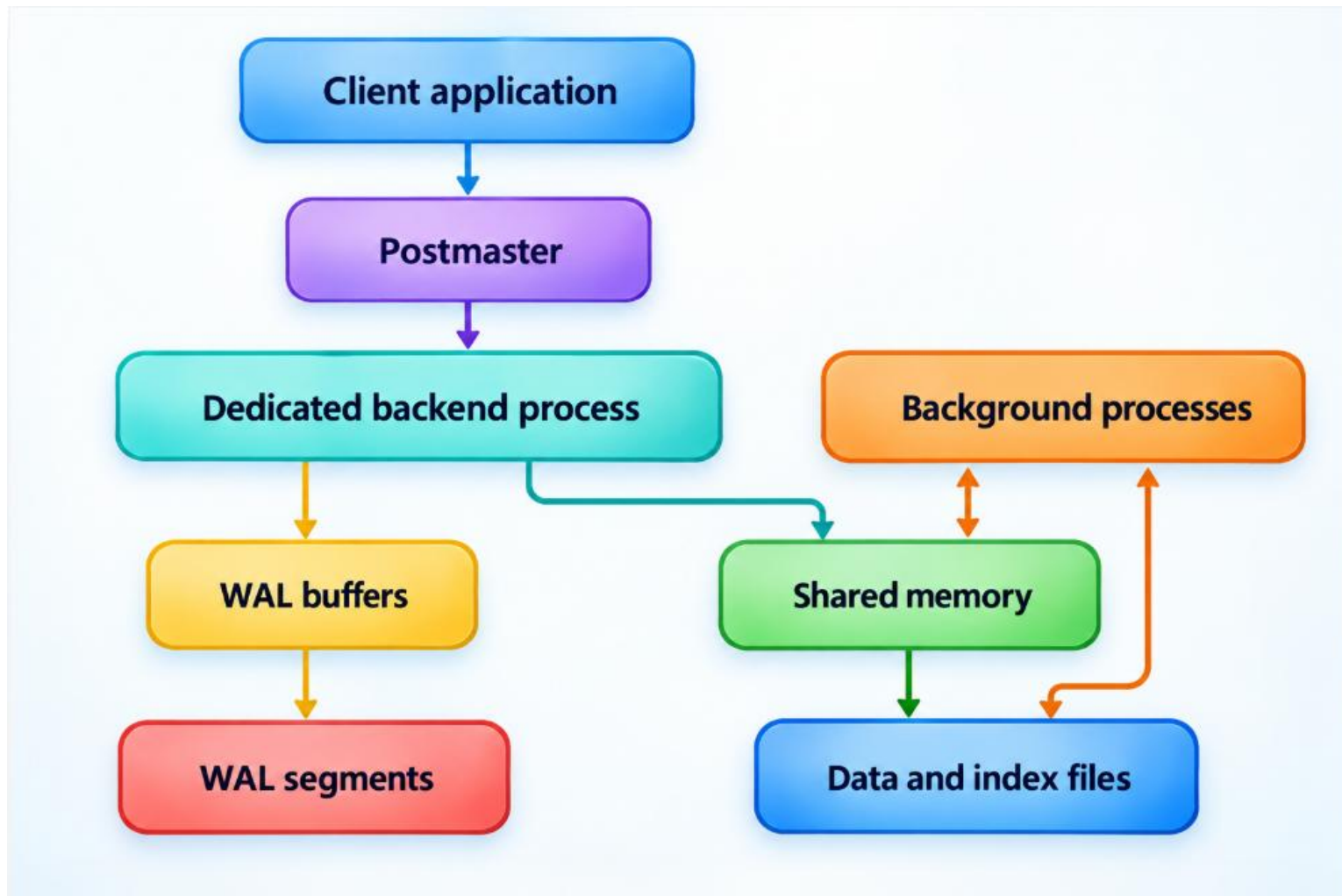
Postgre Architecture

- **7. Write operation flow**
- The essential durability rule is:
 - The WAL record must reach durable storage before the related data page is written.
 - This is why PostgreSQL calls it **Write-Ahead Logging**.

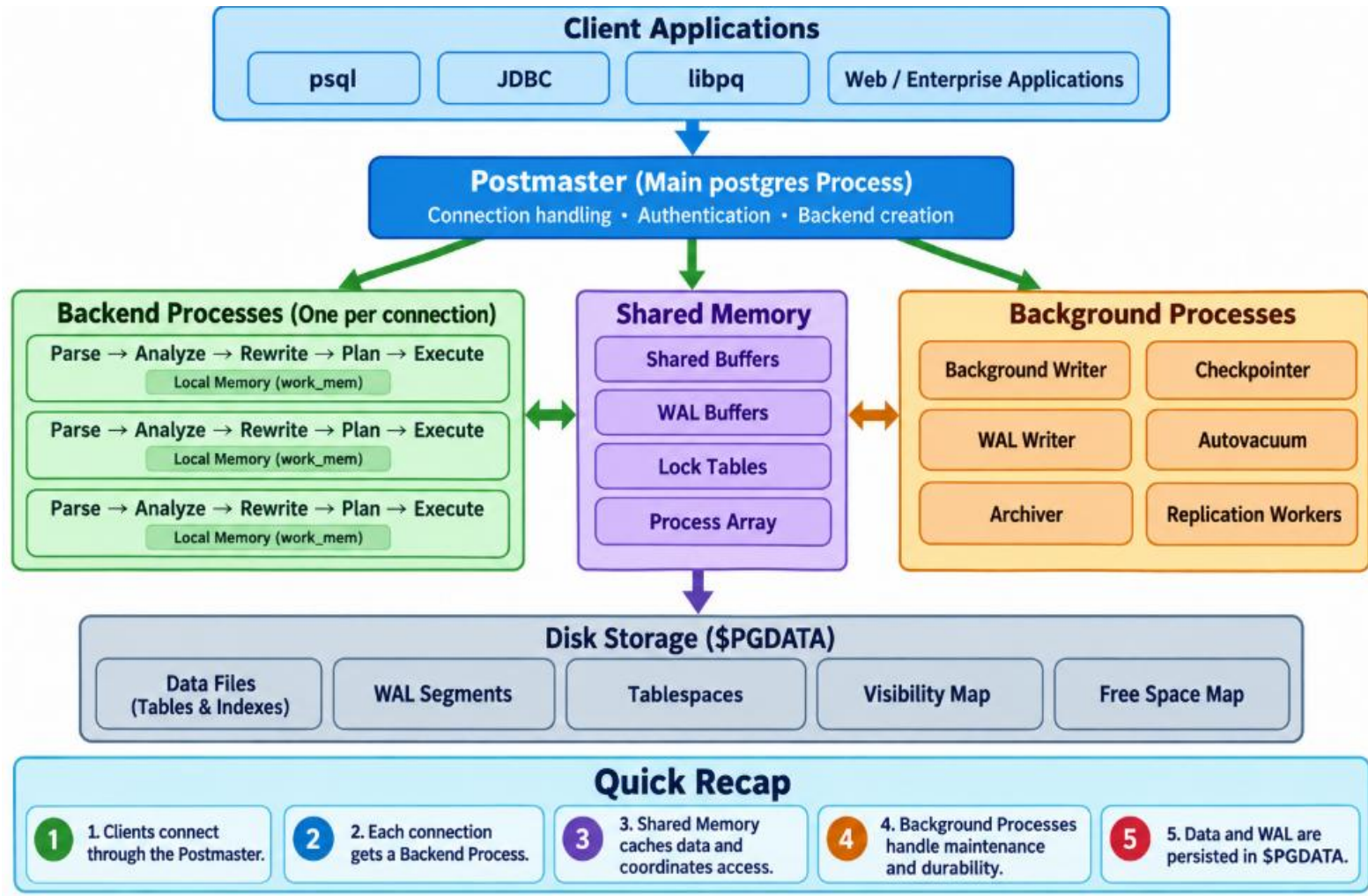
Postgre Architecture

- **8. Read operation flow**
- For a SELECT:
 - The backend parses and plans the query.
 - It searches for the required page in shared buffers.
 - If absent, PostgreSQL reads the page from disk into shared buffers.
 - MVCC visibility rules determine which row versions the transaction can see.
 - The backend returns the result to the client.

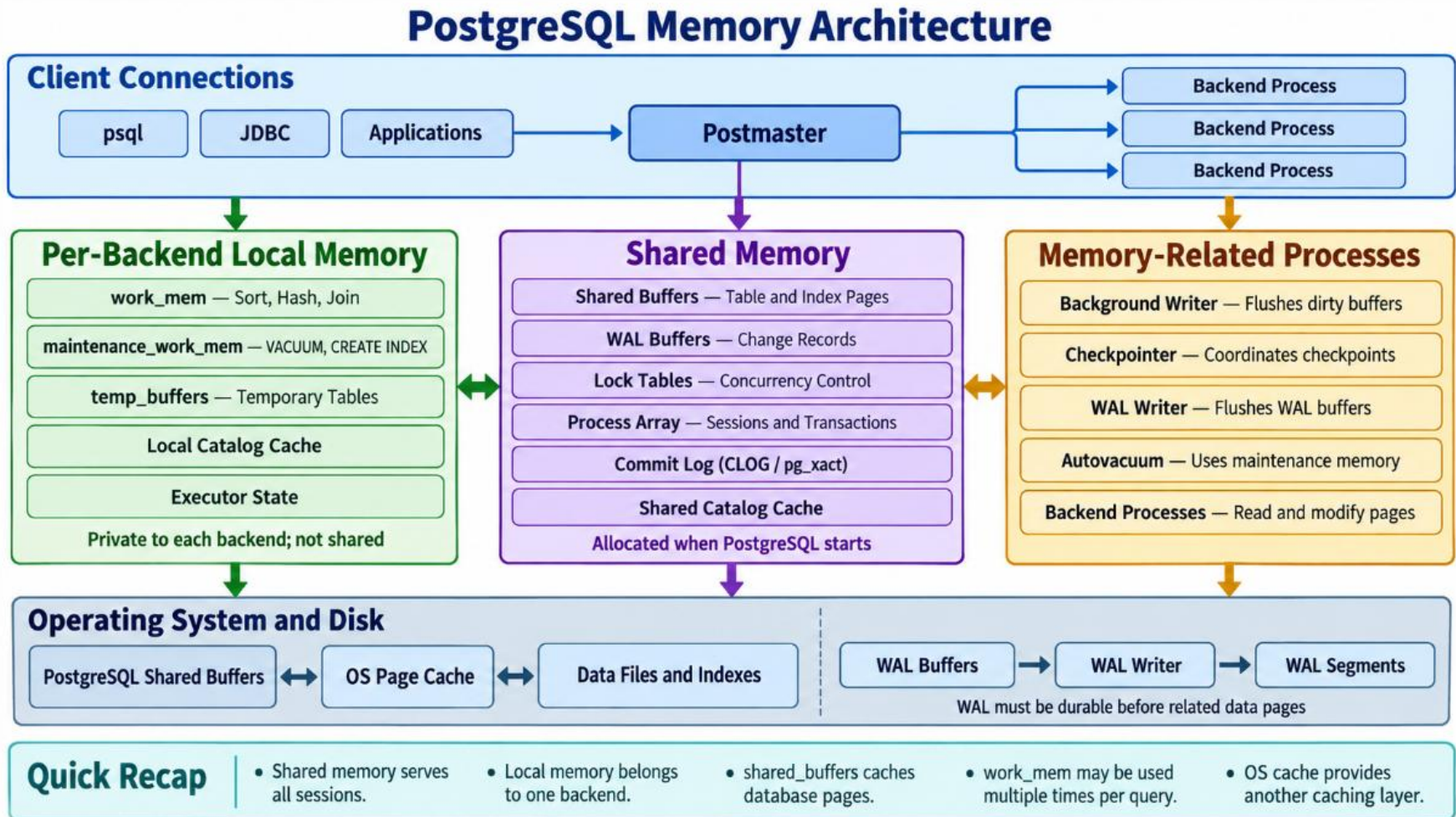
Postgre Architecture - Overall data flow



PostgreSQL Components



Memory Architecture



Memory Architecture

- **1. Shared memory**
- PostgreSQL allocates shared memory when the database server starts. All backend and relevant background processes can access it.
- **Shared buffers**
 - shared_buffers is PostgreSQL's primary database-page cache.
- It stores recently accessed:
 - Table pages
 - Index pages
 - System-catalog pages
 - Modified pages waiting to be written

Memory Architecture

- When PostgreSQL needs a page:
 - It checks shared buffers.
 - If the page is present, it reads it from memory.
 - Otherwise, it obtains the page through the operating system and places it in shared buffers.
 - Modified pages become **dirty buffers**.
 - They are written later by backend processes, the background writer or checkpoint processing

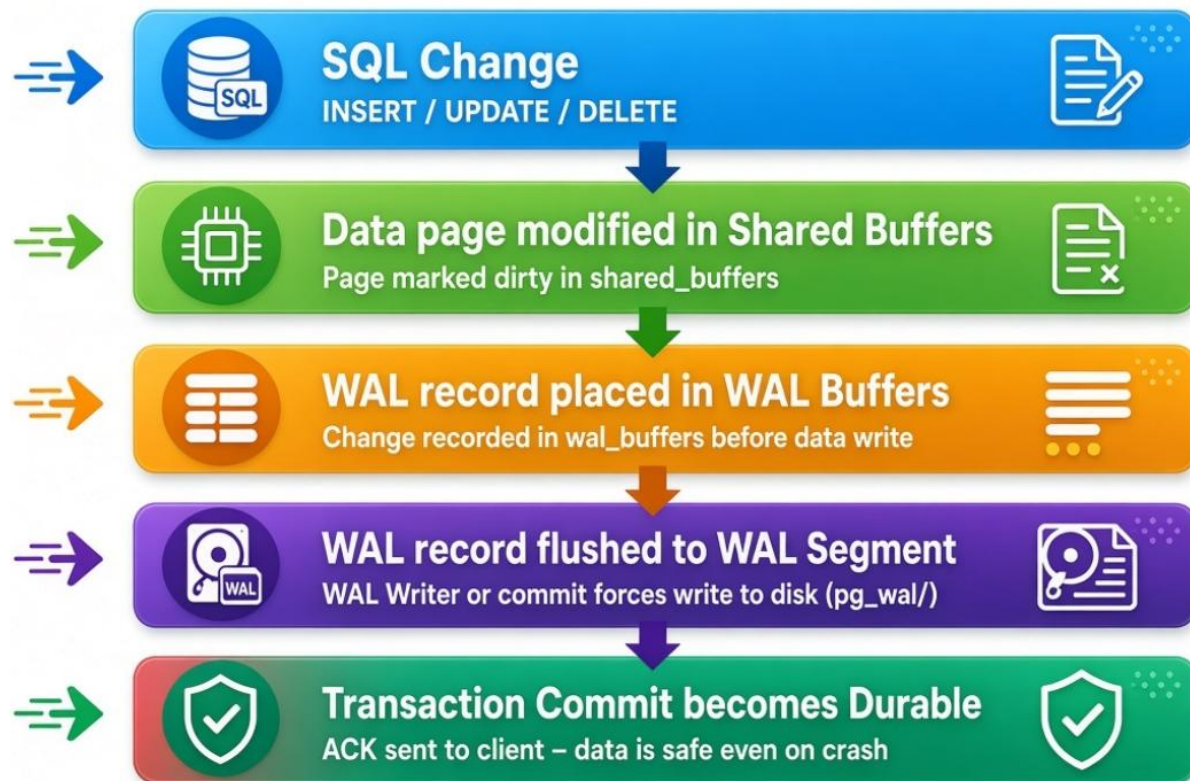
Memory Architecture

- Typical starting configuration:
 - `shared_buffers` = 25% of system RAM
- This is a starting guideline, not a universal rule. Workload testing is required.

WAL buffers

- wal_buffers holds Write-Ahead Log records temporarily before they are written to WAL segment files.

PostgreSQL WAL Write Path – Making a Transaction Durable



WAL buffers

- Important rule:
 - WAL must reach durable storage before its related dirty data page can be written.
- Example configuration:
 - `wal_buffers = -1`
- -1 allows PostgreSQL to determine the value automatically, usually based on `shared_buffers`.

PostgreSQL shared lock structures

- PostgreSQL keeps lock-management information in **shared memory** because all backend processes must see who holds a lock, who is waiting, and which operations conflict.
- Shared lock structures do **not normally contain the table or row data**. They contain coordination metadata such as:
 - Locked resource identity
 - Lock mode
 - Owning transaction or backend
 - Granted or waiting status
 - Waiting-process queue
- This enables PostgreSQL to prevent conflicting operations, detect deadlocks, and wake waiting sessions when locks are released.

1. Table locks

- Table-level locks protect an entire table against incompatible operations.

Lock mode	Typical operation	Main effect
ACCESS SHARE	SELECT	Allows normal concurrent access
ROW SHARE	SELECT FOR UPDATE	Indicates rows may be locked
ROW EXCLUSIVE	INSERT, UPDATE, DELETE	Allows concurrent DML but conflicts with some maintenance operations
SHARE UPDATE EXCLUSIVE	VACUUM, some ALTER TABLE operations	Protects maintenance activity
SHARE	CREATE INDEX	Blocks data-changing operations
ACCESS EXCLUSIVE	DROP TABLE, TRUNCATE, many ALTER TABLE operations	Blocks every other table-lock mode

2. Row-related transaction locks

- When PostgreSQL updates or explicitly locks a row, information is primarily recorded in the row version itself using transaction IDs.
- For example:
- BEGIN;
- `SELECT * FROM accounts WHERE account_id = 101
FOR UPDATE;`

2. Row-related transaction locks

- A second transaction attempting to update that same row must wait:
- UPDATE accounts SET balance = balance - 1000
- WHERE account_id = 101;

Transaction A locks row 101



Row header records Transaction A's identity



Transaction B tries to update row 101



Transaction B waits for Transaction A to finish

2. Row-related transaction locks

- A second transaction attempting to update that same row must wait:
- `UPDATE accounts SET balance = balance - 1000`
- `WHERE account_id = 101;`
- Row-lock information is stored in tuple headers.
- Waiting sessions use transaction-ID locks in the shared lock manager.
- Multiple row lockers may use a MultiXact.
- This is why `max_locks_per_transaction` does not directly limit the number of rows a transaction can lock.

3. Transaction locks

- Every active transaction holds an internal lock associated with its transaction ID or virtual transaction ID.
- Suppose Transaction B encounters a row modified by Transaction A.
- Transaction B can wait for Transaction A's transaction lock:

3. Transaction locks

Transaction A

updates account row



holds transaction-ID lock



Transaction B detects A's transaction ID



B waits for A to COMMIT or ROLLBACK

3. Transaction locks

- After Transaction A finishes:
 - If A commits, B evaluates the committed row version.
 - If A rolls back, B can proceed using the original valid version.
- Transaction locks therefore connect row-level concurrency with transaction completion.

4. Advisory locks

- Advisory locks are **application-defined locks**. PostgreSQL does not automatically associate them with a table or row.
- They are useful for coordinating business operations.
- **Banking example**
- Prevent two workers from processing the same payment:
 - `SELECT pg_advisory_xact_lock(50001);`

4. Advisory locks

- Here, 50001 might represent a payment or customer identifier.
- Two scopes are available:
- **Transaction-level advisory lock**
 - `SELECT pg_advisory_xact_lock(50001);`
- Automatically released on COMMIT or ROLLBACK.

4. Advisory locks

- **Session-level advisory lock**
- `SELECT pg_advisory_lock(50001);`
- `-- Perform work`
- `SELECT pg_advisory_unlock(50001);`
- It remains held until explicitly unlocked or the session terminates.
- Advisory locks work correctly only when every participating application follows the same locking convention.

5. Predicate locks

- Predicate locks are used by transactions running at the `SERIALIZABLE` isolation level.
- They do not block operations in the same way as regular locks.
- Instead, they track read dependencies so PostgreSQL can detect whether concurrent transactions would produce a result impossible under serial execution.

5. Predicate locks

- Example:
- BEGIN TRANSACTION ISOLATION LEVEL
SERIALIZABLE;
- SELECT SUM(balance) FROM accounts WHERE
branch_id = 10;

5. Predicate locks

- The application must retry that transaction.
- Predicate locks can exist at different granularities:
 - Tuple
 - Page
 - Relation
- When many fine-grained predicate locks accumulate, PostgreSQL may promote them to a coarser page- or relation-level predicate lock.

5. Predicate locks

- **max_locks_per_transaction = 64**
- This setting helps PostgreSQL calculate the size of the regular shared lock table.
- `max_locks_per_transaction = 64`
- It means PostgreSQL reserves capacity based on approximately:
 - `max_locks_per_transaction`
 - $\times (\text{max_connections} + \text{max_prepared_transactions})$
- The actual calculation includes additional internal considerations, so treat this as a planning approximation.

5. Predicate locks

- **What counts toward it?**
- It mainly affects the number of **distinct lockable objects**, such as:
 - Tables
 - Indexes
 - Sequences
 - Partitions
 - Other relations and database objects
 - Advisory-lock identifiers
- It does not mean:
 - A transaction can lock only 64 rows.
- A transaction can update or lock far more than 64 rows because row locks are normally stored in tuple headers rather than as individual regular lock-table entries.

5. Predicate locks

- **Example involving partitions**
- A transaction accesses:
 - 40 table partitions
 - 40 indexes
 - Several sequences
- It may need locks on more than 64 distinct objects. If the shared lock table becomes exhausted, PostgreSQL can report:
 - ERROR: out of shared memory
 - HINT: You might need to increase `max_locks_per_transaction`.
- The value is an average capacity calculation, not a strict per-transaction ceiling.
- One transaction can exceed 64 locks if other transactions are using fewer locks and shared capacity remains.

Comparison

Lock category	Purpose	Normally blocks?	Storage/tracking
Table lock	Protect relations and coordinate operations	Yes, when modes conflict	Shared lock table
Row lock	Protect a row from conflicting changes	Yes	Primarily tuple header
Transaction lock	Wait for another transaction to finish	Yes	Shared lock manager
Advisory lock	Application-defined coordination	Yes	Shared lock table
Predicate lock	Detect unsafe serializable dependencies	No direct blocking	Predicate-lock shared memory

Process array

- The process array stores information about active backend processes and transactions. It supports:
 - Transaction snapshots
 - MVCC visibility decisions
 - Tracking active transaction IDs
 - Detecting transactions that are still running

Transaction status data

- PostgreSQL maintains transaction status information indicating whether transactions are:
 - In progress
 - Committed
 - Aborted
- The pg_xact area is commonly associated with this information. Historically, it was called pg_clog.

Other shared-memory structures

- Shared memory also contains structures supporting:
 - Buffer locks and lightweight locks
 - Replication
 - Statistics
 - Background workers
 - Parallel queries
 - Two-phase commits
 - Notifications and process coordination

2. Per-backend local memory

- Every client connection usually receives a dedicated backend process. Each backend has its own private memory.
- This memory is not shared with other connections.
- **work_mem**
- work_mem limits memory for an individual query operation such as:
 - Sort
 - Hash join
 - Hash aggregation
 - Bitmap operation
 - Materialization

2. Per-backend local memory

- `work_mem` = 16MB
- A frequent misunderstanding is that `work_mem` is allocated once per connection.
- It can instead be allocated **once for every eligible operation** in a query.
- For example:
 - One query
 - |—— Sort: up to 16 MB
 - |—— Hash join: up to 16 MB
 - |—— Hash aggregate: up to 16 MB
- Parallel workers can multiply this consumption further.
- A rough risk calculation is:
 - $\text{Memory} \approx \text{active sessions} \times \text{operations per query} \times \text{work_mem}$
- Therefore, setting `work_mem` too high globally can exhaust server memory.

2. Per-backend local memory

- If an operation exceeds the available `work_mem`, PostgreSQL may create temporary files on disk.
- Check temporary-file usage with:
 - `SELECT datname, temp_files, temp_bytes FROM pg_stat_database;`
- **`maintenance_work_mem`**
- This is used by maintenance operations such as:
 - `VACUUM`
 - `CREATE INDEX`
 - `ALTER TABLE ADD FOREIGN KEY`

2. Per-backend local memory

- `maintenance_work_mem` = 512MB
- It can normally be larger than `work_mem` because fewer maintenance operations run simultaneously.
- Autovacuum uses a separate limit:
 - `autovacuum_work_mem` = -1
- With -1, it uses `maintenance_work_mem`. Remember that multiple autovacuum workers can operate concurrently.

2. Per-backend local memory

- **temp_buffers**

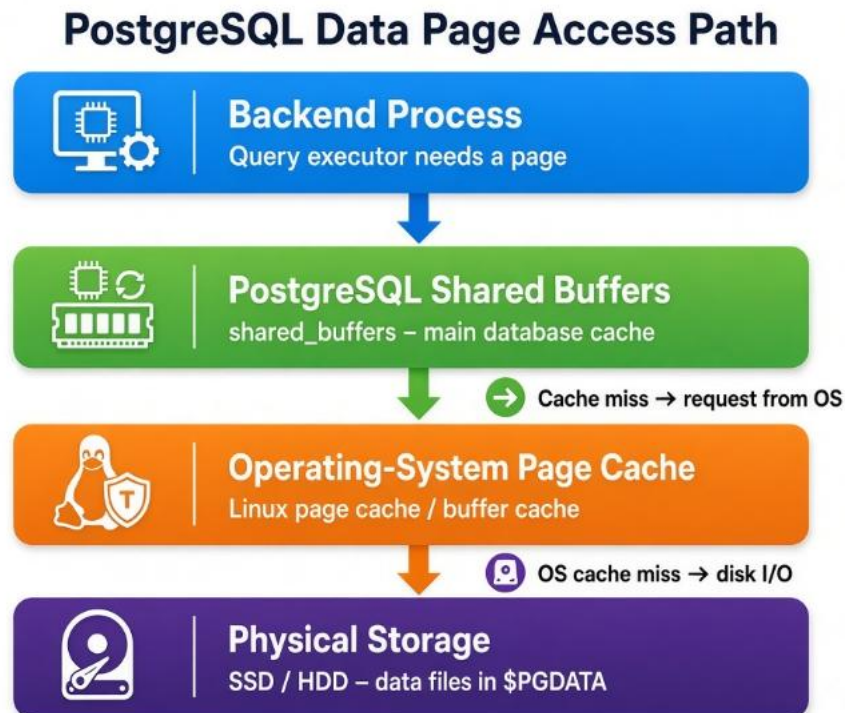
- temp_buffers controls the maximum session-local buffer space used for temporary tables.
- temp_buffers = 8MB
- It applies only to temporary-table pages, not to sort or hash operations.

2. Per-backend local memory

- **Executor and session memory**
- A backend also consumes memory for:
 - Query parsing
 - Query plans
 - Executor state
 - Connection state
 - Catalog caches
 - Prepared statements
 - PL/pgSQL execution
 - Extension-specific data
- Some of this memory exists only during one query; some can remain for the session.

3. Operating-system page cache

- PostgreSQL uses both:
- Its own `shared_buffers`
- The operating system's filesystem cache



3. Operating-system page cache

- Consequently, assigning almost all RAM to shared_buffers is usually undesirable.
- Memory must remain available for:
 - OS page cache
 - Backend processes
 - Query operations
 - Background processes
 - System services
- The planner setting below estimates how much data is likely cached across PostgreSQL and the operating system:
 - effective_cache_size = 12GB
- effective_cache_size does **not allocate memory**. It is only a planner estimate used when comparing plans such as index scans and sequential scans.

4. Memory-related processes

Process	Relationship to memory
Backend process	Reads and modifies shared-buffer pages and uses private local memory
Background writer	Gradually writes dirty shared-buffer pages
Checkpointner	Ensures required dirty pages are written during checkpoints
WAL writer	Flushes WAL buffers to WAL segment files
Autovacuum worker	Uses maintenance memory while vacuuming and analyzing
Parallel worker	Uses shared structures and additional process-local memory

5. Read operation

- Consider:
- `SELECT * FROM vehicles WHERE vehicle_id = 'BMW1001';`
- The simplified flow is:
 - A backend receives and plans the query.
 - PostgreSQL searches shared buffers for the required index and table pages.
 - If found, it uses the cached pages.
 - If absent, it reads them through the OS page cache.
 - The pages are added to shared buffers.
 - MVCC determines which row version is visible.
- The result is returned to the client.

6. Write operation

- Consider:
- `UPDATE vehicles SET battery_level = 90 WHERE vehicle_id = 'BMW1001';`
- The simplified flow is:
 - The backend creates a new row version.
 - Relevant pages in shared buffers are modified and marked dirty.
 - WAL records are placed in WAL buffers.
 - At commit, the WAL records are flushed to durable storage.
 - Dirty table and index pages are written later.
 - The old tuple version is eventually processed by vacuum.

Summary

Parameter	Purpose
shared_buffers	PostgreSQL page cache
wal_buffers	Memory for pending WAL records
work_mem	Memory per eligible query operation
maintenance_work_mem	Memory for maintenance operations
autovacuum_work_mem	Memory per autovacuum worker
temp_buffers	Session memory for temporary tables
effective_cache_size	Planner estimate—not allocated memory
huge_pages	Controls use of operating-system huge pages
max_connections	Influences possible backend and local-memory usage

Summary

Area	Scope	Main purpose
Shared buffers	Entire PostgreSQL instance	Cache table and index pages
WAL buffers	Entire instance	Hold pending WAL records
Lock/process structures	Entire instance	Concurrency and transaction coordination
work_mem	Per query operation	Sorts, joins and aggregations
maintenance_work_mem	Per maintenance operation	Vacuum and index creation
temp_buffers	Per session	Temporary-table pages
OS page cache	Entire operating system	Cache filesystem data

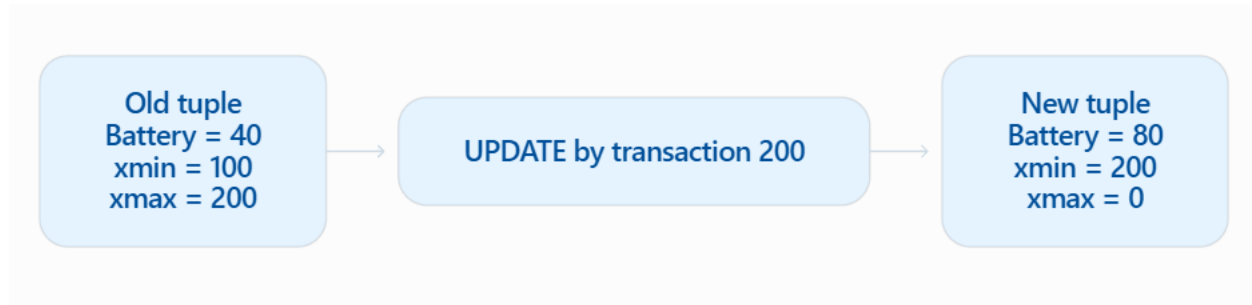
MVCC

- **MVCC—Multi-Version Concurrency Control** allows multiple transactions to read and modify data concurrently without readers unnecessarily blocking writers.
- Instead of overwriting a row directly, PostgreSQL keeps multiple physical versions of that row.

MVCC

- **1. Basic concept**
- Suppose the current row is:
 - Vehicle: BMW1001
 - Battery level: 40
 - Transaction T1 updates the battery level to 80.
 - PostgreSQL does not overwrite 40.
 - It creates a new row version

MVCC



- Transactions whose snapshots were created before transaction 200 may see 40.
- Transactions created after transaction 200 commits normally see 80.
- The old version becomes a dead tuple when no transaction needs it.

MVCC

- **2. Tuple metadata**
- Every PostgreSQL row version, called a **tuple**, contains internal metadata.

Field	Meaning
xmin	Transaction ID that created the tuple
xmax	Transaction ID that deleted or superseded the tuple
ctid	Physical location of the tuple
Infomask	Stores tuple-status information

- **2. Tuple metadata**
- Inspect selected system columns:
 - `SELECT xmin, xmax, ctid, vehicle_id, battery_level`
 - `FROM vehicles;`
- These fields are internal implementation details and should not normally be used as permanent application identifiers.

- **3. INSERT operation**

- INSERT INTO vehicles(vehicle_id, battery_level)
 - VALUES ('BMW1001', 40);
- If transaction ID 100 performs the insert:
 - xmin = 100
 - xmax = 0
- xmin = 100 means transaction 100 created the tuple.
- xmax = 0 generally means it has not been deleted or updated.
- Other transactions can see the row only after transaction 100 commits and their snapshot permits visibility.

MVCC

- **4. UPDATE operation**

- UPDATE vehicles SET battery_level = 80
- WHERE vehicle_id = 'BMW1001';

- Assume transaction 200 performs the update.

- Postgres creates two tuples

Tuple	xmin	xmax	Value
Old version	100	200	40
New version	200	0	80

An update is internally similar to:
Mark old version as superseded

+

Create a new version

This is why frequent updates can produce dead tuples and table bloat.

- **5. DELETE operation**

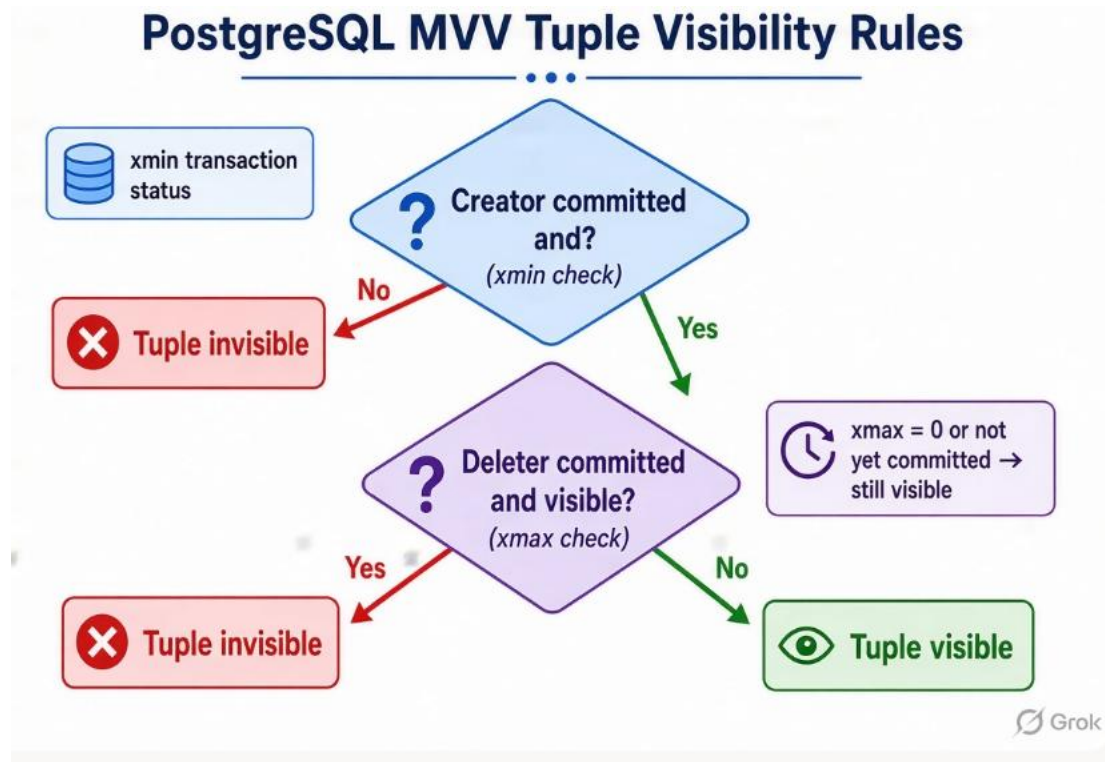
- DELETE FROM vehicles
 - WHERE vehicle_id = 'BMW1001';
- A delete does not immediately remove the tuple from the data page. It normally records the deleting transaction ID in xmax.
 - Before DELETE: xmin = 100, xmax = 0
 - After DELETE: xmin = 100, xmax = 300
 - Older snapshots may still see the row. Later snapshots do not see it after transaction 300 commits.

- **6. Transaction snapshots**
- A snapshot determines which transaction effects are visible.
- Conceptually, a snapshot records:
 - Transactions committed before the snapshot
 - Transactions still active
 - The current transaction
 - Transaction-ID visibility boundaries

- **6. Transaction snapshots**
- Example:
 - Transaction 100: committed
 - Transaction 101: committed
 - Transaction 102: still running
 - Transaction 103: current transaction
- The snapshot can see committed work from 100 and 101, but not uncommitted work from 102.

- **7. Visibility rules**
- A tuple is broadly visible when:
 - Its creating transaction committed.
 - The creation is visible to the current snapshot.
 - It was not deleted, or its deleting transaction is not visible to the snapshot.

• 7. Visibility rules



The actual PostgreSQL visibility rules contain additional cases for the current transaction, aborted transactions, subtransactions and frozen tuples.

MVCC

- **Readers and writers**
- MVCC provides the following general behaviour:

Operation pair	Typical behaviour
Reader and reader	Do not block each other
Reader and writer	Usually do not block each other
Writer and reader	Usually do not block each other
Writer and writer on different rows	Can proceed concurrently
Writer and writer on the same row	One may wait or fail

- MVCC reduces lock contention, but it does not eliminate locking.
- For example:
- `UPDATE vehicles SET battery_level = 90 WHERE vehicle_id = 'BMW1001';`
- A second transaction trying to update the same row may wait for the first transaction to commit or roll back.

- **9. Isolation levels**
- **Read Committed**
- This is PostgreSQL's default isolation level.
 - Every statement receives a new snapshot.
 - Two queries in one transaction can return different committed data.
- BEGIN;
- SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
- SELECT battery_level FROM vehicles WHERE vehicle_id = 'BMW1001';
- -- Another transaction commits an update
- SELECT battery_level FROM vehicles WHERE vehicle_id = 'BMW1001';
- COMMIT;
- The second query may see the new value.

- **9. Isolation levels**
- **Repeatable Read**
 - The transaction generally uses one snapshot.
 - Repeated queries see a stable view.
 - Serialization failures can occur when concurrent changes conflict.
- **BEGIN ISOLATION LEVEL REPEATABLE READ;**

- **10. VACUUM**

- Old tuple versions cannot remain indefinitely.
- VACUUM:
 - Identifies dead tuples no active transaction needs
 - Marks their space as reusable
 - Updates the visibility map
 - Prevents transaction-ID wraparound
 - Does not normally shrink the physical table file

- **10. VACUUM**

- VACUUM vehicles;
- VACUUM ANALYZE also refreshes optimizer statistics:
 - VACUUM (ANALYZE) vehicles;
- VACUUM FULL rewrites and shrinks the table but requires a strong lock:
 - VACUUM FULL vehicles;

- **11. Autovacuum**
- Autovacuum automatically:
 - Removes reusable dead tuple versions
 - Updates table statistics
 - Prevents transaction-ID wraparound
- Check dead tuples:
- `SELECT schemaname, relname, n_live_tup, n_dead_tup, last_autovacuum FROM pg_stat_user_tables`
- `ORDER BY n_dead_tup DESC;`
- Long-running transactions can prevent cleanup because old row versions may still be visible to their snapshots.

- **12. HOT updates**
- A **Heap-Only Tuple update** can reduce index-maintenance cost when:
 - Indexed columns are not changed.
 - The new tuple fits on the same table page.
- Example:
 - `UPDATE vehicles SET battery_level = 80`
 - `WHERE vehicle_id = 'BMW1001';`
 - If `battery_level` is not indexed and the page has enough free space, PostgreSQL may perform a HOT update.

MVCC

- **12. HOT updates**
- Check HOT-update statistics:
- `SELECT`
- `relname,`
- `n_tup_upd,`
- `n_tup_hot_upd`
- `FROM pg_stat_user_tables;`

MVCC

- **13. Transaction-ID wraparound**

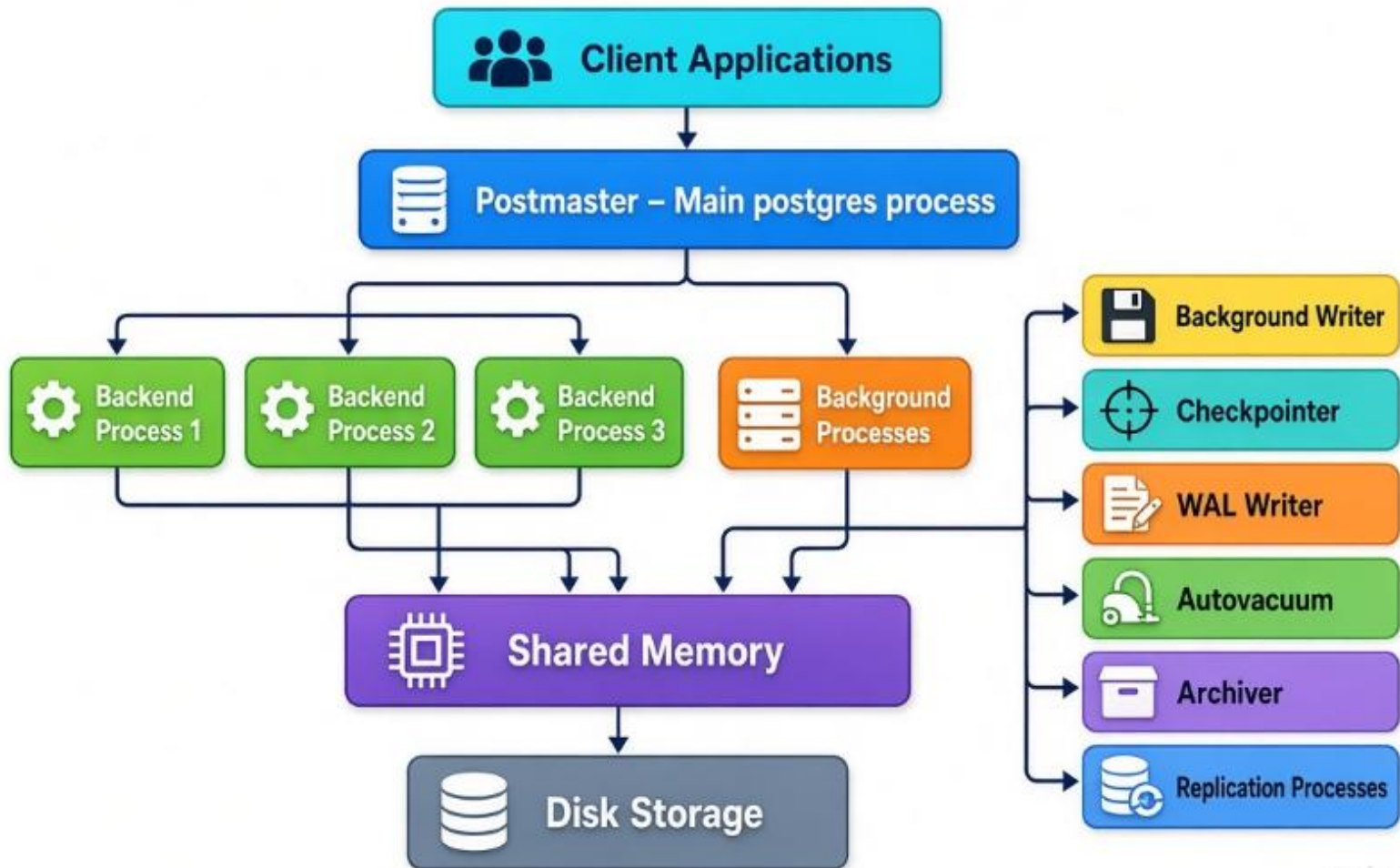
- PostgreSQL transaction IDs are finite and eventually wrap around.
- To prevent old rows from appearing to belong to future transactions, VACUUM freezes sufficiently old tuple transaction IDs.
- Monitor database age:
- `SELECT`
 - `datname,`
 - `age(datfrozenxid)`
- `FROM pg_database`
- `ORDER BY age(datfrozenxid) DESC;`
- Failure to vacuum can eventually force PostgreSQL to restrict writes to protect data integrity.

MVCC

- 14. Advantages and costs

Advantages	Costs
Readers usually do not block writers	Updates create additional row versions
Consistent transaction snapshots	Dead tuples consume space
High concurrent-read performance	Vacuum is essential
Transaction isolation	Long transactions delay cleanup
Rollback support	Heavy updates can cause table and index bloat

Processes



Processes

- **1. Postmaster process**
- The **postmaster**, also called the main postgres process, is the parent PostgreSQL server process.
- Its responsibilities include:
 - Starting PostgreSQL
 - Allocating shared memory
 - Listening for client connections
 - Coordinating authentication
 - Creating backend processes
 - Starting background processes
 - Detecting failed child processes
 - Coordinating shutdown and recovery

Processes

- **1. Postmaster process**
- Start PostgreSQL:
 - `pg_ctl start -D /path/to/pgdata`
- The postmaster listens on the configured address and port:
 - `listen_addresses = '*'`
 - `port = 5432`

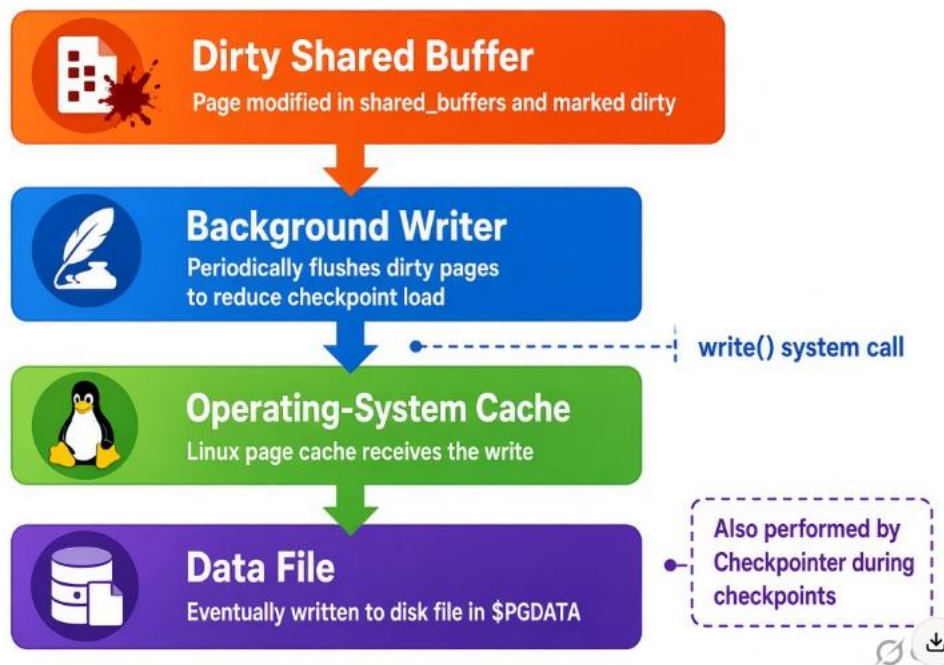
Processes

- **2. Backend processes**
- PostgreSQL normally creates one backend process for each database connection.
 - Client 1 → Backend Process 1
 - Client 2 → Backend Process 2
 - Client 3 → Backend Process 3
- Each backend:
 - Receives SQL
 - Parses and validates it
 - Rewrites the query when necessary
 - Creates an execution plan
 - Executes the plan
 - Reads or modifies data
 - Returns the result
- SQL → Parse → Analyze → Rewrite → Plan → Execute → Result

Processes

- **3. Background writer**
- The background writer gradually writes dirty shared-buffer pages to data files.

PostgreSQL Dirty Buffer Write Path



Processes

- **3. Background writer**
- Its purpose is to reduce the number of dirty pages that backend processes may have to write themselves.
- Important point:
 - It improves write distribution.
 - It does not guarantee transaction durability.
 - WAL provides the primary durability mechanism.
- Useful statistics:
- `SELECT *`
- `FROM pg_stat_bgwriter;`

Processes

• 4. Checkpointer

- The checkpointer coordinates checkpoints.
- During a checkpoint, PostgreSQL ensures that required dirty pages are written to storage and creates a checkpoint record in WAL.

PostgreSQL Checkpoint Process



Processes

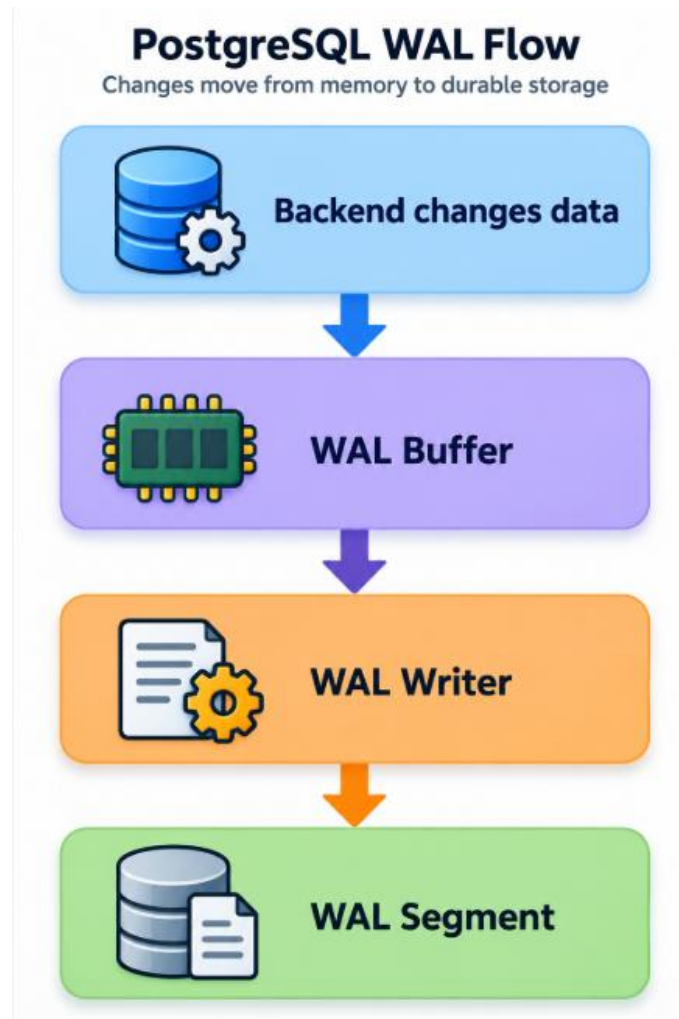
- **4. Checkpointer**
- Checkpoints help limit crash-recovery time. Excessive checkpoints can cause high disk I/O.
- Important settings include:
 - `checkpoint_timeout` = 5min
 - `max_wal_size` = 1GB
 - `checkpoint_completion_target` = 0.9

Processes

- **5. WAL writer**
- The WAL writer flushes WAL records from WAL buffers into WAL segment files.
- At transaction commit, a backend may still need to ensure that the transaction's WAL record is durably flushed before acknowledging success.

Processes

- 5. WAL writer



Processes

- **5. WAL writer**
- WAL supports:
 - Crash recovery
 - Transaction durability
 - Physical replication
 - Point-in-time recovery
 - Continuous archiving

Processes

6. Autovacuum launcher

The autovacuum launcher monitors databases and determines when tables need:

- VACUUM
- ANALYZE
- Anti-wraparound vacuuming

It launches autovacuum workers to perform the operations.

Processes

- **7. Autovacuum workers**
- An autovacuum worker connects to a database and processes selected tables.
- Responsibilities include:
 - Marking dead-tuple space reusable
 - Updating table statistics
 - Maintaining the visibility map
 - Preventing transaction-ID wraparound

Processes

- **8. Archiver process**
- The archiver copies completed WAL segments to a configured archive location.
- Enable archiving:
 - `archive_mode = on`
 - `archive_command = 'copy "%p" "D:\\pgarchive\\%f"'`
- On Linux:
 - `archive_command = 'cp %p /pgarchive/%f'`
- The archiver is required for workflows such as:
 - Point-in-time recovery
 - Continuous backup
 - Long-term WAL retention

Processes

- **9. Startup process**
- The startup process performs recovery when PostgreSQL starts after:
 - A server crash
 - An unclean shutdown
 - Restoring a base backup
 - Starting as a standby server
 - It replays WAL records to restore the database to a consistent state.
 - On a standby server, the startup process continues applying WAL received from the primary.

Processes

- **10. WAL receiver**
- The WAL receiver runs on a physical standby server.
- It:
 - Connects to the primary
 - Receives WAL records
 - Writes them locally
 - Makes them available for replay

Processes

- **12. Logical replication processes**
 - Logical replication uses several processes.
- **Logical replication launcher**
 - Starts and supervises logical-replication workers.
- **Apply worker**
 - Runs on the subscriber and applies changes received from the publisher.
- **Table synchronization worker**
 - Copies initial table data during subscription setup.

Process Summary

Process	Primary responsibility
Postmaster	Accepts connections and manages server processes
Backend	Executes SQL for one client connection
Background writer	Gradually writes dirty buffers
Checkpointer	Coordinates checkpoints
WAL writer	Writes WAL-buffer contents
Autovacuum launcher	Schedules vacuum and analyze work
Autovacuum worker	Vacuums and analyzes tables
Archiver	Archives completed WAL segments
Startup process	Performs recovery and WAL replay
WAL sender	Sends WAL to replication clients
WAL receiver	Receives WAL on a standby
Logical workers	Copy and apply logical changes
Parallel workers	Execute eligible query operations concurrently

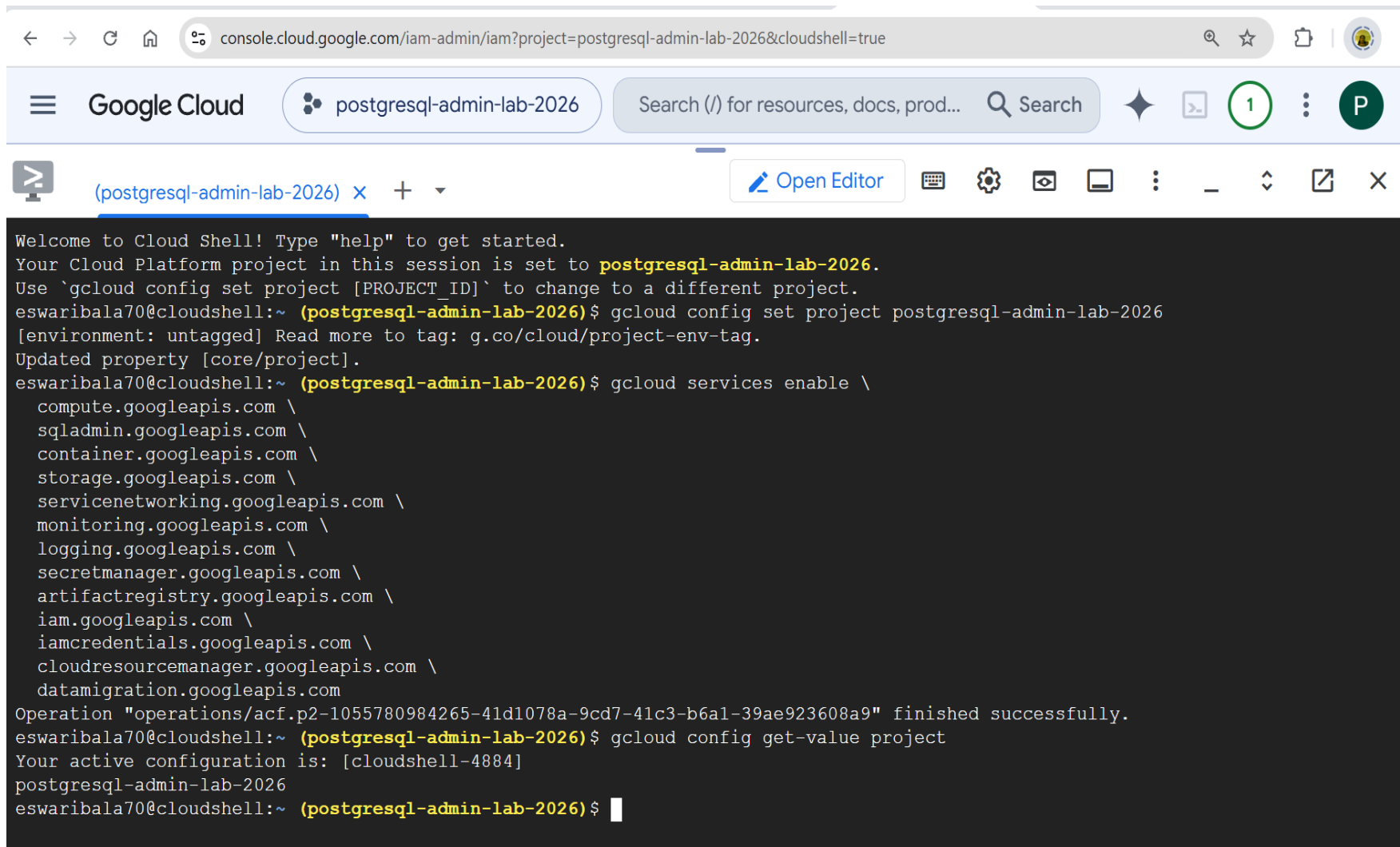
Ubuntu Postgre Installation

- Step 1: Update Package List
 - **sudo apt update**
- Step 2: Install PostgreSQL and Contrib Package
 - **sudo apt install postgresql postgresql-contrib -y**
- Step 3: Check PostgreSQL Service Status
 - **sudo systemctl status postgresql**
- To start/stop manually:
 - **sudo systemctl start postgresql**
 - **sudo systemctl stop postgresql**

Ubuntu Postgre Installation

- Step 4: Switch to the postgres User
 - **sudo -i -u postgres**
- Then access PostgreSQL shell:
 - **psql**

GCP Cloud Shell



```
Welcome to Cloud Shell! Type "help" to get started.
Your Cloud Platform project in this session is set to postgresql-admin-lab-2026.
Use `gcloud config set project [PROJECT_ID]` to change to a different project.
eswaribala70@cloudshell:~ (postgresql-admin-lab-2026)$ gcloud config set project postgresql-admin-lab-2026
[environment: untagged] Read more to tag: g.co/cloud/project-env-tag.
Updated property [core/project].
eswaribala70@cloudshell:~ (postgresql-admin-lab-2026)$ gcloud services enable \
  compute.googleapis.com \
  sqladmin.googleapis.com \
  container.googleapis.com \
  storage.googleapis.com \
  servicenetworking.googleapis.com \
  monitoring.googleapis.com \
  logging.googleapis.com \
  secretmanager.googleapis.com \
  artifactregistry.googleapis.com \
  iam.googleapis.com \
  iamcredentials.googleapis.com \
  cloudresourcemanager.googleapis.com \
  datamigration.googleapis.com
Operation "operations/acf.p2-1055780984265-41d1078a-9cd7-41c3-b6a1-39ae923608a9" finished successfully.
eswaribala70@cloudshell:~ (postgresql-admin-lab-2026)$ gcloud config get-value project
Your active configuration is: [cloudshell-4884]
postgresql-admin-lab-2026
eswaribala70@cloudshell:~ (postgresql-admin-lab-2026)$
```

GCP Cloud Shell

The screenshot shows the Google Cloud console interface for the project 'postgresql-admin-lab-2026'. The left sidebar contains the navigation menu with 'Compute Engine' selected, showing 'Overview', 'Security risk overview', and 'Compute Advisor ...'. The main content area is titled 'Overview' and features a blue 'Create instance' button, which is circled in red. Below this, there is a section for 'Google Cloud incidents' indicating 'No incidents are impacting Compute Engine' with a green checkmark. Further down, there is a 'Design your infrastructure with Compute Advisor' section with a 'Preview' button and four sub-options: 'Evaluate model tuning', 'Monitor performance', 'Resolve cluster stockouts', and 'Get step-by-step guides'.

Google Cloud

postgresql-admin-lab-2026

Search (/) for resources, docs, produc... Search

Compute Engine

Overview

Security risk overview

Compute Advisor ...

Virtual machines

VM instances

Instance templates

Sole-tenant nodes

Machine images

TPUs

Committed use disco...

Marketplace

Overview

Create instance

Is this page helpful? Refresh

Google Cloud incidents

No incidents are impacting Compute Engine

To check for incidents across Google Cloud, visit the Service Health Dashboard

Go to Personalized Service Health

Design your infrastructure with Compute Advisor Preview

What can I help you with?

Evaluate model tuning

Monitor performance

Resolve cluster stockouts

Get step-by-step guides

Lab 1

- **Lab 1 - Install PostgreSQL**
- Objective: install PostgreSQL by three alternative Linux methods and verify the selected installation.
- **Exercise 1A - Ubuntu package installation**
- 1. Update package metadata.
- 2. Install PostgreSQL and useful contributed extensions.
- 3. Verify the automatically created cluster and service.
 - `sudo apt update`
 - `sudo apt install -y postgresql postgresql-contrib`
 - `psql --version`
 - `sudo systemctl status postgresql --no-pager`
 - `sudo pg_lsclusters`
- Connect locally through peer authentication:
 - `sudo -iu postgres psql`
 - `SELECT version();`
 - `SELECT current_user, current_database();`
 - `\q`

Why install PostgreSQL from the official PGDG repository?

- PGDG means **P**ostgre**S**QL **G**lobal **D**evelopment **G**roup.
- It is the official PostgreSQL software repository maintained by the PostgreSQL community.
- Linux distributions often include PostgreSQL in their default repositories, but that package may be an older version.

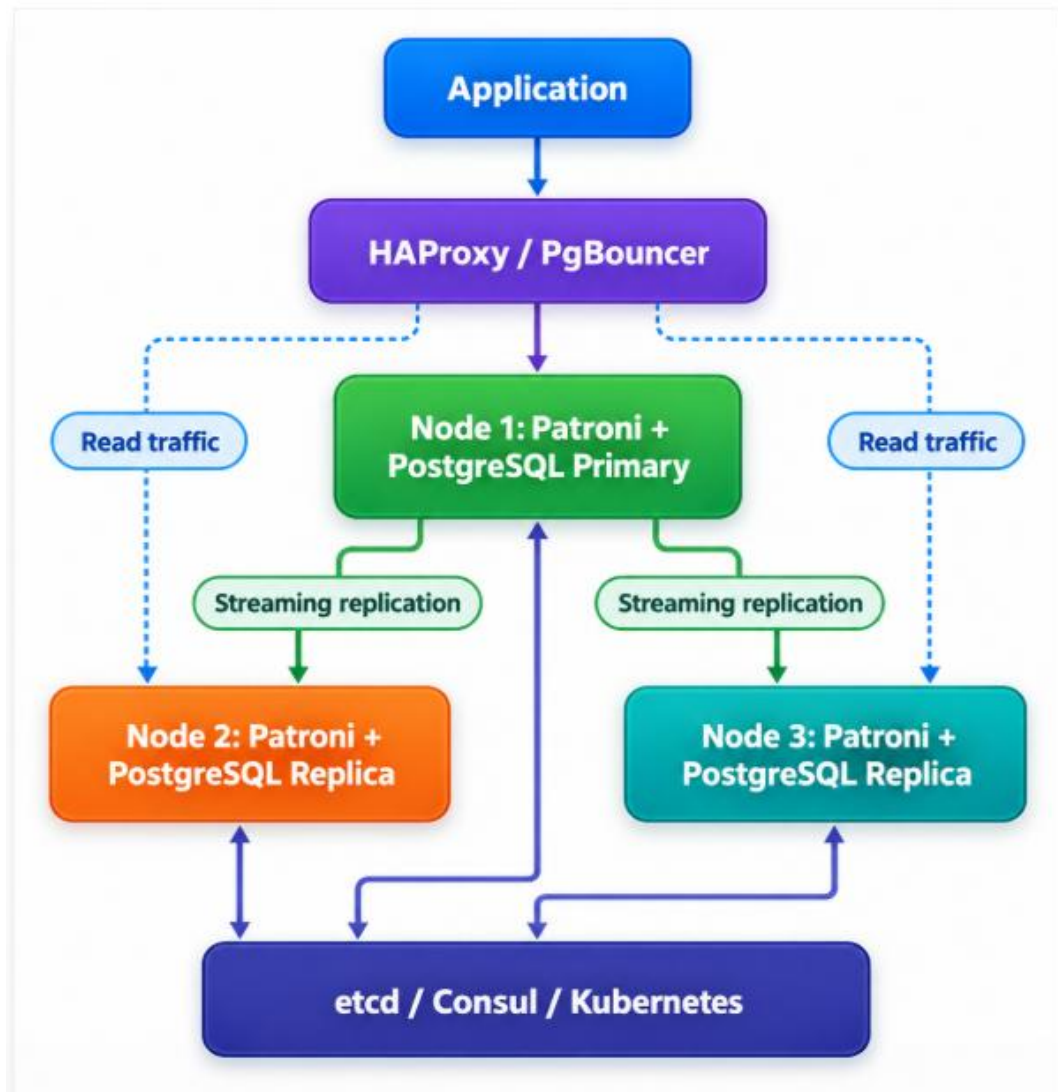
Why install PostgreSQL from the official PGDG repository?

Distribution repository	Official PGDG repository
May contain an older PostgreSQL release	Provides current and supported releases
Usually offers one PostgreSQL version	Allows version-specific installation
Updates follow the Linux distribution schedule	Updates are released by the PostgreSQL community
Limited PostgreSQL extensions	Provides many PostgreSQL-related packages
Package names may be generic	Version-specific packages such as postgresql17-server

Patroni

- **Patroni is an open-source high-availability manager for PostgreSQL.**
- It automates PostgreSQL leader election, replication, failure detection and failover.
- Patroni does not replace PostgreSQL. It runs alongside PostgreSQL and controls its lifecycle.

Patroni



Patroni

- **What Patroni does**

- Initializes the PostgreSQL cluster
- Selects one node as the primary
- Configures PostgreSQL streaming replication
- Continuously monitors database health
- Detects primary-node failure
- Promotes an eligible replica automatically
- Prevents two nodes from acting as primary
- Provides a REST API for monitoring and load balancers
- Supports planned switchover and emergency failover

Patroni Components

Component	Purpose
PostgreSQL	Stores and processes application data
Patroni	Manages PostgreSQL roles and failover
etcd or Consul	Stores cluster state and leader lock
HAProxy	Routes connections to the current primary
PgBouncer	Optional database connection pooling
Watchdog	Optional split-brain protection
patronictl	Patroni administration command

How Patroni Elects Primary

- Each Patroni node communicates with the distributed configuration store.
- One node acquires the leader lock.
- That node runs PostgreSQL as the primary.
- Other nodes run as streaming replicas.
- The primary regularly renews its leader lock.
- If it cannot renew the lock, Patroni demotes or stops PostgreSQL.
- An eligible replica obtains the lock and becomes the new primary.

Why PGDG is preferable for the Patroni lab

- Use PGDG because it provides:
 - Consistent PostgreSQL versions across all Patroni nodes
 - Timely security fixes and bug fixes
 - Version-specific binaries and directories
 - Required packages such as server, client, development libraries and extensions
 - Easier installation of newer PostgreSQL releases
 - Better suitability for production and high-availability environments

Postgre Administration

- 1. Database Creation
- A PostgreSQL **database** is a logical container holding schemas, tables, views, functions, and other objects.
- **Create a database**
 - `sudo -iu postgres psql`
 - `CREATE DATABASE banking_db;`
- Create with an owner and specific settings:
 - `CREATE DATABASE banking_db`
- `OWNER banking_admin`
- `ENCODING 'UTF8'`
- `TEMPLATE template0;`

Postgre Administration

- **Verify and connect**
- \l
- \c banking_db
- SELECT current_database();
- **Rename or delete**
- Users must disconnect before these operations:
- ALTER DATABASE banking_db RENAME TO banking_prod;
- DROP DATABASE banking_prod;

Postgre Administration

- Terminate existing connections when necessary:
- `SELECT pg_terminate_backend(pid)`
- `FROM pg_stat_activity`
- `WHERE datname = 'banking_prod'`
- `AND pid <> pg_backend_pid();`

2. Cluster Management

- A PostgreSQL **cluster** is one PostgreSQL server instance containing:
 - Multiple databases
 - Roles and permissions
 - Configuration files
 - System catalogs
 - Data files
 - WAL files
 - Tablespaces

2. Cluster Management

- **List clusters on Ubuntu**
- pg_lsclusters
- Example:
- Ver Cluster Port Status Owner Data directory
- 18 main 5432 online postgres
/var/lib/postgresql/18/main
- 18 training 5433 online postgres
/var/lib/postgresql/18/training

2. Cluster Management

- **Create an additional cluster**
 - `sudo pg_createcluster 18 training --port 5433 --start`
- **Control a cluster**
 - `sudo pg_ctlcluster 18 training status`
 - `sudo pg_ctlcluster 18 training stop`
 - `sudo pg_ctlcluster 18 training start`
 - `sudo pg_ctlcluster 18 training restart`
 - `sudo pg_ctlcluster 18 training reload`
- reload: rereads reloadable configuration without disconnecting users.
- restart: stops and starts the server; existing connections are interrupted.

2. Cluster Management

- **Connect to a particular cluster**
 - `sudo -iu postgres psql -p 5433 -d postgres`
- **Inspect cluster settings**
 - `SHOW data_directory;`
 - `SHOW config_file;`
 - `SHOW hba_file;`
 - `SHOW port;`
- **Delete a practice cluster**
- This permanently removes its databases and data:
 - `sudo pg_dropcluster 18 training --stop`

3. Roles

- PostgreSQL uses **roles** to manage authentication, authorization, and object ownership.
- A role can represent:
 - An individual user
 - An application account
 - A group of users
 - A database administrator

3. Roles

- **Create a group role**
- A group role normally does not log in:
 - CREATE ROLE banking_readonly NOLOGIN;
 - CREATE ROLE banking_readwrite NOLOGIN;
- **Grant permissions to group roles**
 - GRANT CONNECT ON DATABASE banking_db TO banking_readonly;
 - GRANT USAGE ON SCHEMA public TO banking_readonly;
 - GRANT SELECT ON ALL TABLES IN SCHEMA public TO banking_readonly;

3. Roles

- For new tables created later:
 - ALTER DEFAULT PRIVILEGES IN SCHEMA public
 - GRANT SELECT ON TABLES TO banking_readonly;
- Read-write privileges:
 - GRANT CONNECT ON DATABASE banking_db TO banking_readwrite;
 - GRANT USAGE ON SCHEMA public TO banking_readwrite;
 - GRANT SELECT, INSERT, UPDATE, DELETE
 - ON ALL TABLES IN SCHEMA public
 - TO banking_readwrite;

3. Roles

- **Role membership**

- GRANT banking_readonly TO audit_user;
- GRANT banking_readwrite TO application_user;

- Remove membership:

- REVOKE banking_readwrite FROM application_user;

- **View roles**

- \du

- Or:

- SELECT rolname, rolcanlogin, rolsuper, rolcreatedb, rolcreatorole FROM pg_roles;

4. Users

- PostgreSQL does not maintain a separate user object. **A user is simply a role with the LOGIN attribute.**
- These commands are equivalent:
 - `CREATE USER audit_user PASSWORD 'StrongPassword';`
 - `CREATE ROLE audit_user LOGIN PASSWORD 'StrongPassword';`

4. Users

- **Create application users**
 - CREATE ROLE banking_admin
 - LOGIN
 - PASSWORD 'StrongPassword'
 - CREATEDB;

 - CREATE ROLE application_user
 - LOGIN
 - PASSWORD 'StrongPassword'
 - NOSUPERUSER
 - NOCREATEDB
 - NOCREATEROLE;

4. Users

- **Password and account administration**
 - ALTER ROLE application_user PASSWORD 'NewStrongPassword';
- Set password expiration:
 - ALTER ROLE application_user VALID UNTIL '2026-12-31';
- Disable login temporarily:
 - ALTER ROLE application_user NOLOGIN;
- Enable it again:
 - ALTER ROLE application_user LOGIN;

4. Users

- **Recommended practice**
- Do not make application accounts superusers. Grant only the permissions required by the application.

5. Tablespaces

- A **tablespace** maps a PostgreSQL logical name to a directory on another storage location.
- Typical uses include:
 - Keeping large indexes on fast SSD storage
 - Separating database data from the operating-system disk
 - Distributing I/O across disks
 - Placing archive or reporting data on lower-cost storage

5. Tablespaces

- **Create the operating-system directory**
- `sudo mkdir -p /data/pg_tablespace/fast_storage`
- `sudo chown postgres:postgres /data/pg_tablespace/fast_storage`
- `sudo chmod 700 /data/pg_tablespace/fast_storage`

5. Tablespaces

- The directory must:
 - Be empty
 - Be owned by the PostgreSQL operating-system user
 - Be accessible on the PostgreSQL server
 - Not be placed inside the main data directory

5. Tablespaces

- **Create the tablespace**

- CREATE TABLESPACE fast_storage
- OWNER postgres
- LOCATION '/data/pg_tablespaces/fast_storage';

- **View tablespaces**

- \db

- Or:

- SELECT spcname,
- pg_tablespace_location(oid)
- FROM pg_tablespace;

5. Tablespaces

- **Store objects in the tablespace**
- Create a database:
 - CREATE DATABASE analytics_db TABLESPACE fast_storage;
- Create a table:
 - CREATE TABLE transactions (
 - transaction_id BIGINT GENERATED ALWAYS AS IDENTITY,
 - account_id BIGINT,
 - amount NUMERIC(12,2),
 - created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
 -) TABLESPACE fast_storage;

5. Tablespaces

- Move an existing table:
 - `ALTER TABLE transactions SET TABLESPACE fast_storage;`
- Create an index in the tablespace:
 - `CREATE INDEX idx_transactions_account ON transactions(account_id) TABLESPACE fast_storage;`
- **Drop a tablespace**
- A tablespace must be empty before removal:
 - `DROP TABLESPACE fast_storage;`
- PostgreSQL removes its internal tablespace link, but the administrator should verify and manage the underlying directory separately.

Lab 1

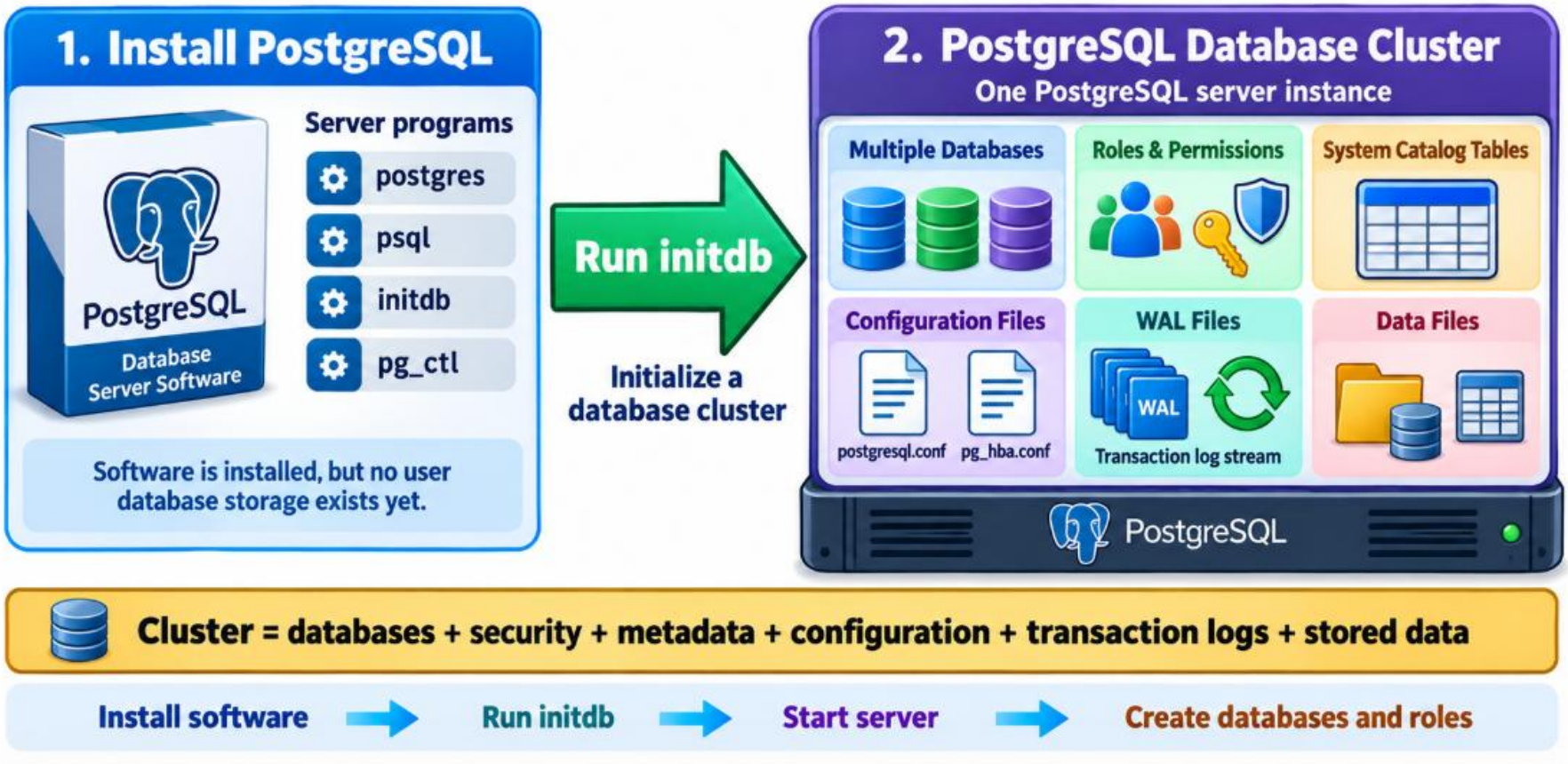
- **Exercise 1B - Install from the official PostgreSQL repository (PGDG)**
- Use PGDG when you need:
 - A specific major version, such as PostgreSQL 18
 - Faster access to new releases
 - Current security and bug-fix updates
 - Multiple PostgreSQL versions on the same Linux machine
 - Official extensions and administration utilities
 - Production-like installation and upgrade practice

Lab 1

- **Exercise 1C - Build PostgreSQL from source**
- Building PostgreSQL from source teaches how PostgreSQL is compiled, installed and controlled without relying on Ubuntu or PGDG packages.
- It is useful when you need:
 - A custom installation location such as `/opt/postgresql/18`
 - Compilation options that packaged installations do not enable
 - Source-code modification or debugging
 - Testing a development, beta or release-candidate version
 - Specific libraries, extensions or performance features
 - A deeper understanding of PostgreSQL installation internals

Lab 2 - Initialize a PostgreSQL cluster

Software provides the engine. initdb creates the storage environment.

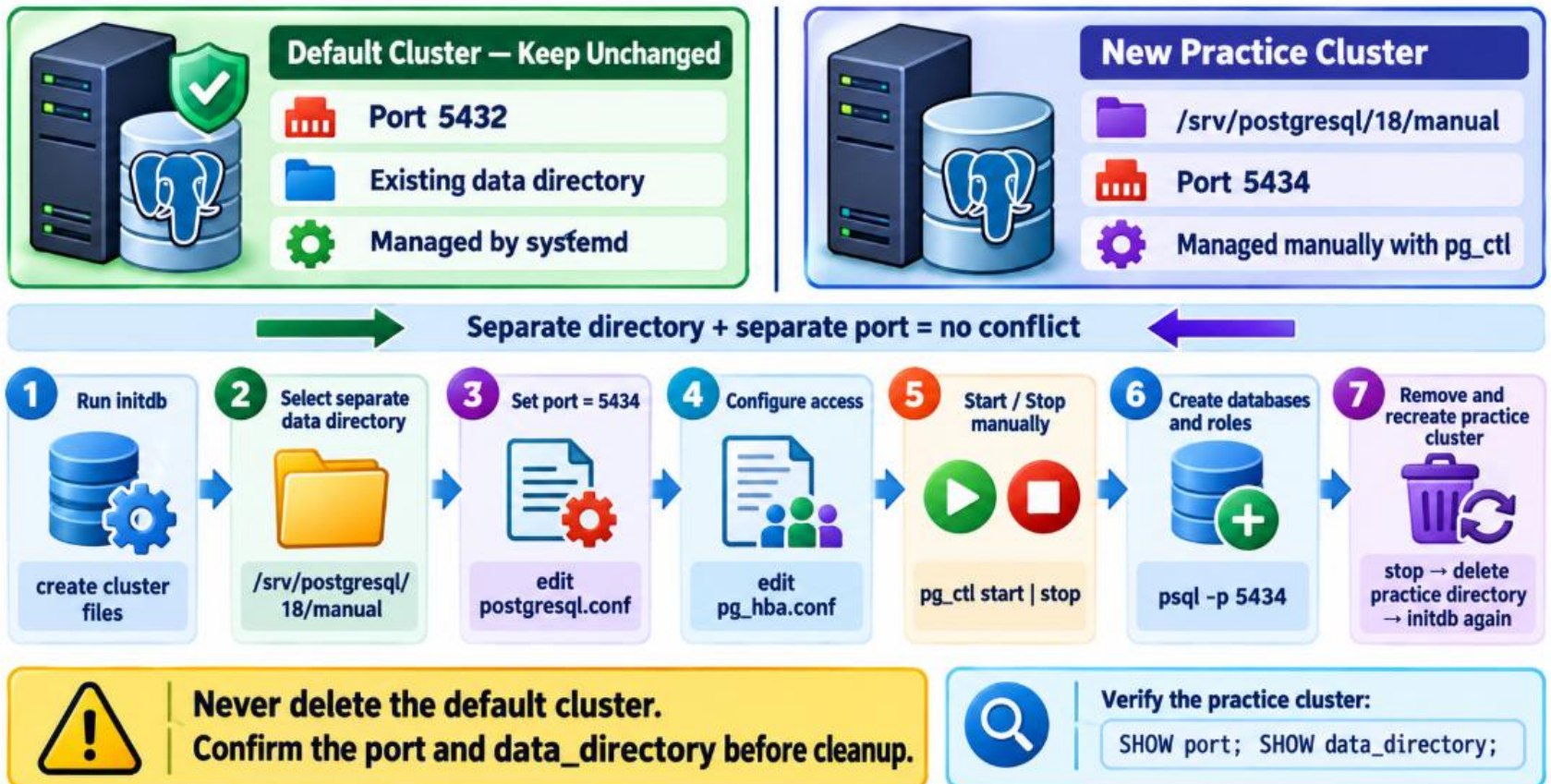


Lab 2 - Initialize a PostgreSQL cluster

- Installing PostgreSQL gives you the **server software**, but PostgreSQL also needs an initialized **database cluster** to store and manage databases.
- A PostgreSQL cluster is one server instance that contains:
 - Multiple databases
 - Roles and permissions
 - System catalog tables
 - Configuration files
 - WAL files
 - Data files

Lab 2 - Initialize a PostgreSQL cluster

PostgreSQL Manual Cluster Lab — Safe Practice on Port 5434



Why create a separate training cluster?

- The lab creates a new cluster without modifying the default cluster so you can safely practise:
 - Running initdb
 - Selecting a separate data directory
 - Using another port, such as 5434
 - Changing postgresql.conf
 - Changing pg_hba.conf
 - Starting and stopping a cluster manually
 - Creating databases and roles
 - Removing and recreating the cluster

Why create a separate training cluster?

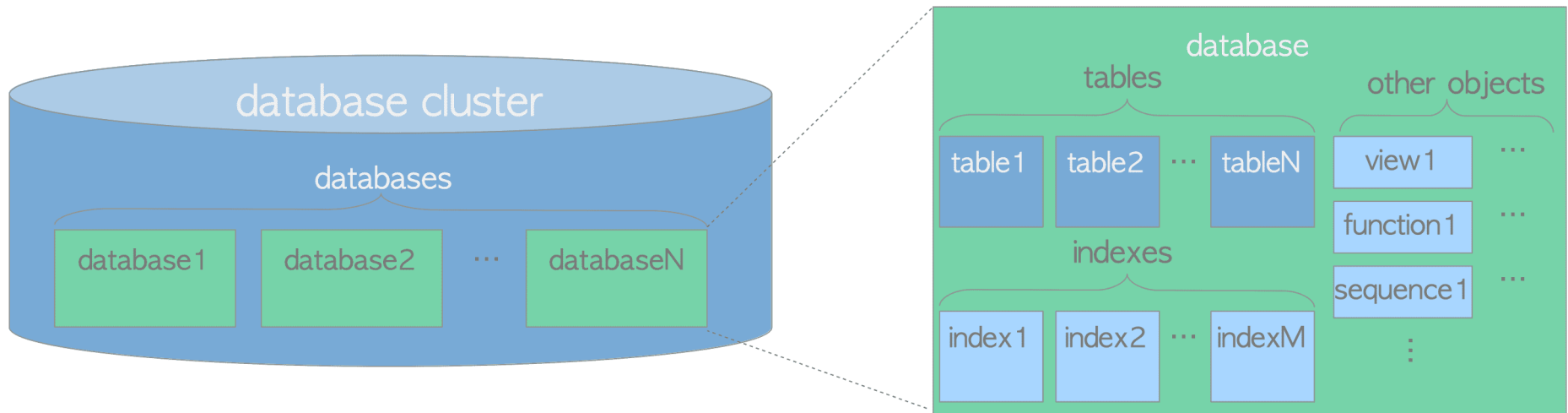
Cluster	Data directory	Port	Purpose
Default	/var/lib/postgresql/18/main	5432	Standard installation
Training	/srv/postgresql/18/manual	5434	Hands-on experiments

Both clusters can run simultaneously because they use different data directories and ports.

Why not use the default cluster?

- Incorrect configuration during training could:
 - Prevent the default server from starting
 - Block existing applications
 - Damage or delete databases
 - Cause port conflicts
 - Change authentication for existing users
- A separate training cluster isolates these risks.

Database Cluster, Databases, and Tables



Database Cluster, Databases, and Tables

- A database cluster is a collection of databases managed by a PostgreSQL server.
- The term 'database cluster' in PostgreSQL does not mean 'a group of database servers'.
- A PostgreSQL server runs on a single host and manages a single database cluster.

Database Cluster, Databases, and Tables

- A database is a collection of database objects.
- In the relational database theory, a database object is a data structure used either to store or to reference data.
- A (heap) table is a typical example of it, and there are many more like an index, a sequence, a view, a function and so on.
- In PostgreSQL, databases themselves are also database objects and are logically separated from each other.
- All other database objects (e.g., tables, indexes, etc) belong to their respective databases.

Database Cluster, Databases, and Tables

- All the database objects in PostgreSQL are internally managed by respective object identifiers (OIDs), which are unsigned 4-byte integers.
- The relations between database objects and the respective OIDs are stored in appropriate system catalogs, depending on the type of objects.
- For example, OIDs of databases and heap tables are stored in `pg_database` and `pg_class` respectively,

Database Cluster, Databases, and Tables

- All the database objects in PostgreSQL are internally managed by respective object identifiers (OIDs), which are unsigned 4-byte integers.
- The relations between database objects and the respective OIDs are stored in appropriate system catalogs, depending on the type of objects.
- For example, OIDs of databases and heap tables are stored in `pg_database` and `pg_class` respectively,

Database Cluster, Databases, and Tables

pgAdmin 4 interface showing the 'Create - Login/Group Role' dialog box. The dialog has tabs for General, Definition, Privileges, Membership, Parameters, Security, and SQL. The General tab is active, showing fields for Name and Comments. A 'Save password?' dialog is overlaid on top, asking to save the password for the 'admin' username. The background shows the pgAdmin dashboard with a tree view on the left listing servers (PostgreSQL 11, 12, 13) and databases (pem, postgres, testdb). The main panel shows 'Server sessions' and 'Server activity' sections.

Save password?

Username: admin

Password:

Buttons: Save, Never

Server sessions

Server activity

State	Wait event	Blocking PIDs
idle	Client: ClientRead	
idle	Client: ClientRead	

Database Cluster, Databases, and Tables

```
SQL Shell (psql)
Server [localhost]:
Database [postgres]:
Port [5432]:
Username [postgres]: admin
Password for user admin:
psql (13.2)
WARNING: Console code page (437) differs from Windows code page (1252)
         8-bit characters might not work correctly. See psql reference
         page "Notes for Windows users" for details.
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, bits: 256, compression: off)
Type "help" for help.

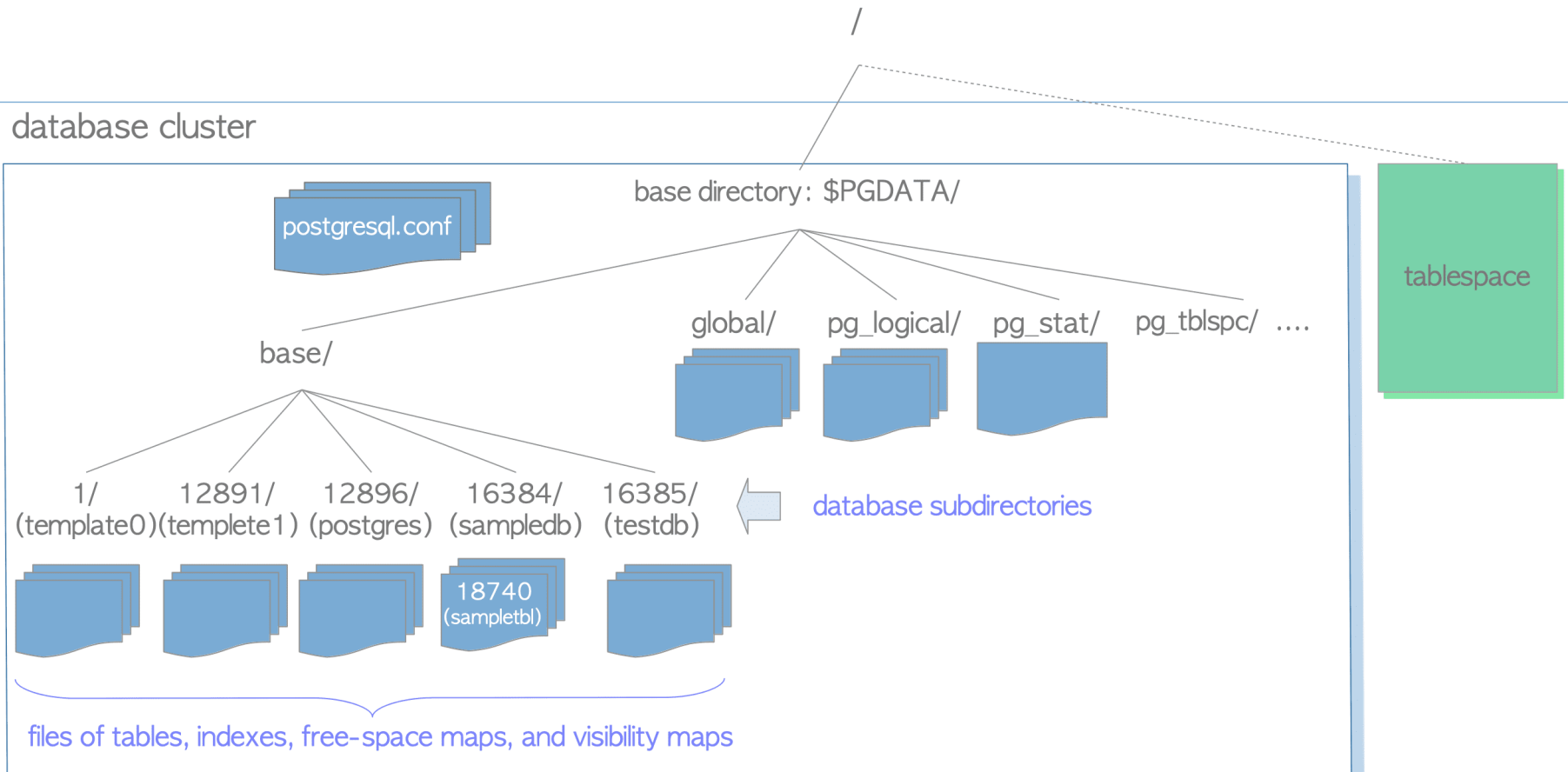
postgres=# SELECT datname, oid FROM pg_database WHERE datname = 'testdb';
 datname | oid
-----+-----
 testdb  | 28956
(1 row)

postgres=#
```

Physical Structure of Database Cluster

- A database cluster basically is one directory referred to as base directory, and it contains some subdirectories and lots of files.
- If we execute the initdb utility to initialize a new database cluster, a base directory will be created under the specified directory.
- Path of the base directory is usually set to the environment variable PGDATA.

Physical Structure of Database Cluster



Layout of a Database Cluster

table 1.1: Layout of files and subdirectories under the base directory (From the official document)

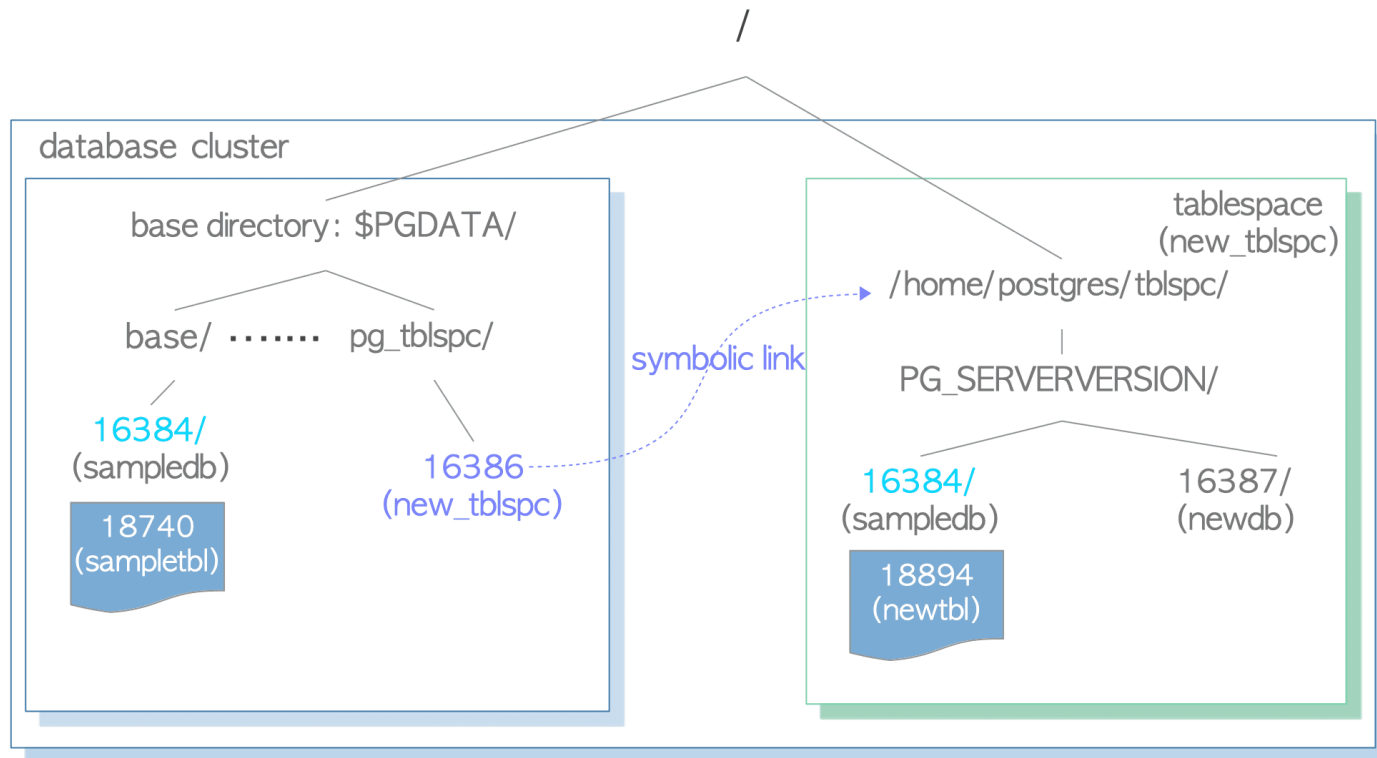
files	description
PG_VERSION	A file containing the major version number of PostgreSQL
pg_hba.conf	A file to control PostgreSQL's client authentication
pg_ident.conf	A file to control PostgreSQL's user name mapping
postgresql.conf	A file to set configuration parameters
postgresql.auto.conf	A file used for storing configuration parameters that are set in ALTER SYSTEM (version 9.4 or later)
postmaster.opts	A file recording the command line options the server was last started with
subdirectories	description
base/	Subdirectory containing per-database subdirectories.
global/	Subdirectory containing cluster-wide tables, such as pg_database and pg_control.
pg_commit_ts/	Subdirectory containing transaction commit timestamp data. Version 9.5 or later.
pg_clog/ (Version 9.6 or earlier)	Subdirectory containing transaction commit state data. It is renamed to <i>pg_xact</i> in Version 10. CLOG will be described in Section 5.4 .
pg_dynshmem/	Subdirectory containing files used by the dynamic shared memory subsystem. Version 9.4 or later.
pg_logical/	Subdirectory containing status data for logical decoding. Version 9.4 or later.
pg_multixact/	Subdirectory containing multitransaction status data (used for shared row locks)
pg_notify/	Subdirectory containing LISTEN/NOTIFY status data
pg_repslot/	Subdirectory containing replication slot data. Version 9.4 or later.
pg_serial/	Subdirectory containing information about committed serializable transactions (version 9.1 or later)
pg_snapshots/	Subdirectory containing exported snapshots (version 9.2 or later). The PostgreSQL's function <code>pg_export_snapshot</code> creates a snapshot information file in this subdirectory.
pg_stat/	Subdirectory containing permanent files for the statistics subsystem.
pg_stat_tmp/	Subdirectory containing temporary files for the statistics subsystem.

Layout of a Database Cluster

<code>pg_subtrans/</code>	Subdirectory containing subtransaction status data
<code>pg_tblspc/</code>	Subdirectory containing symbolic links to tablespaces
<code>pg_twophase/</code>	Subdirectory containing state files for prepared transactions
<code>pg_wal/</code> (Version 10 or later)	Subdirectory containing WAL (Write Ahead Logging) segment files. It is renamed from <i>pg_xlog</i> in Version 10.
<code>pg_xact/</code> (Version 10 or later)	Subdirectory containing transaction commit state data. It is renamed from <i>pg_clog</i> in Version 10. CLOG will be described in Section 5.4 .
<code>pg_xlog/</code> (Version 9.6 or earlier)	Subdirectory containing WAL (Write Ahead Logging) segment files. It is renamed to <i>pg_wal</i> in Version 10.

Tablespaces

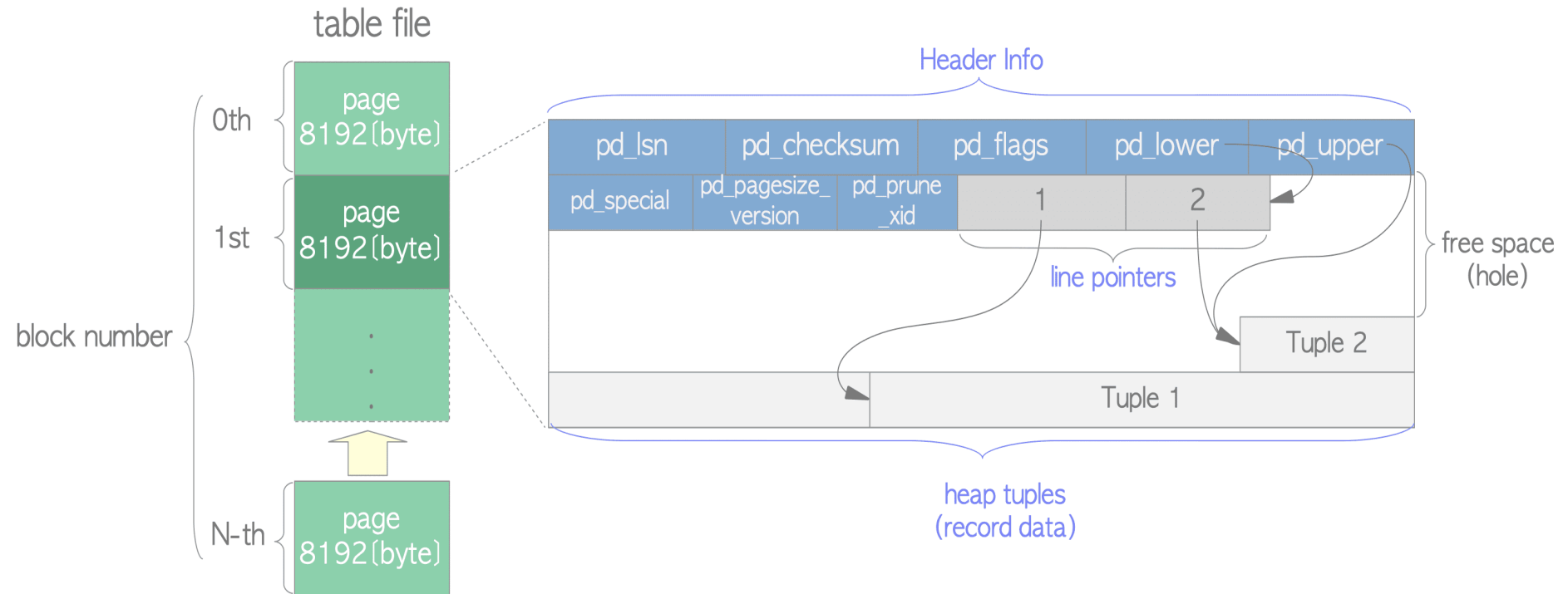
- A tablespace in PostgreSQL is an additional data area outside the base directory.
- This function has been implemented in version 8.0.



Internal Layout of a Heap Table File

- Inside the data file (heap table and index, as well as the free space map and visibility map), it is divided into pages (or blocks) of fixed length, the default is 8192 byte (8 KB).
- Those pages within each file are numbered sequentially from 0, and such numbers are called as block numbers.
- If the file has been filled up, PostgreSQL adds a new empty page to the end of the file to increase the file size.

Internal Layout of a Heap Table File



Internal Layout of a Heap Table File

- heap tuple(s) – A heap tuple is a record data itself. They are stacked in order from the bottom of the page
- line pointer(s) – A line pointer is 4 byte long and holds a pointer to each heap tuple. It is also called an item pointer.
- Line pointers form a simple array, which plays the role of index to the tuples. Each index is numbered sequentially from 1, and called offset number. When a new tuple is added to the page, a new line pointer is also pushed onto the array to point to the new one.

Internal Layout of a Heap Table File

- header data – A header data defined by the structure `PageHeaderData` is allocated in the beginning of the page. It is 24 byte long and contains general information about the page.
- The major variables of the structure are described below.
 - `pd_lsn` – This variable stores the LSN of XLOG record written by the last change of this page. It is an 8-byte unsigned integer, related to the WAL (Write-Ahead Logging) mechanism. The details are described in Chapter 9.
 - `pd_checksum` – This variable stores the checksum value of this page. (Note that this variable is supported in version 9.3 or later; in earlier versions, this part had stored the `timelined` of the page.)

Internal Layout of a Heap Table File

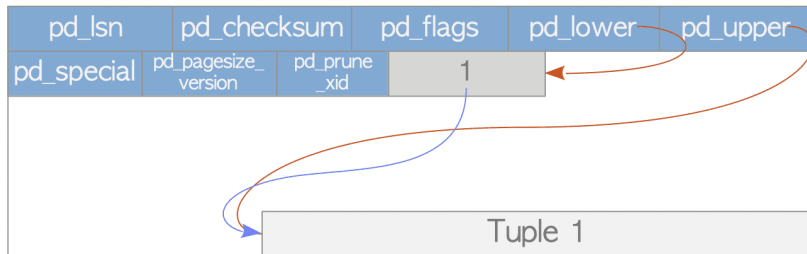
- `pd_lower`, `pd_upper` – `pd_lower` points to the end of line pointers, and `pd_upper` to the beginning of the newest heap tuple.
- `pd_special` – This variable is for indexes. In the page within tables, it points to the end of the page. (In the page within indexes, it points to the beginning of special space which is the data area held only by indexes and contains the particular data according to the kind of index types such as B-tree, GiST, GiN, etc.)

Internal Layout of a Heap Table File

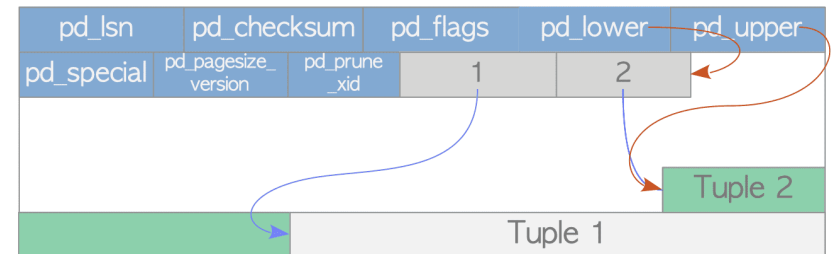
- An empty space between the end of line pointers and the beginning of the newest tuple is referred to as free space or hole.
- To identify a tuple within the table, tuple identifier (TID) is internally used.
- A TID comprises a pair of values: the block number of the page that contains the tuple, and the offset number of the line pointer that points to the tuple.

The Methods of Writing and Reading Tuples

(a) Before insertion of Tuple 2



(b) After insertion of Tuple 2



The Methods of Writing and Reading Tuples

- Writing Heap Tuples
 - Suppose a table composed of one page which contains just one heap tuple.
 - The `pd_lower` of this page points to the first line pointer, and both the line pointer and the `pd_upper` point to the first heap tuple.
 - When the second tuple is inserted, it is placed after the first one.
 - The second line pointer is pushed onto the first one, and it points to the second tuple.
 - The `pd_lower` changes to point to the second line pointer, and the `pd_upper` to the second heap tuple.

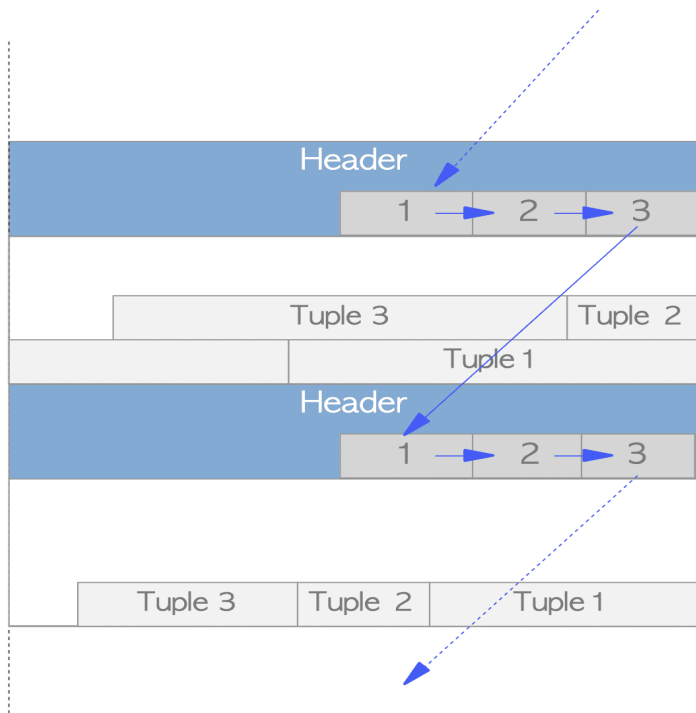
The Methods of Writing and Reading Tuples

- Reading Heap Tuples
 - Two typical access methods, sequential scan and B-tree index scan, are outlined here:
 - Sequential scan – All tuples in all pages are sequentially read by scanning all line pointers in each page.
 - B-tree index scan – An index file contains index tuples, each of which is composed of an index key and a TID pointing to the target heap tuple.
 - If the index tuple with the key that you are looking for has been found, PostgreSQL reads the desired heap tuple using the obtained TID value.
 - It means that the target heap tuple is 2nd tuple in the 7th page within the table, so PostgreSQL can read the desired heap tuple without unnecessary scanning in the pages.

The Methods of Writing and Reading Tuples

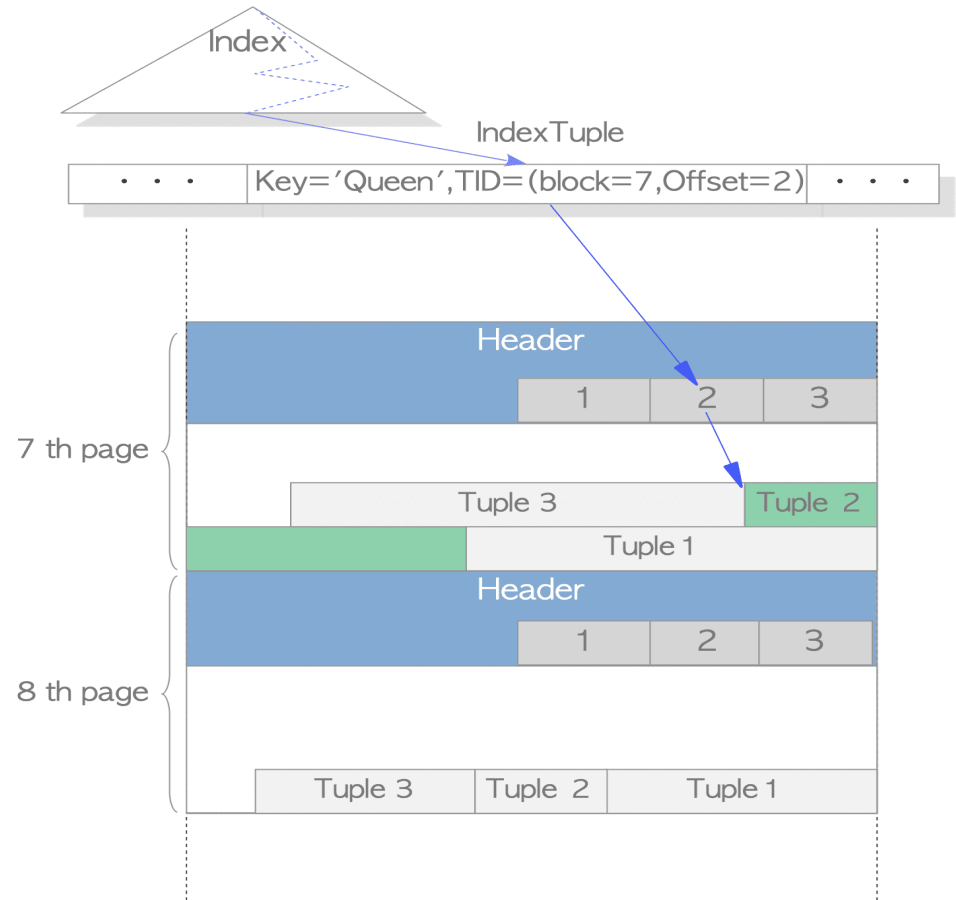
(a) Sequential Scan

SELECT * FROM tbl;



(b) Index Scan

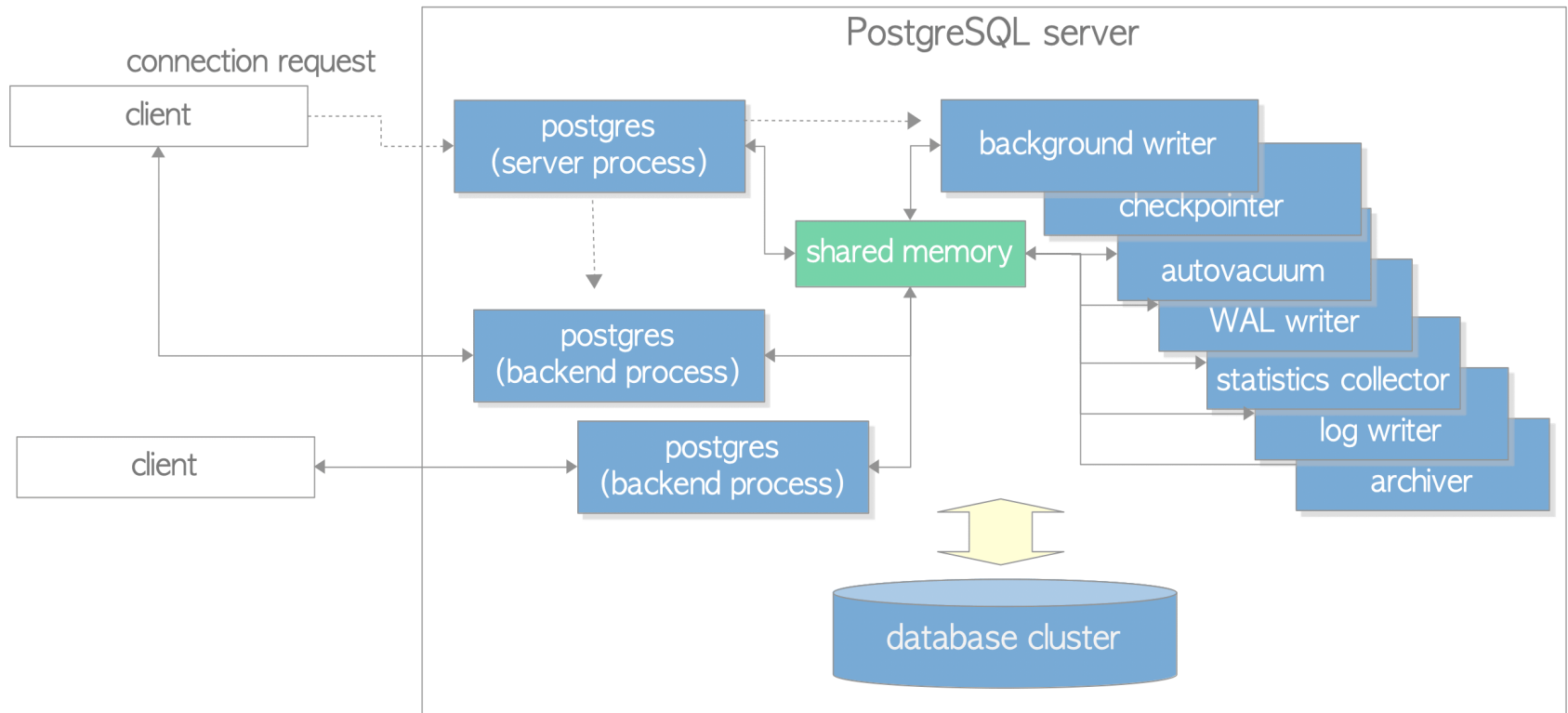
SELECT * FROM tbl WHERE col = 'Queen';



Process and Memory Architecture

- PostgreSQL is a client/server type relational database management system with the multi-process architecture and runs on a single host.
- A collection of multiple processes cooperatively managing one database cluster is usually referred to as a 'PostgreSQL server', and it contains the following types of processes:
 - A postgres server process is a parent of all processes related to a database cluster management.
 - Each backend process handles all queries and statements issued by a connected client.
 - Various background processes perform processes of each feature (e.g., VACUUM and CHECKPOINT processes) for database management.
 - In the replication associated processes, they perform the streaming replication.
 - In the background worker process supported from version 9.3, it can perform any processing implemented by users. As not going into detail here, refer to the official document.

Process and Memory Architecture



Postgres Server Process

- A postgres server process is a parent of all in a PostgreSQL server.
- In the earlier versions, it was called 'postmaster'.
- By executing the `pg_ctl` utility with start option, a postgres server process starts up.
- Then, it allocates a shared memory area in memory, starts various background processes, starts replication associated processes and background worker processes if necessary, and waits for connection requests from clients.
- Whenever receiving a connection request from a client, it starts a backend process. (And then, the started backend process handles all queries issued by the connected client.)
- A postgres server process listens to one network port, the default port is 5432. Although more than one PostgreSQL server can be run on the same host, each server should be set to listen to different port number in each other, e.g., 5432, 5433, etc.

Backend Processes

- A backend process, which is also called postgres, is started by the postgres server process and handles all queries issued by one connected client.
- It communicates with the client by a single TCP connection, and terminates when the client gets disconnected.
- As it is allowed to operate only one database, you have to specify a database you want to use explicitly when connecting to a PostgreSQL server.
- PostgreSQL allows multiple clients to connect simultaneously; the configuration parameter `max_connections` controls the maximum number of the clients (default is 100).
- If many clients such as WEB applications frequently repeat the connection and disconnection with a PostgreSQL server, it increases both costs of establishing connections and of creating backend processes because PostgreSQL has not implemented a native connection pooling feature.
- Such circumstance has a negative effect on the performance of database server. To deal with such a case, a pooling middleware (either `pgbouncer` or `pgpool-II`) is usually used.

Backend Processes

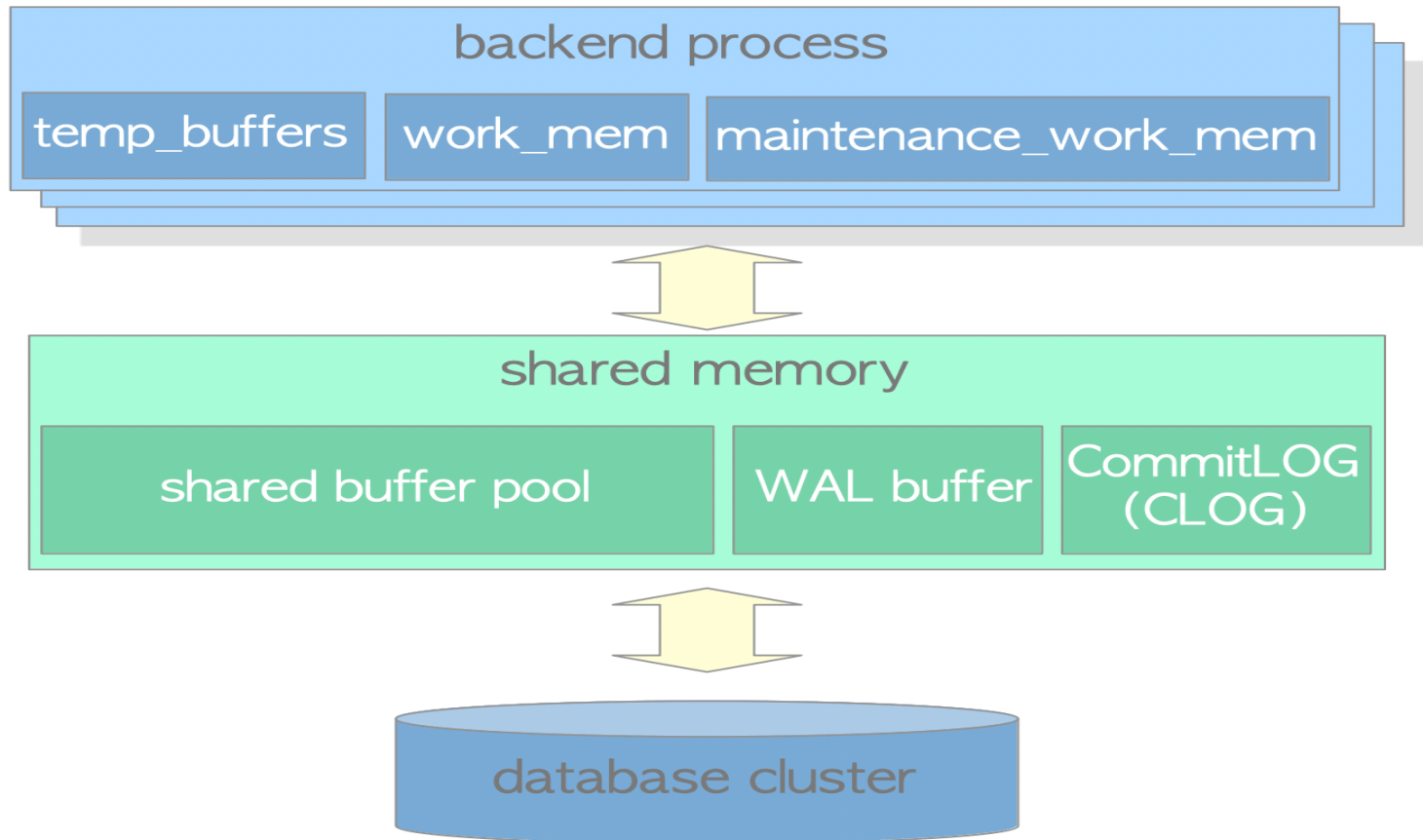
Table 2.1: background processes.

process	description
background writer	In this process, dirty pages on the shared buffer pool are written to a persistent storage (e.g., HDD, SSD) on a regular basis gradually. (In version 9.1 or earlier, it was also responsible for checkpoint process.)
checkpointer	In this process in version 9.2 or later, checkpoint process is performed.
autovacuum launcher	The autovacuum-worker processes are invoked for vacuum process periodically. (More precisely, it requests to create the autovacuum workers to the postgres server.)
WAL writer	This process writes and flushes periodically the WAL data on the WAL buffer to persistent storage.
statistics collector	In this process, statistics information such as for <code>pg_stat_activity</code> and for <code>pg_stat_database</code> , etc. is collected.
logging collector (logger)	This process writes error messages into log files.
archiver	In this process, archiving logging is executed.

Memory Architecture

- Memory architecture in PostgreSQL can be classified into two broad categories:
 - Local memory area – allocated by each backend process for its own use.
 - Shared memory area – used by all processes of a PostgreSQL server.

Memory Architecture



Memory Architecture

- Local Memory Area
 - Each backend process allocates a local memory area for query processing; each area is divided into several sub-areas – whose sizes are either fixed or variable.
- Shared Memory Area
 - A shared memory area is allocated by a PostgreSQL server when it starts up. This area is also divided into several fix sized sub-areas.

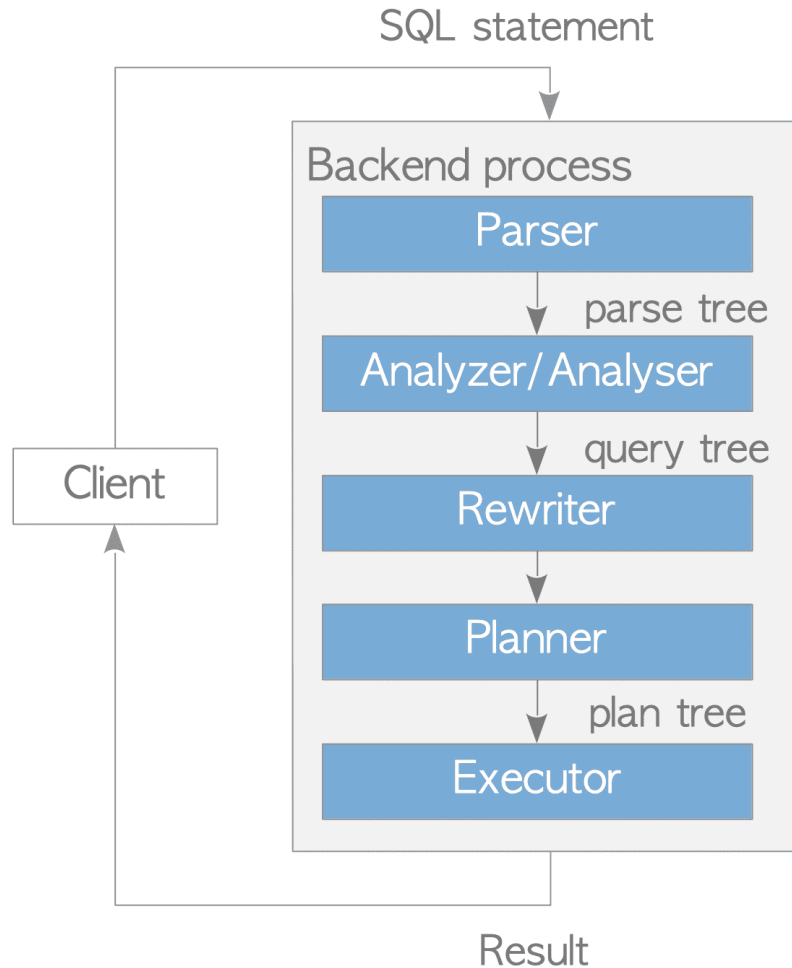
Memory Architecture

sub-area	description
work_mem	Executor uses this area for sorting tuples by ORDER BY and DISTINCT operations, and for joining tables by merge-join and hash-join operations.
maintenance_work_mem	Some kinds of maintenance operations (e.g., VACUUM, REINDEX) use this area.
temp_buffers	Executor uses this area for storing temporary tables.

sub-area	description
shared buffer pool	PostgreSQL loads pages within tables and indexes from a persistent storage to here, and operates them directly.
WAL buffer	To ensure that no data has been lost by server failures, PostgreSQL supports the WAL mechanism. WAL data (also referred to as XLOG records) are transaction log in PostgreSQL; and WAL buffer is a buffering area of the WAL data before writing to a persistent storage.
commit log	Commit Log(CLOG) keeps the states of all transactions (e.g., in_progress, committed, aborted) for Concurrency Control (CC) mechanism.

In addition to them, PostgreSQL allocates several areas as shown below:

Query Processing



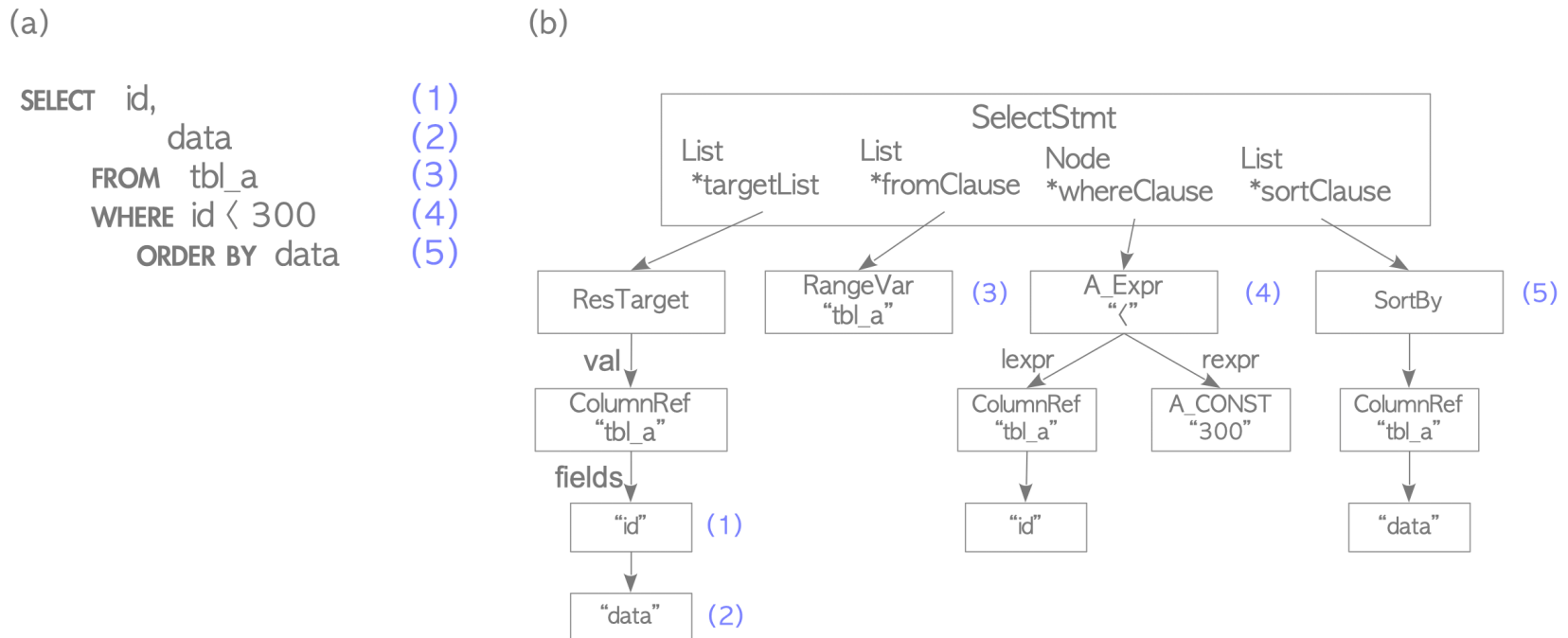
Query Processing

- Parser
 - The parser generates a parse tree from an SQL statement in plain text.
- Analyzer/Analyser
 - The analyzer/analyser carries out a semantic analysis of a parse tree and generates a query tree.
- Rewriter
 - The rewriter transforms a query tree using the rules stored in the rule system if such rules exist.
- Planner
 - The planner generates the plan tree that can most effectively be executed from the query tree.
- Executor
 - The executor executes the query via accessing the tables and indexes in the order that was created by the plan tree.

Query Processing

- Parser

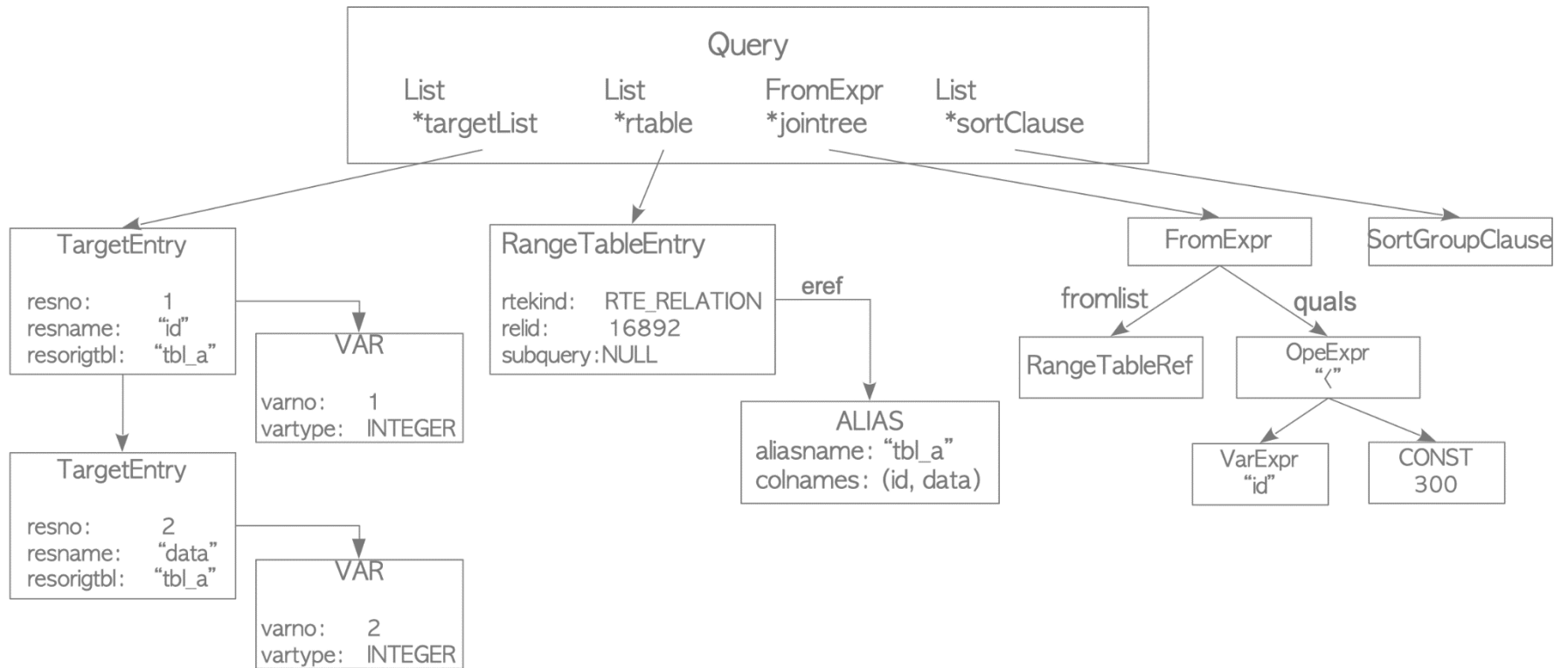
- The parser generates a parse tree that can be read by subsequent subsystems from an SQL statement in plain text.



Query Processing

- Analyzer/Analyser
 - The analyzer/analyser runs a semantic analysis of a parse tree generated by the parser and generates a query tree.
 - The root of a query tree is the Query structure defined in `parsenodes.h`; this structure contains metadata of its corresponding query such as the type of this command (SELECT, INSERT or others) and several leaves; each leaf forms a list or a tree and holds data of the individual particular clause.

Query Processing



Query Processing

- The target list is a list of columns that are the result of this query. In this example, this list is composed of two columns: 'id' and 'data'.
- If the input query tree uses '*' (asterisk), the analyzer/analyser will explicitly replace it to all of the columns.
- The range table is a list of relations that are used in this query.
- In this example, this table holds the information of the table 'tbl_a' such as the oid of this table and the name of this table.
- The join tree stores the FROM clause and the WHERE clauses.
- The sort clause is a list of SortGroupClause.

Architectural Fundamentals

- In database jargon, PostgreSQL uses a client/server model.
- A PostgreSQL session consists of the following cooperating processes (programs):
 - A server process, which manages the database files, accepts connections to the database from client applications, and performs database actions on behalf of the clients.
 - The database server program is called postgres.
 - The user's client (frontend) application that wants to perform database operations.
 - Client applications can be very diverse in nature: a client could be a text-oriented tool, a graphical application, a web server that accesses the database to display web pages, or a specialized database maintenance tool.
 - Some client applications are supplied with the PostgreSQL distribution; most are developed by users.

Architectural Fundamentals

- As is typical of client/server applications, the client and the server can be on different hosts.
- In that case they communicate over a TCP/IP network connection.
- As files can be accessed on a client machine might not be accessible (or might only be accessible using a different file name) on the database server machine.

Architectural Fundamentals

- The PostgreSQL server can handle multiple concurrent connections from clients.
- To achieve this, it starts (“forks”) a new process for each connection.
- From that point on, the client and the new server process communicate without intervention by the original postgres process.
- Thus, the master server process is always running, waiting for client connections, whereas client and associated server processes come and go. (All of this is of course invisible to the user. We only mention it here for completeness.)

Postgre vs Other RDBMS

Feature	PostgreSQL	MySQL	Oracle DB	SQL Server	SQLite
Type	Open-source ORDBMS	Open-source RDBMS	Commercial RDBMS	Commercial RDBMS	Embedded RDBMS
Developer	PostgreSQL Global Dev.	Oracle Corporation	Oracle Corporation	Microsoft	D. Richard Hipp (Public)
ACID Compliance	✓ Full	✓ (depends on engine)	✓ Full	✓ Full	✓ Full
Licensing	PostgreSQL License	GPL	Commercial (proprietary)	Commercial (proprietary)	Public Domain
Platform Support	Cross-platform	Cross-platform	Cross-platform	Windows, Linux, macOS	Cross-platform
Extensibility	✓ Highly Extensible	✗ Limited	✓ with PL/SQL	✓ with .NET CLR	✗ Limited
Stored Procedures	✓ PL/pgSQL	✓ (SQL/PSM, limited)	✓ PL/SQL	✓ T-SQL	✓ Limited

Postgre vs Other RDBMS

Feature	PostgreSQL	MySQL	Oracle DB	SQL Server	SQLite
MVCC Support	✓ Native MVCC	✓ (InnoDB only)	✓	✓	✗ Simulated
JSON & NoSQL Features	✓ Advanced (JSONB, GIN)	✓ Basic JSON support	✗ Basic	✓ Limited	✓ Basic
Replication	✓ Streaming, Logical	✓ Basic (async)	✓ Advanced	✓ Advanced	✗ None
Partitioning	✓ Declarative	✓ (manual or native)	✓ Advanced	✓ Advanced	✗ None
GIS Support	✓ PostGIS	✓ via plugins	✓ Spatial	✓ Spatial	✗ None
Concurrency	✓ Excellent (MVCC)	✓ Good (InnoDB)	✓ Excellent	✓ Good	✗ Poor

PostgreSQL Strengths

- **Advanced SQL Compliance:** Full support for SQL standard, including window functions, CTEs, and more.
- **Extensibility:** You can define custom data types, operators, functions, and index methods.
- **Strong JSON/JSONB:** Can be used for hybrid NoSQL use-cases.
- **Rich Indexing:** Supports B-tree, GiST, GIN, BRIN, Hash, covering a wide range of use-cases.
- **MVCC:** Superior concurrency without locking readers.
- **Open Source & Free for Commercial Use:** No licensing restrictions.

When to Choose What?

Use Case	Recommended DB	Reason
Web applications needing performance + ease	MySQL	Fast, easy, well-supported in LAMP stacks
Enterprise-grade systems with complex logic	Oracle DB	Strong security, clustering, and tooling
Windows-heavy enterprise systems	SQL Server	Good .NET integration, SSIS/SSRS tools
Research, spatial/geospatial apps	PostgreSQL (PostGIS)	Best GIS support, complex query support
Mobile or embedded applications	SQLite	Lightweight, file-based, no server needed
Cloud-native or analytics-heavy systems	PostgreSQL	Modern features, cloud support, and analytics

Security & Role Management

DB	Role Management	Row-Level Security	Encryption
PostgreSQL	✓ Roles, schemas	✓ Built-in	✓ SSL, TDE via extensions
MySQL	✓ Users, privileges	✗ Limited	✓ SSL only
Oracle	✓ Advanced	✓ VPD, Label Sec	✓ TDE, Data Redaction
SQL Server	✓ Windows/SQL Auth	✓ RLS	✓ TDE, Always Encrypted
SQLite	✗ File-based	✗ None	✗ Application-level only

Key Features of PostgreSQL

- **1. ACID Compliance**
 - Guarantees **Atomicity, Consistency, Isolation, and Durability**.
 - Ensures reliable and safe transactions.
- **2. MVCC (Multi-Version Concurrency Control)**
 - Allows **concurrent reads and writes** without locking.
 - Improves performance in high-concurrency environments.

Key Features of PostgreSQL

- **3. Advanced SQL Support**
- Full support for:
 - **Window Functions**
 - **Common Table Expressions (CTEs)**
 - **Subqueries, Joins, and Set operations**
 - **Stored procedures, triggers, and views**

Key Features of PostgreSQL

- **4. Extensibility**
- Add custom:
 - Data types
 - Functions (in PL/pgSQL, Python, Perl, etc.)
 - Operators
 - Indexing methods
 - Procedural languages

Key Features of PostgreSQL

- **5. Rich Data Types**
- Native support for:
 - Numeric, Integer, Boolean, Text, Date/Time
 - Arrays
 - JSON / JSONB (binary-optimized JSON)
 - hstore (key-value pairs)
 - Geometric types
 - UUID, XML, and more

Key Features of PostgreSQL

- **6. Indexing Options**
- Powerful indexing support:
 - **B-tree** (default)
 - **Hash**
 - **GIN** (for full-text search and JSON)
 - **GiST** (for spatial data)
 - **BRIN** (for large sorted datasets)
 - **Expression & Partial indexes**

Key Features of PostgreSQL

- 7. Full Text Search
 - Built-in full-text search capabilities using GIN/GiST indexes.
 - Ranking, highlighting, stemming, and tokenization.
- 8. JSON & NoSQL Support
 - Native JSON and JSONB data types.
 - Querying, indexing, and storing semi-structured data.
 - Acts as a hybrid SQL + NoSQL database.

Key Features of PostgreSQL

- 9. Data Integrity and Constraints
 - Supports:
 - Primary, foreign, and unique keys
 - Check constraints
 - Exclusion constraints (custom conflict rules)
- 10. Partitioning
 - Declarative partitioning on range/list/hash keys.
 - Efficient data management and performance for large tables.

Key Features of PostgreSQL

- **11. Replication & High Availability**
- Built-in:
 - **Streaming replication**
 - **Logical replication**
 - **Hot standby, Point-in-Time Recovery (PITR)**
- Tools: **pgBackRest, Barman, Patroni, PgBouncer**

Key Features of PostgreSQL

- **12. Security Features**

- Role-based authentication and permissions
- SSL encryption
- **Row-Level Security (RLS)**
- Integration with LDAP, GSSAPI, PAM

Key Features of PostgreSQL

- **13. Stored Procedures & Triggers**
 - Functions written in PL/pgSQL or other languages
 - BEFORE, AFTER, INSTEAD OF triggers
 - Event-driven automation and data integrity enforcement
- **14. Internationalization & Localization**
 - Unicode support
 - Collation and case-sensitivity customization
 - Multilingual data storage

Key Features of PostgreSQL

- **15. Tools and Ecosystem**

- GUI: pgAdmin, Dbeaver
- CLI: psql, pg_dump, pg_restore, vacuumdb
- Extensions: PostGIS, pg_stat_statements, TimescaleDB

- **16. Open Source with Strong Community**

- PostgreSQL Global Development Group
- Active mailing lists, regular updates, and transparent roadmap
- Free for commercial and personal use

Writing Simple Queries

- Creating a Database
 - To create a new database, in this example named mydb, you use the following command:
 - `$ create database mydb;`
- Accessing a Database
 - Running the PostgreSQL interactive terminal program, called psql, which allows you to interactively enter, edit, and execute SQL commands.
 - Using an existing graphical frontend tool like pgAdmin or an office suite with ODBC or JDBC support to create and manipulate a database.

Writing Simple Queries

- Accessing a Database
 - \c mydb;

```
mydb=> SELECT version();
```

```
version
```

```
-----  
-----  
PostgreSQL 13.2 on x86_64-pc-linux-gnu, compiled by gcc (Debian  
4.9.2-10) 4.9.2, 64-bit  
(1 row)
```

```
mydb=> SELECT current_date;  
date
```

```
-----  
2016-01-07  
(1 row)
```

```
mydb=> SELECT 2 + 2;  
?column?
```

```
-----  
4  
(1 row)
```


Writing Simple Queries

The `psql` program has a number of internal commands that are not SQL commands. They begin with the backslash character, “\”. For example, you can get help on the syntax of various PostgreSQL SQL commands by typing:

```
mydb=> \h
```

To get out of `psql`, type:

```
mydb=> \q
```

Data Types

Name	Aliases	Description
<code>bigint</code>	<code>int8</code>	signed eight-byte integer
<code>bigserial</code>	<code>serial8</code>	autoincrementing eight-byte integer
<code>bit [(n)]</code>		fixed-length bit string
<code>bit varying [(n)]</code>	<code>varbit</code> <code>[(n)]</code>	variable-length bit string
<code>boolean</code>	<code>bool</code>	logical Boolean (true/false)
<code>box</code>		rectangular box on a plane
<code>bytea</code>		binary data (“byte array”)
<code>character [(n)]</code>	<code>char [(n)]</code>	fixed-length character string
<code>character varying [(n)]</code>	<code>varchar</code> <code>[(n)]</code>	variable-length character string
<code>cidr</code>		IPv4 or IPv6 network address
<code>circle</code>		circle on a plane
<code>date</code>		calendar date (year, month, day)
<code>double precision</code>	<code>float8</code>	double precision floating-point number (8 bytes)
<code>inet</code>		IPv4 or IPv6 host address
<code>integer</code>	<code>int</code> , <code>int4</code>	signed four-byte integer

Data Types

<code>interval [fields] [(p)]</code>		time span
<code>json</code>		textual JSON data
<code>jsonb</code>		binary JSON data, decomposed
<code>line</code>		infinite line on a plane
<code>lseg</code>		line segment on a plane
<code>macaddr</code>		MAC (Media Access Control) address
<code>macaddr8</code>		MAC (Media Access Control) address (EUI-64 format)
<code>money</code>		currency amount
<code>numeric [(p, s)]</code>	<code>decimal [(p, s)]</code>	exact numeric of selectable precision
<code>path</code>		geometric path on a plane
<code>pg_lsn</code>		PostgreSQL Log Sequence Number
<code>pg_snapshot</code>		user-level transaction ID snapshot
<code>point</code>		geometric point on a plane

Data Types

Name	Aliases	Description
<code>polygon</code>		closed geometric path on a plane
<code>real</code>	<code>float4</code>	single precision floating-point number (4 bytes)
<code>smallint</code>	<code>int2</code>	signed two-byte integer
<code>smallserial</code>	<code>serial2</code>	autoincrementing two-byte integer
<code>serial</code>	<code>serial4</code>	autoincrementing four-byte integer
<code>text</code>		variable-length character string
<code>time [(p)] [without time zone]</code>		time of day (no time zone)
<code>time [(p)] with time zone</code>	<code>timetz</code>	time of day, including time zone
<code>timestamp [(p)] [without time zone]</code>		date and time (no time zone)
<code>timestamp [(p)] with time zone</code>	<code>timestampz</code>	date and time, including time zone
<code>tsquery</code>		text search query
<code>tsvector</code>		text search document
<code>txid_snapshot</code>		user-level transaction ID snapshot (deprecated; see <code>pg_snapshot</code>)
<code>uuid</code>		universally unique identifier
<code>xml</code>		XML data

Numeric Types

Name	Storage Size	Description	Range
<code>smallint</code>	2 bytes	small-range integer	-32768 to +32767
<code>integer</code>	4 bytes	typical choice for integer	-2147483648 to +2147483647
<code>bigint</code>	8 bytes	large-range integer	-9223372036854775808 to +9223372036854775807
<code>decimal</code>	variable	user-specified precision, exact	up to 131072 digits before the decimal point; up to 16383 digits after the decimal point
<code>numeric</code>	variable	user-specified precision, exact	up to 131072 digits before the decimal point; up to 16383 digits after the decimal point
<code>real</code>	4 bytes	variable-precision, inexact	6 decimal digits precision
<code>double precision</code>	8 bytes	variable-precision, inexact	15 decimal digits precision
<code>smallserial</code>	2 bytes	small autoincrementing integer	1 to 32767
<code>serial</code>	4 bytes	autoincrementing integer	1 to 2147483647
<code>bigserial</code>	8 bytes	large autoincrementing integer	1 to 9223372036854775807

Monetary Types

Name	Storage Size	Description	Range
money	8 bytes	currency amount	-92233720368547758.08 to +92233720368547758.07

Binary Input Type

Name	Storage Size	Description
bytea	1 or 4 bytes plus the actual binary string	variable-length binary string

Character Types

Name	Description
<code>character varying(n), varchar(n)</code>	variable-length with limit
<code>character(n), char(n)</code>	fixed-length, blank padded
<code>text</code>	variable unlimited length

Boolean Type

Name	Storage Size	Description
boolean	1 byte	state of true or false

Date/Time Types

Name	Storage Size	Description	Low Value	High Value	Resolution
timestamp [(p)] [without time zone]	8 bytes	both date and time (no time zone)	4713 BC	294276 AD	1 microsecond
timestamp [(p)] with time zone	8 bytes	both date and time, with time zone	4713 BC	294276 AD	1 microsecond
date	4 bytes	date (no time of day)	4713 BC	5874897 AD	1 day
time [(p)] [without time zone]	8 bytes	time of day (no date)	00:00:00	24:00:00	1 microsecond
time [(p)] with time zone	12 bytes	time of day (no date), with time zone	00:00:00+1559	24:00:00-1559	1 microsecond
interval [fields] [(p)]	16 bytes	time interval	-178000000 years	178000000 years	1 microsecond

Date/Time Types

Example	Description
1999-01-08	ISO 8601; January 8 in any mode (recommended format)
January 8, 1999	unambiguous in any <code>datestyle</code> input mode
1/8/1999	January 8 in MDY mode; August 1 in DMY mode
1/18/1999	January 18 in MDY mode; rejected in other modes
01/02/03	January 2, 2003 in MDY mode; February 1, 2003 in DMY mode; February 3, 2001 in YMD mode
1999-Jan-08	January 8 in any mode

Time Input

Example	Description
04:05:06.789	ISO 8601
04:05:06	ISO 8601
04:05	ISO 8601
040506	ISO 8601
04:05 AM	same as 04:05; AM does not affect value
04:05 PM	same as 16:05; input hour must be <= 12
04:05:06.789-8	ISO 8601
04:05:06-08:00	ISO 8601
04:05-08:00	ISO 8601
040506-08	ISO 8601
04:05:06 PST	time zone specified by abbreviation
2003-04-12 04:05:06 America/New_York	time zone specified by full name

Oracle Data Types vs Postgre Data Types

Oracle type	Possible PostgreSQL types
CHAR	char, varchar, text
NCHAR	char, varchar, text
VARCHAR	char, varchar, text
VARCHAR2	char, varchar, text, json
NVARCHAR2	char, varchar, text
CLOB	char, varchar, text, json
LONG	char, varchar, text
RAW	uuid, bytea
BLOB	bytea
BFILE	bytea (read-only)
LONG RAW	bytea
NUMBER	numeric, float4, float8, char, varchar, text
NUMBER(n,m) with m<=0	numeric, float4, float8, int2, int4, int8,
	boolean, char, varchar, text
FLOAT	numeric, float4, float8, char, varchar, text

Oracle Data Types vs Postgre Data Types

FLOAT	numeric, float4, float8, char, varchar, text
BINARY_FLOAT	numeric, float4, float8, char, varchar, text
BINARY_DOUBLE	numeric, float4, float8, char, varchar, text
DATE	date, timestamp, timestamptz, char, varchar, text
TIMESTAMP	date, timestamp, timestamptz, char, varchar, text
TIMESTAMP WITH TIME ZONE	date, timestamp, timestamptz, char, varchar, text
TIMESTAMP WITH LOCAL TIME ZONE	
INTERVAL YEAR TO MONTH	interval, char, varchar, text
INTERVAL DAY TO SECOND	interval, char, varchar, text
MDSYS.SDO_GEOMETRY	geometry (see “PostGIS support” below)

Enumerated Type

Declaration of Enumerated Types

Enum types are created using the CREATE TYPE command, for example:

```
CREATE TYPE mood AS ENUM ('sad', 'ok', 'happy');
```

Once created, the enum type can be used in table and function definitions much like any other type:

```
CREATE TYPE mood AS ENUM ('sad', 'ok', 'happy');
CREATE TABLE person (
    name text,
    current_mood mood
);
INSERT INTO person VALUES ('Moe', 'happy');
SELECT * FROM person WHERE current_mood = 'happy';
 name | current_mood
-----+-----
  Moe  |    happy
(1 row)
```

Geometric Types

Name	Storage Size	Description	Representation
point	16 bytes	Point on a plane	(x,y)
line	32 bytes	Infinite line	{A,B,C}
lseg	32 bytes	Finite line segment	((x1,y1),(x2,y2))
box	32 bytes	Rectangular box	((x1,y1),(x2,y2))
path	16+16n bytes	Closed path (similar to polygon)	((x1,y1),...)
path	16+16n bytes	Open path	[(x1,y1),...]
polygon	40+16n bytes	Polygon (similar to closed path)	((x1,y1),...)
circle	24 bytes	Circle	<(x,y),r> (center point and radius)

Geometric Types

```
CREATE EXTENSION postgis;
```

```
CREATE TABLE geometries (name varchar, geom geometry);
```

```
INSERT INTO geometries VALUES
```

```
  ('Point', 'POINT(0 0)'),
```

```
  ('Linestring', 'LINESTRING(0 0, 1 1, 2 1, 2 2)'),
```

```
  ('Polygon', 'POLYGON((0 0, 1 0, 1 1, 0 1, 0 0))'),
```

```
  ('PolygonWithHole', 'POLYGON((0 0, 10 0, 10 10, 0 10, 0 0),(1 1, 1 2, 2 2, 2 1, 1 1))'),
```

```
  ('Collection', 'GEOMETRYCOLLECTION(POINT(2 0),POLYGON((0 0, 1 0, 1 1, 0 1, 0 0)))');
```

```
SELECT name, ST_AsText(geom) FROM geometries;
```

Network Address Types

Name	Storage Size	Description
<code>cidr</code>	7 or 19 bytes	IPv4 and IPv6 networks
<code>inet</code>	7 or 19 bytes	IPv4 and IPv6 hosts and networks
<code>macaddr</code>	6 bytes	MAC addresses
<code>macaddr8</code>	8 bytes	MAC addresses (EUI-64 format)

Network Address Types

```
CREATE TABLE inet_test (  
    address INET  
);
```

```
INSERT INTO inet_test (address) VALUES  
( '198.24.10.0/24' );
```

```
INSERT INTO inet_test (address) VALUES  
( '198.24.10.0' );
```

```
INSERT INTO inet_test (address) VALUES ( '198.10/8' );
```

cidr Type Input

cidr Input	cidr Output	abbrev(cidr)
192.168.100.128/25	192.168.100.128/25	192.168.100.128/25
192.168/24	192.168.0.0/24	192.168.0/24
192.168/25	192.168.0.0/25	192.168.0.0/25
192.168.1	192.168.1.0/24	192.168.1/24
192.168	192.168.0.0/24	192.168.0/24
128.1	128.1.0.0/16	128.1/16
128	128.0.0.0/16	128.0/16
128.1.2	128.1.2.0/24	128.1.2/24
10.1.2	10.1.2.0/24	10.1.2/24
10.1	10.1.0.0/16	10.1/16
10	10.0.0.0/8	10/8
10.1.2.3/32	10.1.2.3/32	10.1.2.3/32
2001:4f8:3:ba::/64	2001:4f8:3:ba::/64	2001:4f8:3:ba/64
2001:4f8:3:ba: 2e0:81ff:fe22:d1f1/128	2001:4f8:3:ba: 2e0:81ff:fe22:d1f1/128	2001:4f8:3:ba: 2e0:81ff:fe22:d1f1/128
::ffff:1.2.3.0/120	::ffff:1.2.3.0/120	::ffff:1.2.3/120
::ffff:1.2.3.0/128	::ffff:1.2.3.0/128	::ffff:1.2.3.0/128

Bit String Types

- Bit strings are strings of 1's and 0's. They can be used to store or visualize bit masks.
- There are two SQL bit types: `bit(n)` and `bit varying(n)`, where `n` is a positive integer.
- `bit` type data must match the length `n` exactly; it is an error to attempt to store shorter or longer bit strings.
- `bit varying` data is of variable length up to the maximum length `n`; longer strings will be rejected.
- Writing `bit` without a length is equivalent to `bit(1)`, while `bit varying` without a length specification means unlimited length.

Bit String Types

```
CREATE TABLE test (a BIT(3), b BIT VARYING(5));  
INSERT INTO test VALUES (B'101', B'00');  
INSERT INTO test VALUES (B'10', B'101');
```

ERROR: bit string length 2 does not match type bit(3)

```
INSERT INTO test VALUES (B'10'::bit(3), B'101');  
SELECT * FROM test;
```

UUID Type

- The data type uuid stores Universally Unique Identifiers (UUID) as defined by RFC 4122, ISO/IEC 9834-8:2005, and related standards.
- (This identifier is a 128-bit quantity that is generated by an algorithm chosen to make it very unlikely that the same identifier will be generated by anyone else in the known universe using the same algorithm.
- Therefore, for distributed systems, these identifiers provide a better uniqueness guarantee than sequence generators, which are only unique within a single database.

UUID Type

- PostgreSQL also accepts the following alternative forms for input: use of upper-case digits, the standard format surrounded by braces, omitting some or all hyphens, adding a hyphen after any group of four digits.
- Examples are:
- A0EEBC99-9C0B-4EF8-BB6D-6BB9BD380A11
- {a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11}
- a0eebc999c0b4ef8bb6d6bb9bd380a11
- a0ee-bc99-9c0b-4ef8-bb6d-6bb9-bd38-0a11
- {a0eebc99-9c0b4ef8-bb6d6bb9-bd380a11}

UUID Type

```
CREATE EXTENSION IF NOT EXISTS "uuid-ossf";
SELECT uuid_generate_v1();
SELECT uuid_generate_v4();
CREATE TABLE contacts (
    contact_id uuid DEFAULT uuid_generate_v4 (),
    first_name VARCHAR NOT NULL,
    last_name VARCHAR NOT NULL,
    email VARCHAR NOT NULL,
    phone VARCHAR,
    PRIMARY KEY (contact_id)
);
```

UUID Type

```
INSERT INTO contacts (  
    first_name,  
    last_name,  
    email,  
    phone  
)  
VALUES  
(  
    'John',  
    'Smith',  
    'john.smith@example.com',  
    '408-237-2345'  
)
```

UUID Type

```
(  
    'Jane',  
    'Smith',  
    'jane.smith@example.com',  
    '408-237-2344'  
) ,  
(  
    'Alex',  
    'Smith',  
    'alex.smith@example.com',  
    '408-237-2343'  
);
```

XML Type

- The xml data type can be used to store XML data.
- Its advantage over storing XML data in a text field is that it checks the input values for well-formedness, and there are support functions to perform type-safe operations on it;
- Use of this data type requires the installation to have been built with `configure --with-libxml`.

XML Type

- The xml type can store well-formed “documents”, as defined by the XML standard, as well as “content” fragments, which are defined by reference to the more permissive “document node”¹ of the XQuery and XPath data model.
- Roughly, this means that content fragments can have more than one top-level element or character node.
- The expression `xmlvalue IS DOCUMENT` can be used to evaluate whether a particular xml value is a full document or only a content fragment.

XML Type

To produce a value of type `xml` from character data, use the function `xmlparse`:

```
XMLPARSE ( { DOCUMENT | CONTENT } value)
```

Examples:

```
XMLPARSE (DOCUMENT '<?xml version="1.0"?><book><title>Manual</title><chapter>...</chapter></book>')
XMLPARSE (CONTENT 'abc<foo>bar</foo><bar>foo</bar>')
```

While this is the only way to convert character strings into XML values according to the SQL standard, the PostgreSQL-specific syntaxes:

```
xml '<foo>bar</foo>'
'<foo>bar</foo>'::xml
```

can also be used.

Xml Type

The `xml` type does not validate input values against a document type declaration (DTD), even when the input value specifies a DTD. There is also currently no built-in support for validating against other XML schema languages such as XML Schema.

The inverse operation, producing a character string value from `xml`, uses the function `xmlserialize`:

```
XMLSERIALIZE ( { DOCUMENT | CONTENT } value AS type )
```

`type` can be `character`, `character varying`, or `text` (or an alias for one of those). Again, according to the SQL standard, this is the only way to convert between type `xml` and character types, but PostgreSQL also allows you to simply cast the value.

When a character string value is cast to or from type `xml` without going through `XMLPARSE` or `XMLSERIALIZE`, respectively, the choice of `DOCUMENT` versus `CONTENT` is determined by the “XML option” session configuration parameter, which can be set using the standard command:

```
SET XML OPTION { DOCUMENT | CONTENT } ;
```

or the more PostgreSQL-like syntax

```
SET xmloption TO { DOCUMENT | CONTENT } ;
```

The default is `CONTENT`, so all forms of XML data are allowed.

JSON Types

- JSON data types are for storing JSON (JavaScript Object Notation) data, as specified in RFC 71592.
- Such data can also be stored as text, but the JSON data types have the advantage of enforcing that each stored value is valid according to the JSON rules.
- There are also assorted JSON-specific functions and operators available for data stored in these data types;
- PostgreSQL offers two types for storing JSON data: json and jsonb.

JSON Types

- The json and jsonb data types accept almost identical sets of values as input.
- The major practical difference is one of efficiency.
- The json data type stores an exact copy of the input text, which processing functions must reparse on each execution; while jsonb data is stored in a decomposed binary format that makes it slightly slower to input due to added conversion overhead, but significantly faster to process, since no reparsing is needed.
- jsonb also supports indexing, which can be a significant advantage.

JSON Types

- Because the json type stores an exact copy of the input text, it will preserve semantically-insignificant white space between tokens, as well as the order of keys within JSON objects.
- Also, if a JSON object within the value contains the same key more than once, all the key/value pairs are kept.
- By contrast, jsonb does not preserve white space, does not preserve the order of object keys, and does not keep duplicate object keys.
- If duplicate keys are specified in the input, only the last value is kept.
- In general, most applications should prefer to store JSON data as jsonb, unless there are quite specialized needs, such as legacy assumptions about ordering of object keys.

JSON Types

JSON primitive type	PostgreSQL type	Notes
string	text	\u0000 is disallowed, as are Unicode escapes representing characters not available in the database encoding
number	numeric	NaN and infinity values are disallowed
boolean	boolean	Only lowercase true and false spellings are accepted
null	(none)	SQL NULL is a different concept

Arrays

```
CREATE TABLE sal_emp (  
    name                text,  
    pay_by_quarter      integer[],  
    schedule             text[]  
);
```

```
INSERT INTO sal_emp  
VALUES ('Bill',  
    '{10000, 10000, 10000, 10000}',  
    '{{"meeting", "lunch"}, {"training", "presentation"}}');
```

```
INSERT INTO sal_emp  
VALUES ('Carol',  
    '{20000, 25000, 25000, 25000}',  
    '{{"breakfast", "consulting"}, {"meeting", "lunch"}}');
```

Sequence

- CREATE [TEMPORARY | TEMP] SEQUENCE [IF NOT EXISTS] name
- [AS data_type]
- [INCREMENT [BY] increment]
- [MINVALUE minvalue | NO MINVALUE] [MAXVALUE maxvalue | NO MAXVALUE]
- [START [WITH] start] [CACHE cache] [[NO] CYCLE]
- [OWNED BY { table_name.column_name | NONE }]

Sequence Parameters

- **TEMPORARY** or **TEMP** — PostgreSQL can create a temporary sequence within a session. Once the session ends, the sequence is automatically dropped.
- **IF NOT EXISTS** — Creates a sequence even if a sequence with an identical name already exists. Replaces the existing sequence.
- **AS** — A new option in PostgreSQL 10. It is for specifying the data type of the sequence. The available options are `smallint`, `integer`, and `bigint` (default). This also determines the maximum and minimum values.
- **INCREMENT BY** — An optional parameter with a default value of 1. Positive values generate sequence values in ascending order. Negative values generate sequence values in descending sequence.

Sequence Parameters

- **START WITH** — The same as Oracle. This is an optional parameter having a default of 1. It uses the MINVALUE for ascending sequences and the MAXVALUE for descending sequences.
- **MAXVALUE | NO MAXVALUE** — Defaults are between 263 for ascending sequences and -1 for descending sequences.
- **MINVALUE | NO MINVALUE** — Defaults are between 1 for ascending sequences and -263 for descending sequences.
- **CYCLE | NO CYCLE** — If the sequence value reaches MAXVALUE or MINVALUE, the CYCLE parameter instructs the sequence to return to the initial value (MINVALUE or MAXVALUE). The default is NO CYCLE.

Sequence Parameters

- **CACHE** — Note that in PostgreSQL, the **NOCACHE** isn't supported. By default, when not specifying the **CACHE** parameter, no sequence values will be pre-cached into memory, which is equivalent to the Oracle **NOCACHE** parameter. The minimum value is 1.
- **OWNED BY | OWNBY NON** — Specifies that the sequence object is to be associated with a specific column in a table, which isn't supported by Oracle. When dropping this type of sequence, an error will be returned because of the sequence/table association.

Arrays

The result of the previous two inserts looks like this:

```
SELECT * FROM sal_emp;
  name |          pay_by_quarter          |          schedule
-----+-----
+-----
  Bill | {10000,10000,10000,10000} | {{meeting,lunch},
{training,presentation}}
  Carol | {20000,25000,25000,25000} | {{breakfast,consulting},
{meeting,lunch}}
(2 rows)
```

Accessing Arrays

```
SELECT name FROM sal_emp WHERE pay_by_quarter[1] <>
    pay_by_quarter[2];
```

```
name
-----
Carol
(1 row)
```

```
SELECT pay_by_quarter[3] FROM sal_emp;
```

```
pay_by_quarter
-----
                10000
                25000
(2 rows)
```

```
SELECT schedule[1:2][1:1] FROM sal_emp WHERE name = 'Bill';
```

```
schedule
-----
{{meeting},{training}}
(1 row)
```

Accessing Arrays

```
SELECT schedule[1:2][2] FROM sal_emp WHERE name = 'Bill';
```

It is possible to omit the *lower-bound* and/or *upper-bound* of a slice specifier; the missing bound is replaced by the lower or upper limit of the array's subscripts. For example:

```
SELECT schedule[:2][2:] FROM sal_emp WHERE name = 'Bill';
```

```
          schedule
-----
{{lunch},{presentation}}
(1 row)
```

```
SELECT schedule[:] [1:1] FROM sal_emp WHERE name = 'Bill';
```

```
          schedule
-----
{{meeting},{training}}
(1 row)
```

Accessing Arrays

```
SELECT array_dims(schedule) FROM sal_emp WHERE name = 'Carol';
```

```
array_dims
-----
[1:2] [1:2]
(1 row)
```

```
SELECT array_length(schedule, 1) FROM sal_emp WHERE name = 'Carol';
```

```
array_length
-----
2
(1 row)
```

`cardinality` returns the total number of elements in an array across all dimensions. It is effectively the number of rows a call to `unnest` would yield:

```
SELECT cardinality(schedule) FROM sal_emp WHERE name = 'Carol';
```

```
cardinality
-----
4
(1 row)
```

Modifying Arrays

```
UPDATE sal_emp SET pay_by_quarter = '{25000,25000,27000,27000}'  
WHERE name = 'Carol';
```

or using the ARRAY expression syntax:

```
UPDATE sal_emp SET pay_by_quarter = ARRAY[25000,25000,27000,27000]  
WHERE name = 'Carol';
```

An array can also be updated at a single element:

```
UPDATE sal_emp SET pay_by_quarter[4] = 15000  
WHERE name = 'Bill';
```

or updated in a slice:

```
UPDATE sal_emp SET pay_by_quarter[1:2] = '{27000,27000}'  
WHERE name = 'Carol';
```

Modifying Arrays

```
SELECT ARRAY[1,2] || ARRAY[3,4];
      ?column?
-----
 {1,2,3,4}
(1 row)
```

```
SELECT ARRAY[5,6] || ARRAY[[1,2],[3,4]];
      ?column?
-----
 {{5,6},{1,2},{3,4}}
(1 row)
```

```
SELECT array_dims(ARRAY[1,2] || ARRAY[3,4,5]);
      array_dims
-----
 [1:5]
(1 row)
```

```
SELECT array_dims(ARRAY[[1,2],[3,4]] || ARRAY[[5,6],[7,8],[9,0]]);
      array_dims
-----
 [1:5][1:2]
(1 row)
```

Modifying Arrays

```
SELECT array_prepend(1, ARRAY[2,3]);
       array_prepend
-----
 {1,2,3}
(1 row)
```

```
SELECT array_append(ARRAY[1,2], 3);
       array_append
-----
 {1,2,3}
(1 row)
```

```
SELECT array_cat(ARRAY[1,2], ARRAY[3,4]);
       array_cat
-----
 {1,2,3,4}
(1 row)
```

```
SELECT array_cat(ARRAY[[1,2],[3,4]], ARRAY[5,6]);
       array_cat
-----
 {{1,2},{3,4},{5,6}}
(1 row)
```

```
SELECT array_cat(ARRAY[5,6], ARRAY[[1,2],[3,4]]);
       array_cat
-----
 {{5,6},{1,2},{3,4}}
```

Searching in Arrays

To search for a value in an array, each value must be checked. This can be done manually, if you know the size of the array. For example:

```
SELECT * FROM sal_emp WHERE pay_by_quarter[1] = 10000 OR  
                             pay_by_quarter[2] = 10000 OR  
                             pay_by_quarter[3] = 10000 OR  
                             pay_by_quarter[4] = 10000;
```

However, this quickly becomes tedious for large arrays, and is not helpful if the size of the array is unknown. An alternative method is described in Section 9.24. The above query could be replaced by:

```
SELECT * FROM sal_emp WHERE 10000 = ANY (pay_by_quarter);
```

In addition, you can find rows where the array has all values equal to 10000 with:

```
SELECT * FROM sal_emp WHERE 10000 = ALL (pay_by_quarter);
```


Searching in Arrays

```
SELECT
  array_position(ARRAY['sun','mon','tue','wed','thu','fri','sat'],
    'mon');
array_position
-----
                2

(1 row)
```

```
SELECT array_positions(ARRAY[1, 4, 3, 1, 3, 4, 2, 1], 1);
array_positions
-----
{1,4,8}
(1 row)
```

Composite Types

- A composite type represents the structure of a row or record; it is essentially just a list of field names and their data types.
- PostgreSQL allows composite types to be used in many of the same ways that simple types can be used.
- For example, a column of a table can be declared to be of a composite type.

Declaration of Composite Types

```
CREATE TYPE complex AS (  
    r          double precision,  
    i          double precision  
);  
  
CREATE TYPE inventory_item AS (  
    name          text,  
    supplier_id   integer,  
    price         numeric  
);
```

Constructing Composite Values

- To write a composite value as a literal constant, enclose the field values within parentheses and separate them by commas.
- We can put double quotes around any field value, and must do so if it contains commas or parentheses.
- Thus, the general format of a composite constant is the following:
- `'( val1 , val2 , ... )'`

Constructing Composite Values

An example is:

```
' ("fuzzy dice",42,1.99) '
```

which would be a valid value of the `inventory_item` type defined above. To make a field be NULL, write no characters at all in its position in the list. For example, this constant specifies a NULL third field:

```
' ("fuzzy dice",42,) '
```

If you want an empty string rather than NULL, write double quotes:

```
' ("",42,) '
```

Here the first field is a non-NULL empty string, the third is NULL.

Accessing Composite Types

```
SELECT item.name FROM on_hand WHERE item.price > 9.99;
```

This will not work since the name `item` is taken to be a table name, not a column name of `on_hand`, per SQL syntax rules. You must write it like this:

```
SELECT (item).name FROM on_hand WHERE (item).price > 9.99;
```

or if you need to use the table name as well (for instance in a multitable query), like this:

```
SELECT (on_hand.item).name FROM on_hand WHERE (on_hand.item).price  
> 9.99;
```

Built in Range Types

- `int4range` — Range of integer
- `int8range` — Range of bigint
- `numrange` — Range of numeric
- `tsrange` — Range of timestamp without time zone
- `tstzrange` — Range of timestamp with time zone
- `daterange` — Range of date

Built in Range Types

```
CREATE TABLE reservation (room int, during tsrange);
INSERT INTO reservation VALUES
    (1108, '[2010-01-01 14:30, 2010-01-01 15:30)');

-- Containment
SELECT int4range(10, 20) @> 3;

-- Overlaps
SELECT numrange(11.1, 22.2) && numrange(20.0, 30.0);

-- Extract the upper bound
SELECT upper(int8range(15, 25));

-- Compute the intersection
SELECT int4range(10, 20) * int4range(15, 25);

-- Is the range empty?
SELECT isempty(numrange(1, 5));
```


PostgreSQL Full Text Search Entities

- Document: A set of data on which your full-text search will take place is called Document.
- In PostgreSQL, you can create a document using either a single column or multiple columns or even using multiple tables.
- Lexemes: The PostgreSQL document during processing is parsed into multiple tokens, which consist of words, sentences, etc. of the text present in the document.
- These tokens are modified to form more meaningful entities called lexemes.

PostgreSQL Full Text Search Entities

- Dictionaries: The conversion of Tokens into Lexemes is done using dictionaries.
- PostgreSQL offers both options of either using the built-in dictionaries or creating your own dictionaries.
- These dictionaries are responsible to determine which stop words are not important and whether differently derived words have the same base language.

to_tsvector

- To create a vector for a sentence, you can use the `to_tsvector` function as follows:
- `SELECT to_tsvector('The quick brown fox jumped over the lazy dog.');`
- `to_tsvector -----'brown':3 'dog':9 'fox':4 'jump':5 'lazi':8 'quick':2`

to_tsquery

- This function accepts a list of words that would be checked against the normalized vector created with the `to_tsvector()` function.
- You can use the following Postgres functions to convert text fields to tsquery values such as `to_tsquery`, `plainto_tsquery`, and `phraseto_tsquery`.
- `SELECT to_tsquery('The quick brown fox jumped over the lazy dog')`

Searching

- After creating tsvector from your text data and tsquery from your search terms, you can further perform a full-text search using the @@ operator.
- @@ operator checks if tsquery matches tsvector.
- For instance, if the word to be queried is “fox” for the above-mentioned example, then:
- `SELECT to_tsvector('The quick brown fox jumped over the lazy dog') @@ to_tsquery('fox');`
- Refer courses table in queries.txt

The SQL Language

- Creating a New Table

```
CREATE TABLE weather (  
  city varchar(80),  
  temp_lo int, -- low temperature  
  temp_hi int, -- high temperature  
  prcp real, -- precipitation  
  date date  
);
```

Populating a Table With Rows

- The INSERT statement is used to populate a table with rows:
- `INSERT INTO weather VALUES ('San Francisco', 46, 50, 0.25, '1994-11-27');`
- Note that all data types use rather obvious input formats.
- Constants that are not simple numeric values usually must be surrounded by single quotes ('), as in the example

What is Normalization?

- **Normalization** is the process of organizing data in a database to:
 - Eliminate **redundant data** (repetition).
 - Ensure **data dependencies** make sense.
 - Improve **data integrity**.

Goals of Normalization

- Minimize duplication of data.
- Protect the database against logical or structural anomalies.
- Make database maintenance easier and more efficient.

Normal Forms (NF)

- Normalization is done through a series of **normal forms**, each building on the previous:
-  **1. First Normal Form (1NF)**
- **Atomic values** only (no repeating groups or arrays).
- Each column should contain **unique and indivisible** values.

Normal Forms (NF)

- Normalization is done through a series of **normal forms**, each building on the previous:
-  **1. First Normal Form (1NF)**
- **Atomic values** only (no repeating groups or arrays).
- Each column should contain **unique and indivisible** values.

First Normal Form

Student	Subjects
John	Math, Science

Student	Subject
John	Math
John	Science

2. Second Normal Form (2NF)

- 1NF + every non-key attribute is fully dependent on the entire primary key (no partial dependency).
- Applies only to tables with composite keys.

Violation Example:

OrderID	ProductID	ProductName
---------	-----------	-------------

✓ Fix:

Split into:

- Orders (OrderID, ProductID)
- Products (ProductID, ProductName)

Third Normal Form (3NF)

- 2NF + no transitive dependencies (non-key attribute depends on another non-key attribute).

Violation Example:

| EmployeeID | DepartmentID | DepartmentName |

✓ Fix:

- Employee (EmployeeID, DepartmentID)
- Department (DepartmentID, DepartmentName)

Boyce-Codd Normal Form (BCNF)

- BCNF (Boyce-Codd Normal Form) is a stronger version of the Third Normal Form (3NF).
- **Rule:**
A relation is in **BCNF** if and only if every **determinant** is a **candidate key**.
- A **determinant** is any attribute (or set of attributes) on which some other attribute is **fully functionally dependent**.
- A **candidate key** is a minimal set of attributes that can uniquely identify a tuple.

Example That Is in 3NF but Not in BCNF

Course	Instructor	Room
DBMS	Alice	R1
OS	Bob	R2
DBMS	Alice	R1

Functional Dependencies (FDs):

- Course $\rightarrow$ Instructor  (Each course has one instructor)
- Instructor $\rightarrow$ Course  Not always true
- Instructor $\rightarrow$ Room  (Each instructor has a fixed room)

Why It's in 3NF but Not in BCNF?

- All non-key attributes are dependent on keys, so it satisfies 3NF.
- But Instructor $\rightarrow$ Room violates BCNF because:
 - Instructor is not a candidate key, but it's a determinant.
- Hence, not in BCNF.

Fix: Decompose into BCNF

Split the table into **two relations**:

◆ Table 1: Instructor_Room

Instructor	Room
Alice	R1
Bob	R2

FD: Instructor $\rightarrow$ Room  Instructor is the key here.

Fix: Decompose into BCNF

◆ Table 2: Course_Instructor

Course	Instructor
DBMS	Alice
OS	Bob

FD: Course $\rightarrow$ Instructor  Course is the key here.

Higher Normal Forms (4NF, 5NF)

- Handle multi-valued dependencies and join dependencies.
- Rarely used in everyday business applications but important in theoretical or complex systems.

Example: Joining employee and department tables

dept_id	dept_name
1	HR
2	IT
3	Finance

Example: Joining employee and department tables

emp_id	emp_name	dept_id
101	Alice	2
102	Bob	1
103	Charlie	3
104	David	NULL

Create Tables with Constraints

-- 1. Create Department table

```
CREATE TABLE IF NOT EXISTS department (  
    dept_id SERIAL PRIMARY KEY,  
    dept_name VARCHAR(100) NOT NULL  
);
```

-- 2. Create Employee table with Foreign Key referencing Department

```
CREATE TABLE IF NOT EXISTS employee (  
    emp_id SERIAL PRIMARY KEY,  
    emp_name VARCHAR(100) NOT NULL,  
    dept_id_fk INTEGER,  
    CONSTRAINT fk_department  
        FOREIGN KEY (dept_id_fk)  
        REFERENCES department(dept_id)  
        ON DELETE SET NULL  
        ON UPDATE CASCADE  
);
```


Create .sql file

- Sudo -i -u postgres
- Psql
- \i /home/ubuntu-user/Documents/Labs/Lab01/script.sql

Describe Table

- \d department

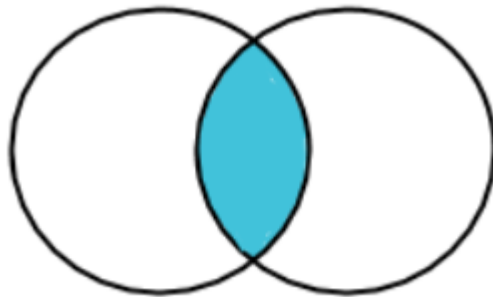
```
Table "public.department"
Column | Type | Collation | Nullable | Default
-----+-----+-----+-----+-----
dept_id | integer | | not null | nextval('department_dept_id_seq'::regclass)
dept_name | text | | not null |
Indexes:
    "department_pkey" PRIMARY KEY, btree (dept_id)
```

Insert into table

- Insert into department(dept_name) values('HR');
- Insert into
employee(emp_name,dept_id_fk)values('param',1);

Joins

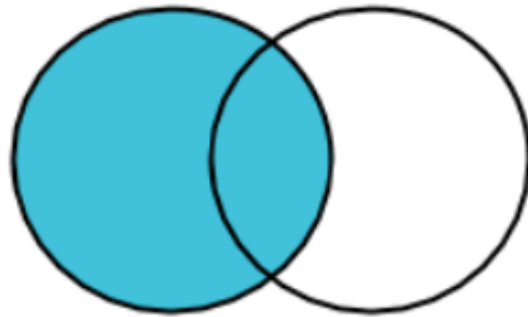
- **INNER JOIN:** Match rows in both tables



```
SELECT e.emp_id, e.emp_name, d.dept_name  
FROM employee e  
INNER JOIN department d ON e.dept_id_fk = d.dept_id;
```

Joins

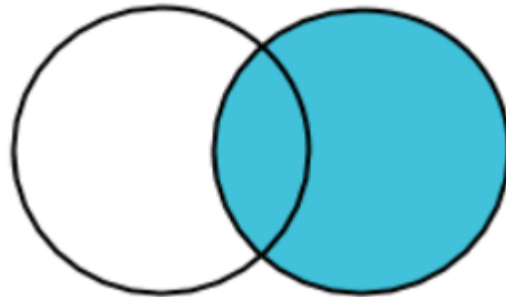
- **LEFT JOIN: All rows from left + matched rows from right**



```
SELECT e.emp_id, e.emp_name, d.dept_name  
FROM employee e  
LEFT JOIN department d ON e.dept_id = d.dept_id;
```

Joins

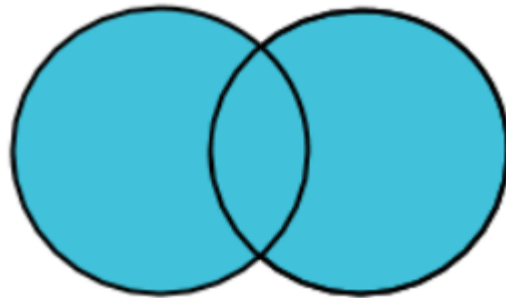
- **RIGHT JOIN: All rows from right + matched rows from left**



```
SELECT e.emp_id, e.emp_name, d.dept_name  
FROM employee e  
RIGHT JOIN department d ON e.dept_id = d.dept_id;
```

Joins

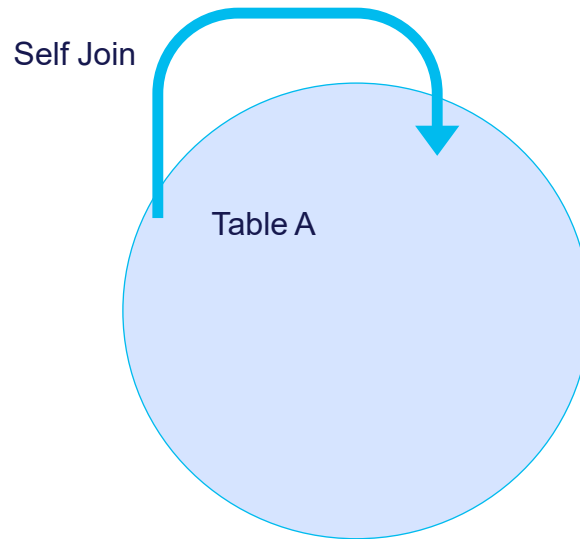
- **FULL OUTER JOIN: All rows from both tables, with NULLs where no match**



```
SELECT e.emp_id, e.emp_name, d.dept_name  
FROM employee e  
FULL OUTER JOIN department d ON e.dept_id = d.dept_id;
```

Joins

- **Self Join: Join a table with itself (e.g., employee → manager)**



Joins – Self Join

- **CREATE TABLE employee (**
- **emp_id SERIAL PRIMARY KEY,**
- **emp_name VARCHAR(100),**
- **manager_id INTEGER -- References another employee**
- **);**

Joins – Self Join

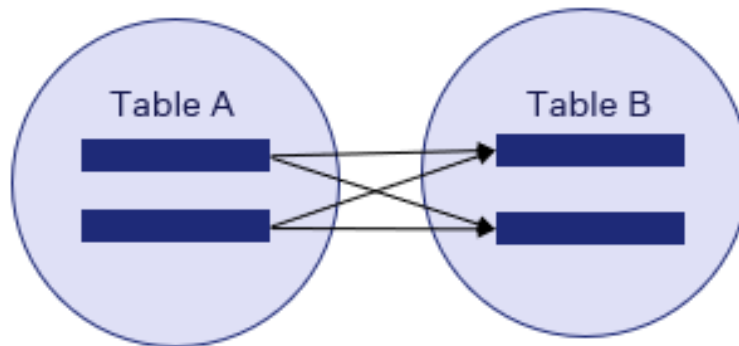
employee_id	employee_name	manager_name
1	Alice	NULL
2	Bob	Alice
3	Charlie	Alice
4	David	Bob

Joins – Self Join

- **SELECT**
- **e.emp_id AS employee_id,**
- **e.emp_name AS employee_name,**
- **m.emp_name AS manager_name**
- **FROM employee e**
- **LEFT JOIN employee m ON e.manager_id = m.emp_id;**

Joins

- **Cross Join: Cartesian product of two tables**



```
SELECT e.emp_name, d.dept_name  
FROM employee e  
CROSS JOIN department d;
```

Join with subquery

- **Get employees only from the IT department, using a subquery in the join:**
- `SELECT e.emp_id, e.emp_name, d.dept_name`
- `FROM employee e`
- `JOIN (`
- `SELECT dept_id, dept_name`
- `FROM department`
- `WHERE dept_name = 'IT'`
- `) AS d ON e.dept_id = d.dept_id;`

Joins Summary

Join Type	Matching Rows	Non-Matching Rows
INNER	✓	✗
LEFT	✓ (left)	NULLs from right
RIGHT	✓ (right)	NULLs from left
FULL	✓	NULLs from both

Views

- A VIEW is a pseudo table in PostgreSQL; it is not a solid table but appears as an ordinary table to select.
- A view can also represent joined tables.
- It can contain all rows of a table or selected rows from one or more tables.
- A View simplifies users to perform the following aspects:
 - It structures data naturally and intuitively and makes it easy to find.
 - We can authorize permission to users over a view, which has a complete record that the users are authorized to see.
 - It restricts access to the data such that users can only see limited data instead of complete data.
 - A view provides a dependable layer, even the columns of necessary table modification.
 - It summarizes data from various tables to generate reports.
 - A view helps us to describe the difficulty of a statement as we can write a command of view based on a complex query with the help of a SELECT command.

Views

CREATE [OR REPLACE] VIEW view-name AS

SELECT column(s)

FROM table(s)

[WHERE condition(s)];

CREATE VIEW Price_View AS

SELECT id, price

FROM Price

WHERE price > 200;

Clear the screen

- `\! cls`

EXPLAIN – See the Query Plan

- Shows how PostgreSQL will execute a query, including:
 - Join types
 - Index usage
 - Cost estimates (not actual execution)

```
EXPLAIN SELECT * FROM employee WHERE dept_id = 2;
```

EXPLAIN ANALYZE – Run the Query and Show Actual Execution Details

- It **executes the query** and shows:
 - Actual time taken per step
 - Rows processed
 - Loops
 - Index usage
 - Total execution time

```
EXPLAIN ANALYZE SELECT * FROM employee WHERE dept_id = 2;
```

```
Seq Scan on employee (cost=0.00..25.00 rows=5 width=32) (actual  
time=0.021..0.023 rows=2 loops=1)
```

```
  Filter: (dept_id = 2)
```

```
    Rows Removed by Filter: 3
```

```
Planning Time: 0.100 ms
```

```
Execution Time: 0.050 ms
```

Key Terms Explained:

Term	Meaning
Seq Scan	Sequential scan of the table (may be slow on large tables)
Index Scan	Uses an index – usually faster
cost=X..Y	Estimated startup cost and total cost
rows=N	Estimated number of rows
actual time	Actual time to fetch rows
loops	How many times this step was executed
Rows Removed by Filter	Rows read but discarded by the WHERE clause

Best Practices

- Use EXPLAIN for safe preview.
- Use EXPLAIN ANALYZE to benchmark real execution.
- If the query is slow, look for:
 - Sequential scan instead of index scan
 - High number of loops
 - Large difference between estimated and actual rows

Explain and Analyze

```
public | profile | table | enterprisedb
public | starrating | table | admin
(5 rows)
```

```
accountdb=# explain analyze select * from producttransaction;
               QUERY PLAN
```

```
-----
Seq Scan on producttransaction (cost=0.00..1.02 rows=2 width=143) (actual time=10.385..10.388 rows=2 loops=1)
Planning Time: 7.873 ms
Execution Time: 29.357 ms
(3 rows)
```

```
accountdb=# explain analyze select * from customer;
               QUERY PLAN
```

```
-----
Seq Scan on customer (cost=0.00..1.01 rows=1 width=40) (actual time=0.022..0.024 rows=1 loops=1)
Planning Time: 0.103 ms
Execution Time: 0.044 ms
(3 rows)
```

```
accountdb=# explain analyze select * from globalcountryinfo;
               QUERY PLAN
```

```
-----
Seq Scan on globalcountryinfo (cost=0.00..1.01 rows=1 width=34) (actual time=27.068..27.072 rows=1 loops=1)
Planning Time: 372.275 ms
Execution Time: 27.098 ms
(3 rows)
```

```
accountdb=#
```

Materialized View

(3 rows)

```
accountdb=# explain analyze select * from customer;
               QUERY PLAN
```

```
-----
Seq Scan on customer  (cost=0.00..1.01 rows=1 width=40) (actual time=0.022..0.024 rows=1 loops=1)
Planning Time: 0.103 ms
Execution Time: 0.044 ms
```

(3 rows)

```
accountdb=# explain analyze select * from globalcountryinfo;
               QUERY PLAN
```

```
-----
Seq Scan on globalcountryinfo  (cost=0.00..1.01 rows=1 width=34) (actual time=27.068..27.072 rows=1 loops=1)
Planning Time: 372.275 ms
Execution Time: 27.098 ms
```

(3 rows)

```
accountdb=# create Materialized View mv_globalcountryinfo as select * from globalcountryinfo;
SELECT 1
```

```
accountdb=# explain analyze select * from mv_globalcountryinfo;
               QUERY PLAN
```

```
-----
Seq Scan on mv_globalcountryinfo  (cost=0.00..23.10 rows=1310 width=34) (actual time=0.015..0.015 rows=1 loops=1)
Planning Time: 0.294 ms
Execution Time: 0.027 ms
```

(3 rows)

```
accountdb=#
```

Refreshing a MATERIALIZED VIEW

EDB-PSQL - 13

```
accountdb=# refresh materialized view mv_globalcountryinfo;  
REFRESH MATERIALIZED VIEW  
accountdb=#
```


Postgres Index

- An Index is the structure or object by which we can retrieve specific rows or data faster.
- Indexes can be created using one or multiple columns or by using the partial data depending on your query requirement conditions.
- Index will create a pointer to the actual rows in the specified table.

Postgres Index

Table

	Column A	Column B
RowId 1	Some value 1	Some another value 1
RowId 2	Some value 2	Some another value 2
RowId 3	Some value 3	Some another value 3
RowId 4	Some value 1	Some another value 4
RowId 5	Some value 5	Some another value 5
RowId 6	Some value 6	Some another value 6
RowId 7	Some value 1	Some another value 7
RowId 8	Some value 8	Some another value 8

Index

Some value 1	"RowId 1" "RowId 4" "RowId 7"
Some value 2	RowId 2
Some value 3	RowId 3
Some value 5	RowId 5
Some value 6	RowId 6
Some value 8	RowId 8

Index by column "Column A"

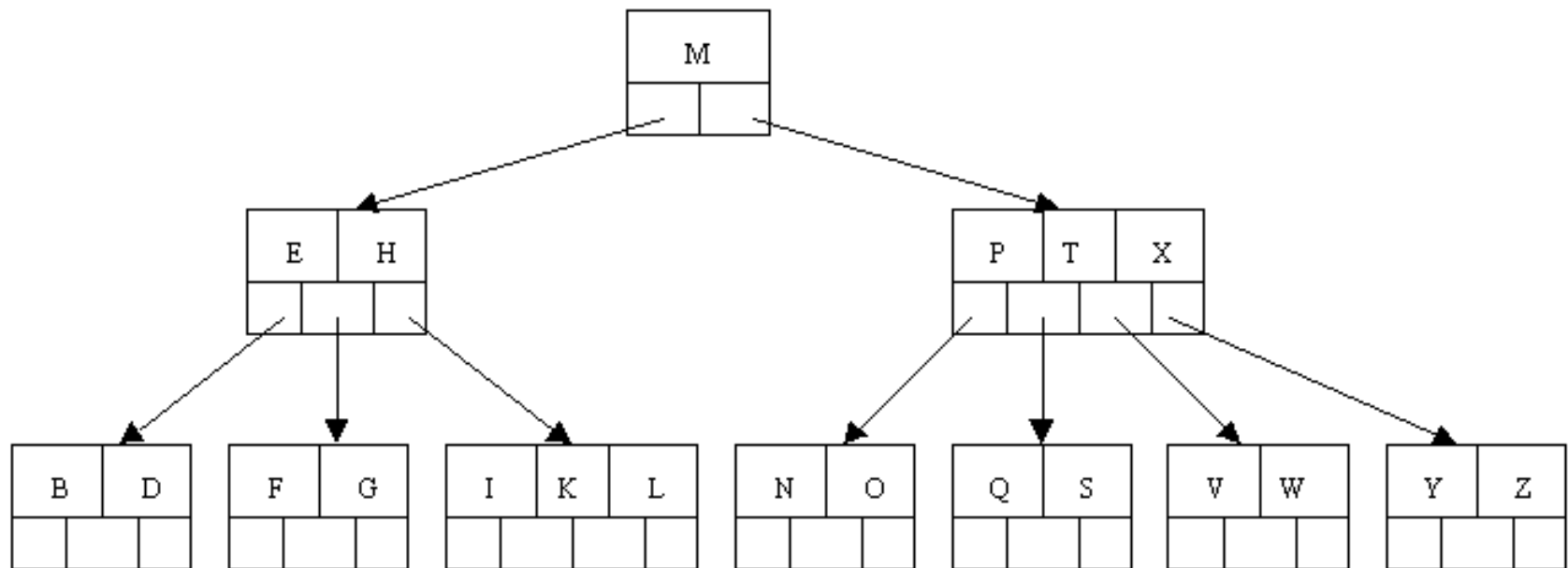
Postgres Index

- Types Of Indexes
 - PostgreSQL server provides following types of indexes, which each uses a different algorithm:
 - B-tree
 - Hash
 - GiST
 - SP-GiST
 - GIN
 - BRIN

B-tree index

- The most common and widely used index type is the B-tree index.
- This is the default index type for the `CREATE INDEX` command, unless you explicitly mention the type during index creation.
- If the indexed column is used to perform the comparison by using comparison operators such as `<`, `<=`, `=`, `>=`, and `>`, then the Postgres optimizer uses the index created by the B-tree option for the specified column.

B-tree index



B-tree index

- Advantages:
 - Retain sorting data
 - Support the search for the unary and binary predicates
 - Allow the entire sequence of data to estimate cardinality (number of entries) for the entire index (and therefore the table), range, and with arbitrary precision without scanning
- Disadvantages:
 - For their construction is require to perform a full sorting pairs (row, RowId) (slow operation)
 - Take up a lot of disk space. Index on unique “Integers” weights twice more as the column (because additionaly RowId need stored)
 - Recording unbalances tree constantly, and begins to store data sparsely, and the access time is increased by increasing the amount of disk information. What is why, B-Tree indexes require monitoring and periodic rebuilding

List indexes

- SELECT
- tablename,
- indexname,
- indexdef
- FROM
- pg_indexes
- WHERE
- schemaname = 'public'
- ORDER BY
- tablename,
- indexname;

List indexes

EDB-PSQL - 13

```
Output: transaction_id, transaction_date_time, amount, mode, status
Filter: (producttransaction.amount > 0)
Planning Time: 0.137 ms
Execution Time: 0.033 ms
(5 rows)
```

```
accountdb=# SELECT
accountdb=#      tablename,
accountdb=#      indexname,
accountdb=#      indexdef
accountdb=# FROM
accountdb=#      pg_indexes
accountdb=# WHERE
accountdb=#      schemaname = 'public'
accountdb=# ORDER BY
accountdb=#      tablename,
accountdb=#      indexname;
```

tablename	indexname	indexdef
customer	customer_pkey	CREATE UNIQUE INDEX customer_pkey ON public.customer USING btree (customer_id)
globalcountryinfo	globalcountryinfo_pkey	CREATE UNIQUE INDEX globalcountryinfo_pkey ON public.globalcountryinfo USING btree (countrycode)
producttransaction	amount_index	CREATE INDEX amount_index ON public.producttransaction USING btree (amount)
producttransaction	producttransaction_pkey	CREATE UNIQUE INDEX producttransaction_pkey ON public.producttransaction USING btree (transaction_id)
profile	profile_pkey	CREATE UNIQUE INDEX profile_pkey ON public.profile USING btree (profile_id)

(5 rows)

```
accountdb=#
```

Type here to search



06:27 31/03/2021 ENG

B-Tree Index

```
EDB-PSQL - 13
mode          | character varying(50) |          |          |
status        | boolean                |          |          |
Indexes:
  "producttransaction_pkey" PRIMARY KEY, btree (transaction_id)

accountdb=# create index amount_index on producttransaction(amount);
CREATE INDEX
accountdb=# explain (analyze,verbose) select * from producttransaction;
               QUERY PLAN
-----
Seq Scan on public.producttransaction  (cost=0.00..1.02 rows=2 width=143) (actual time=0.016..0.017 rows=2 loops=1)
  Output: transaction_id, transaction_date_time, amount, mode, status
Planning Time: 0.626 ms
Execution Time: 0.032 ms
(4 rows)

accountdb=# \d producttransaction
              Table "public.producttransaction"
  Column      |          Type          | Collation | Nullable |          Default
-----
transaction_id | bigint                 |           | not null | nextval('producttransaction_transaction_id_seq'::regclass)
transaction_date_time | timestamp with time zone |           |           |
amount         | bigint                 |           |           |
mode           | character varying(50)  |           |           |
status         | boolean                |           |           |
Indexes:
  "producttransaction_pkey" PRIMARY KEY, btree (transaction_id)
  "amount_index" btree (amount)

accountdb=#
```

B-Tree Index

```
index_demo=# explain (analyze,verbose) select * from public."Track" where "Name"='Princess of the Dawn'
              QUERY PLAN
-----
Bitmap Heap Scan on public."Track"  (cost=4.29..8.30 rows=1 width=71) (actual time=0.923..0.924 rows=1)
  Output: "TrackId", "Name", "AlbumId", "Bytes", "UnitPrice"
  Recheck Cond: (("Track"."Name")::text = 'Princess of the Dawn'::text)
  Heap Blocks: exact=1
-> Bitmap Index Scan on "Track_Name_idx"  (cost=0.00..4.29 rows=1 width=0) (actual time=0.578..0.579 rows=1)
    Index Cond: (("Track"."Name")::text = 'Princess of the Dawn'::text)

Planning time: 0.128 ms

Execution time: 0.953 ms

(8 rows)
```

B-Tree Index

EDB-PSQL - 13

```
accountdb=# refresh materialized view mv_globalcountryinfo;
REFRESH MATERIALIZED VIEW
accountdb=# explain (analyze,verbose) select * from globalcountryinfo;
               QUERY PLAN
```

```
-----
Seq Scan on public.globalcountryinfo  (cost=0.00..1.01 rows=1 width=34) (actual time=0.016..0.017 rows=1 loops=1)
   Output: countrycode, country_state
Planning Time: 0.074 ms
Execution Time: 0.037 ms
(4 rows)
```

```
accountdb=# explain (analyze,verbose) select * from mv_globalcountryinfo;
               QUERY PLAN
```

```
-----
Seq Scan on public.mv_globalcountryinfo  (cost=0.00..23.10 rows=1310 width=34) (actual time=0.017..0.017 rows=1 loops=1)
   Output: countrycode, country_state
Planning Time: 0.412 ms
Execution Time: 0.030 ms
(4 rows)
```

```
accountdb=# explain (analyze,verbose) select * from producttransaction;
               QUERY PLAN
```

```
-----
Seq Scan on public.producttransaction  (cost=0.00..1.02 rows=2 width=143) (actual time=0.019..0.021 rows=2 loops=1)
   Output: transaction_id, transaction_date_time, amount, mode, status
Planning Time: 0.079 ms
Execution Time: 0.033 ms
(4 rows)
```

```
accountdb=#
```

B-Tree Index

Here is a query which can use the created B-tree index and then again try to select the data with the same **WHERE** clause:

```
index_demo_=# create index "Track_Name_idx" on public."Track" ("Name");

CREATE INDEX

index_demo_=#

index_demo_=# explain (analyze,verbose) select * from public."Track" where "Name"='Princess of the Dawn';

                                QUERY PLAN

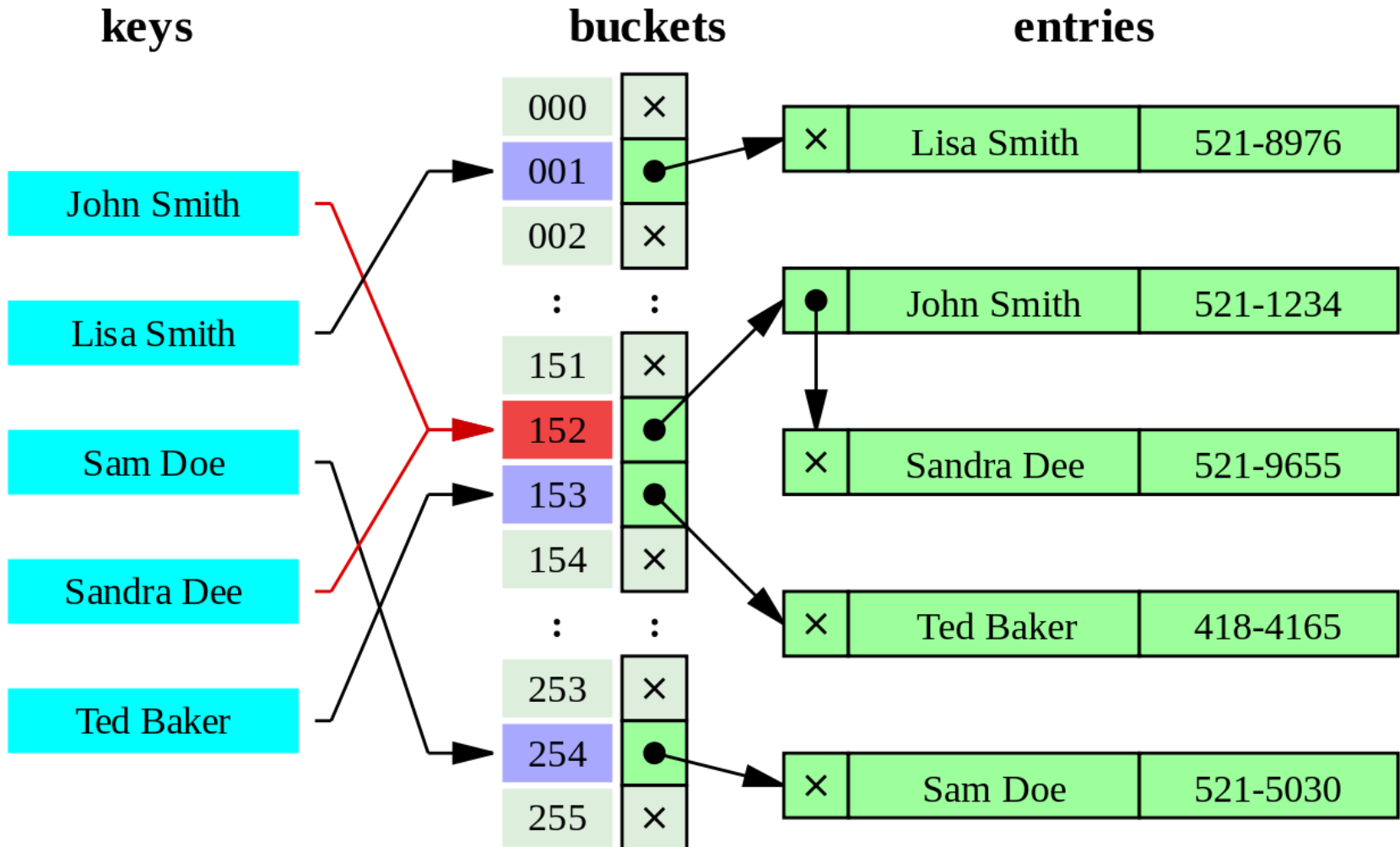
-----
Index Scan using "Track_Name_idx" on public."Track"  (cost=0.28..8.30 rows=1 width=71) (actual time=0.051..0.051 rows=1)
  Output: "TrackId", "Name", "AlbumId", "MediaTypeId", "GenreId", "Composer", "Milliseconds", "Bytes"
  Index Cond: (("Track"."Name")::text = 'Princess of the Dawn'::text)

Planning time: 0.151 ms

Execution time: 0.571 ms

(5 rows)
```

Hash index



Hash index

- Advantages:
 - Very fast search $O(1)$
 - Stability - the index does not need to be rebuild
- Disadvantages:
- Hash is very sensitive to collisions. In the case of “bad” data distribution, most of the entries will be concentrated in a few bouquets, and in fact the search will occur through collision resolution.

Hash index

- The Hash index can be used only if the equality condition = is being used in the query.

```
index_demo_=# create index "Track_Name_idx" on public."Track" using HASH ("Name");
```

CREATE INDEX

```
index_demo_=# \d+ public."Track"
```

Table "public.Track"

Column	Type	Collation	Nullable	Default	Storage	Stats target	Description
TrackId	integer		not null		plain		
Name	character varying(200)		not null		extended		
AlbumId	integer				plain		
MediaTypeId	integer		not null		plain		
GenreId	integer				plain		
Composer	character varying(220)				extended		
Milliseconds	integer		not null		plain		
Bytes	integer				plain		
UnitPrice	numeric(10,2)		not null		main		

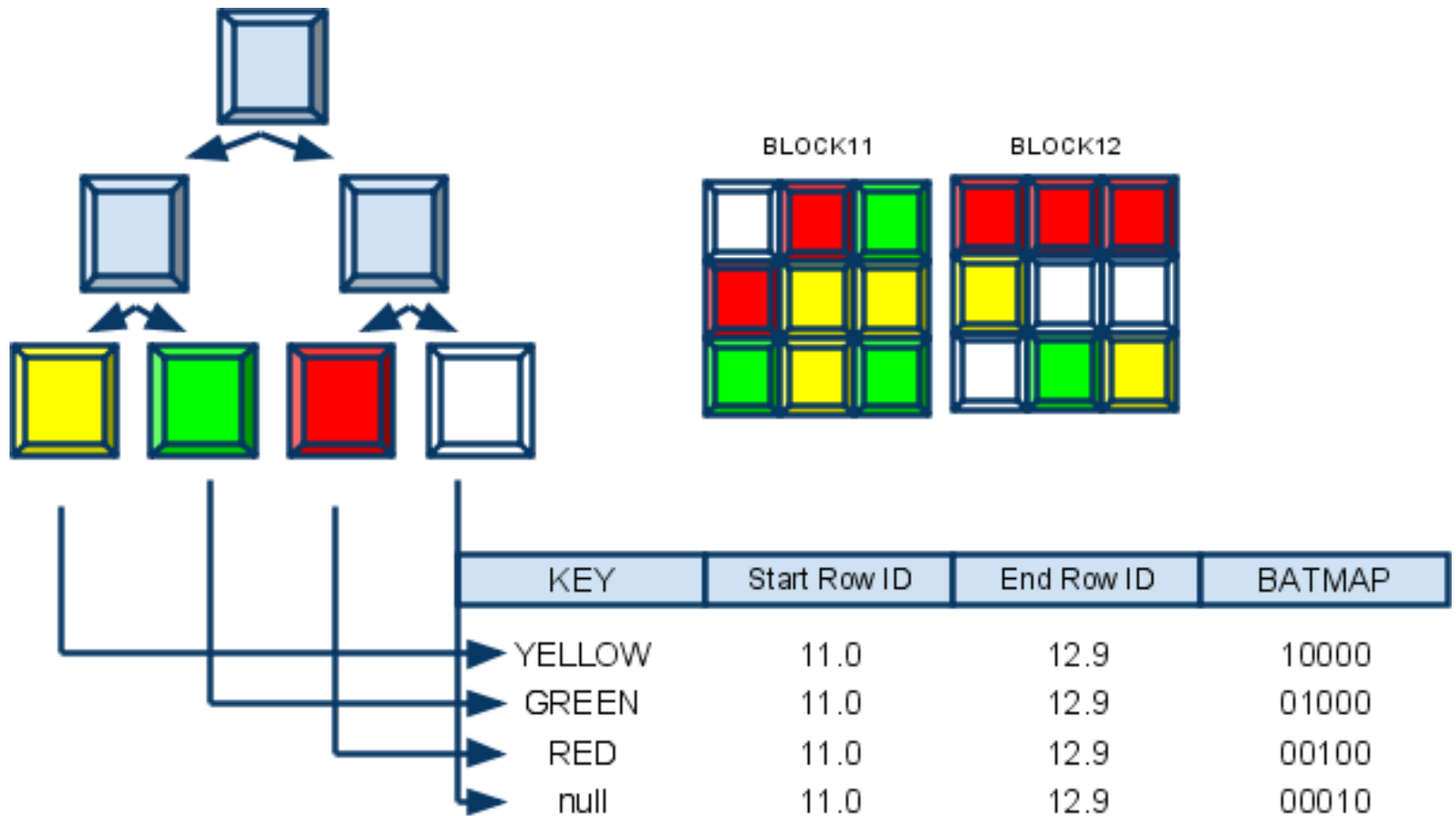
Indexes:

```
"PK_Track" PRIMARY KEY, btree ("TrackId")
"Track_Name_idx" hash ("Name")
```

Bitmap index

- Bitmap index create a separate bitmap (a sequence of 0 and 1) for each possible value of the column, where each bit corresponds to a string with an indexed value. Bitmap indexes are optimal for data where bit unique values (example, gender field).

Bitmap index



Bitmap index

- Advantages:
 - Compact representation (small amount of disk space)
 - Fast reading and searching for the predicate “is”
 - Effective algorithms for packing masks (even more compact representation, than indexed data)
- Disadvantages:
 - You can not change the method of encoding values in the process of updating the data. From this it follows that if the distribution data has changed, it is required the index to be completely rebuild

GiST index (Generalized Search Tree index)

- GiST stands for Generalized Search Tree Index.
- The main point of GiST is to be able to index queries that simply are not indexable using B-tree.
- So GiST is useful when you have queries that are not btree-indexable.
- The column can be of tsvector or tsquery type.
- There is a well-known, and pretty fast, geographic extension to PostgreSQL — PostGIS, which uses GiST for its purposes. GiST indexes are what is used if you want to speed up full text searches.

GiST index (Generalized Search Tree index)

Create a table with some geographical data points such as circle or point and index it using GiST:

```
index_demo_=# create table geo_point(circle_details circle);
```

```
CREATE TABLE
```

```
index_demo_=# create index ON geo_point using gist(circle_details);
```

```
CREATE INDEX
```

```
index_demo_=# \d+ geo_point
```

```
Table "public.geo_point"
```

Column	Type	Collation	Nullable	Default	Storage	Stats target	Description
--------	------	-----------	----------	---------	---------	--------------	-------------

-----+-----+-----+-----+-----+-----+-----+-----							
---	--	--	--	--	--	--	--

circle_details	circle				plain		
----------------	--------	--	--	--	-------	--	--

Indexes:

```
"geo_point_circle_details_idx" gist (circle_details)
```

```
index_demo_=#
```

GiST index (Generalized Search Tree index)

Create a table with some geographical data points such as circle or point and index it using GiST:

```
index_demo_=# create table geo_point(circle_details circle);
```

```
CREATE TABLE
```

```
index_demo_=# create index ON geo_point using gist(circle_details);
```

```
CREATE INDEX
```

```
index_demo_=# \d+ geo_point
```

```
Table "public.geo_point"
```

Column	Type	Collation	Nullable	Default	Storage	Stats target	Description
--------	------	-----------	----------	---------	---------	--------------	-------------

-----+-----+-----+-----+-----+-----+-----+-----							
---	--	--	--	--	--	--	--

circle_details	circle				plain		
----------------	--------	--	--	--	-------	--	--

Indexes:

```
"geo_point_circle_details_idx" gist (circle_details)
```

```
index_demo_=#
```

SP-GiST (Space-Partitioned GiST)

- SP-GiST stands for “space-partitioned GiST.”
- This index type supports non-balanced data structures such as quadrees, k-d trees, and radix trees.

GIN (Generalized Inverted Index)

- GIN stands for “Generalized Inverted Index.” This are the “Inverted Indexes” that get used for multiple component values such as arrays.
- The standard distribution of PostgreSQL includes a GIN operator class for arrays, which supports indexed queries using these operators: <@, @>, =, and &&.

BRIN (Block Range index)

- BRIN stands for “Block Range index.” This featured index type was introduced with Postgres v 9.5.
- “Because a BRIN index is very small, scanning the index adds little overhead compared to a sequential scan, but may avoid scanning large parts of the table that are known not to contain matching tuples.”
- “The size of the block range is determined at index creation time by the `pages_per_range` storage parameter.
- The number of index entries will be equal to the size of the relation in pages divided by the selected value for `pages_per_range`.
- Therefore, the smaller the number, the larger the index becomes (because of the need to store more index entries), but at the same time the summary data stored can be more precise and more data blocks can be skipped during an index scan.”

BRIN (Block Range index)

- In a BRIN index, PostgreSQL reads your selected column's maximum and minimum values for each 8k-size page of stored data, and then stores that information (page number and minimum and maximum values of column) into the BRIN index.

BRIN (Block Range index)

- B-tree indexes have entries for each row in your table, duplicating the data in the indexed columns.
- This permits super-fast index-only scans, but can be wasteful of disk space.
- A B-tree index also stores the data in sorted order, which permits very fast lookups of single rows.
- It also stores the full location of each row in the table, which can require a lot of work when running queries that return many rows.
- However, a BRIN index contains only the minimum and maximum values contained in a group of database pages.
- Therefore, the reason for using the BRIN index is that this is a very straightforward way to optimize for speed, and the BRIN index only takes up very little space compared to a B-tree index.
- BRIN is designed for handling very large tables in which certain columns have some natural correlation with their physical location within the table.

BRIN (Block Range index)

- A good example of where you could use a BRIN index is for data you would consider immutable.
- For the example that follows, the BRIN index is 72kb in size, while the B-tree index is 676mb in size.
- Yet the performance is pretty much identical for selects.

-

BRIN (Block Range index)

```
index_demo_=# create index "Track_TrackId_idx" on public."Track" using brin("TrackId");
```

```
CREATE INDEX
```

```
index_demo_=# \d+ public."Track"
```

Table "public.Track"

Column	Type	Collation	Nullable	Default	Storage	Stats target	Description
TrackId	integer		not null		plain		
Name	character varying(200)		not null		extended		
AlbumId	integer				plain		
MediaTypeId	integer		not null		plain		
GenreId	integer				plain		
Composer	character varying(220)				extended		
Milliseconds	integer		not null		plain		
Bytes	integer				plain		
UnitPrice	numeric(10,2)		not null		main		

```
Indexes:
```

```
"PK_Track" PRIMARY KEY, btree ("TrackId")
```

```
"Track_TrackId_idx" brin ("TrackId")
```

```
index_demo_=# \di+
```

Partial Index

- The partial index is useful in case you have commonly used WHERE conditions which use constant values as follows:

```
SELECT *  
FROM table_name  
WHERE column_name = constant_value;
```

Partial Index

- You can optimize this query by creating an index for the active column as follows:
- `CREATE INDEX idx_customer_active`
- `ON customer(active);`
- This index fulfills its purpose, however, it includes many rows that are never searched, namely all the active customers.
- To define an index that includes only inactive customers, you use the following statement:
- **`CREATE INDEX idx_customer_inactive`**
- **`ON customer(active)`**
- **`WHERE active = 0;`**

Built-In Functions- String

Function	Description	Example
LENGTH(string)	Returns the number of characters	LENGTH('hello') → 5
LOWER(string)	Converts to lowercase	LOWER('HELLO') → 'hello'
UPPER(string)	Converts to uppercase	UPPER('hello') → 'HELLO'
INITCAP(string)	Capitalizes first letter of each word	INITCAP('hello world') → 'Hello World'
TRIM([LEADING TRAILING BOTH] 'x' FROM str)		
BTRIM(str, chars)	Trims specified characters from both ends	BTRIM('xyHelloxy', 'xy') → 'Hello'
CONCAT(str1, str2, ...)	Concatenates strings	CONCAT('Hello', ' ', 'World') → 'Hello World'
`		`

Built-In Functions- String

Function	Description	Example
SUBSTRING(str FROM x FOR y)	Extracts substring	SUBSTRING('abcdef' FROM 2 FOR 3) → 'bcd'
LEFT(str, n)	Leftmost n characters	LEFT('abcdef', 3) → 'abc'
RIGHT(str, n)	Rightmost n characters	RIGHT('abcdef', 3) → 'def'
REPLACE(str, from, to)	Replace substring	REPLACE('hello', 'l', 'x') → 'hexxo'
POSITION(substr IN str)	Position of substring	POSITION('e' IN 'hello') → 2
STRPOS(str, substr)	Alias for POSITION	STRPOS('hello', 'e') → 2
OVERLAY(str PLACING str2 FROM pos FOR len)	Replaces part of string	OVERLAY('abcdef' PLACING 'xyz' FROM 2 FOR 3) → 'axyzef'

Built-In Functions- String

Function	Description	Example
REVERSE(str)	Reverses a string	REVERSE('abc') → 'cba'
LPAD(str, len, padstr)	Pads left side	LPAD('123', 5, '0') → '00123'
RPAD(str, len, padstr)	Pads right side	RPAD('123', 5, '0') → '12300'
REGEXP_REPLACE(str, pattern, replacement)	Regex-based replacement	REGEXP_REPLACE('abc123', '[0-9]', '', 'g') → 'abc'
REGEXP_MATCHES(str, pattern)	Returns regex match array	REGEXP_MATCHES('abc123', '[a-z]+') → {abc}

```
SELECT LENGTH('PostgreSQL'), LOWER('HELLO'), SUBSTRING('abcdef'
FROM 2 FOR 3), REPLACE('banana', 'a', 'x'), LPAD('42', 5, '0');
```

Built in Date Functions

Function	Description	Example
CURRENT_DATE	Returns today's date	SELECT CURRENT_DATE;
CURRENT_TIME	Returns current time	SELECT CURRENT_TIME;
CURRENT_TIMESTAMP or NOW()	Returns current date and time	SELECT NOW();
AGE(timestamp)	Interval between now and given timestamp	AGE('2000-01-01')
AGE(timestamp1, timestamp2)	Interval between two timestamps	AGE('2025-01-01', '2000-01-01')
DATE_PART(field, source)	Extract part of a date (year, month, etc.)	DATE_PART('year', NOW())
EXTRACT(field FROM source)	Same as DATE_PART	EXTRACT(DAY FROM NOW())
DATE_TRUNC(field, source)	Truncates date/time to a unit	DATE_TRUNC('month', NOW()) → 2025-07-01 00:00:00
TO_CHAR(timestamp, format)	Format a timestamp	TO_CHAR(NOW(), 'YYYY-MM-DD')
TO_DATE(text, format)	Convert string to date	TO_DATE('2025-07-20', 'YYYY-MM-DD')

Built in Date Functions

Function	Description	Example
TO_TIMESTAMP(text, format)	Convert string to timestamp	TO_TIMESTAMP('2025-07-20 15:00', 'YYYY-MM-DD HH24:MI')
CLOCK_TIMESTAMP()	Returns actual current time (not cached)	SELECT CLOCK_TIMESTAMP();
STATEMENT_TIMESTAMP()	Timestamp when current statement started	SELECT STATEMENT_TIMESTAMP();
TRANSACTION_TIMESTAMP()	Timestamp when transaction started	SELECT TRANSACTION_TIMESTAMP();
JUSTIFY_DAYS(interval)	Converts excess days into months	JUSTIFY_DAYS(INTERVAL '90 days')
JUSTIFY_INTERVAL(interval)	Normalizes months and days	JUSTIFY_INTERVAL(INTERVAL '1 year 25 months')

```
-- Add 7 days to today
SELECT CURRENT_DATE + INTERVAL '7 days';

-- Subtract 1 month
SELECT CURRENT_DATE - INTERVAL '1 month';
```

Date Functions

- SELECT
- EXTRACT(YEAR FROM CURRENT_DATE) AS year,
- EXTRACT(MONTH FROM CURRENT_DATE) AS month,
- EXTRACT(DAY FROM CURRENT_DATE) AS day;

Date Functions

- `SELECT '2025-01-01'::date < '2025-12-31'::date;`
- `-- returns true`
- `SELECT TO_CHAR(NOW(), 'YYYY-MM-DD
HH24:MI:SS');`
- `-- Output: '2025-07-20 18:34:12'`

Built-in Functions - JSON/JSONB Functions in PostgreSQL

Feature	JSON	JSONB
Storage	As text	Binary format
Speed	Slower to query	Faster and indexed
Ordering	Preserved	Not preserved
Duplicate Keys	Allowed	Deduplicated (last wins)

Built-in Functions - JSON/JSONB Functions in PostgreSQL

```
CREATE TABLE employees (  
  id SERIAL PRIMARY KEY,  
  name TEXT,  
  data JSONB -- store nested info  
);
```

```
INSERT INTO employees (name, data) VALUES  
(  
  'Alice', '{"age": 30, "skills": ["SQL", "Python"]}'  
,  
  'Bob', '{"age": 25, "skills": ["Java", "React"]}'  
);
```

Built-in Functions - JSON/JSONB Functions in PostgreSQL

Operator / Function	Description	Example
->	Get JSON object field by key (returns JSON)	data -> 'age'
->>	Get JSON object field as text	data ->> 'age'
#>	Get nested JSON object	data #> '{skills,0}'
#>>	Get nested JSON as text	data #>> '{skills,0}'
@>	Does JSONB contain the specified key/value?	data @> '{"age": 30}'
<@	Is JSONB contained in another?	'{ "age": 30 }' <@ data
?	Does key exist?	data ? 'age'
`?`	`	Any of the keys exist?
?&	All keys exist?	data ?& array['age', 'skills']
jsonb_array_elements()	Expands JSON array to rows	see below
jsonb_each_text()	Extracts key-value pairs as text	see below

Built-in Functions - JSON/JSONB Functions in PostgreSQL

```
SELECT name, (data ->> 'age')::int AS age FROM employees;
```

```
SELECT * FROM employees WHERE data @> '{"age": 30}';
```

```
SELECT name, jsonb_array_elements_text(data->'skills') AS skill  
FROM employees;
```

```
SELECT name, key, value  
FROM employees, jsonb_each_text(data);
```

```
CREATE INDEX idx_employee_data ON employees USING GIN  
(data);
```

Moving Data the Copy command

- import and export data using CSV files in PostgreSQL
 - 1. Creating a .csv file
 - 2. Creating the table structure
 - 3. Importing from a psql prompt
 - 4. Importing from a shell prompt
 - 5. Exporting data to a .csv file

Moving Data the Copy command

- Creating a .csv file
- The first step is to create a .csv file. You can use any existing file, or you can use the data below that comprises a basic .csv file:
- Capital_City,State,Abbreviation,Zip_Code
- Montgomery,Alabama,AL,36104
- Juneau,Alaska,AK,99801
- Phoenix,Arizona,AZ,85001
- Little Rock,Arkansas,AR,72201

Moving Data the Copy command

- CREATE TABLE usa
- (Capital_city varchar,
- State varchar,
- Abbreviation varchar(2),
- zip_code numeric(5)
-);

Importing from a psql prompt

- `\COPY <table name> FROM 'location + file_name' DELIMITER ',' CSV HEADER;`
- To understand the execution steps, let's look at each item in this command:
- `\COPY`: This is the command to copy the record to / from the .csv file.
- `<table name>`: Provides the table name where you want to import the data.
- `FROM`: Specifies that we are going to import from a file (we will also be using `TO` in order to export it to a file at a later stage).

Importing from a psql prompt

- 'location + file_name': An absolute path to the file (make sure you have read access to the file).
- DELIMITER ',': Specifies the delimiter, which in our case is a comma: ','.
- CSV Specifies the file type from which we are going to import.
- HEADER Signifies that we have a header row in our .csv file and while importing we should ignore the first row (similarly, while exporting we can use this to specify whether we want to include or exclude the header file).

Importing from a psql prompt

EDB-PSQL - 13

```
accountdb=# \copy usa from 'd:/basic.csv' delimiter ',' csv header;  
COPY 4  
accountdb=#
```

Exporting data to a .csv file

EDB-PSQL - 13

```
accountdb=# \copy usa from 'd:/basic.csv' delimiter ',' csv header;  
COPY 4  
accountdb=# \COPY usa TO 'd:/output.csv' DELIMITER ',' CSV HEADER;  
COPY 4  
accountdb=#
```


Understanding And Reading the PostgreSQL System Catalog (Meta Data)

- The PostgreSQL System Catalog is a schema with tables and views that contain metadata about all the other objects inside the database and more.
- With it, we can discover when various operations happen, how tables or indexes are accessed, and even whether or not the database system is reading information from memory or needing to fetch data from disk.

Understanding And Reading the PostgreSQL System Catalog (Meta Data)

- PostgreSQL stores the metadata information about the database and cluster in the schema 'pg_catalog'.
- This information is partially used by PostgreSQL itself to keep track of things itself, but it also is presented so external people / processes can understand the inside of the databases too.

Understanding And Reading the PostgreSQL System Catalog (Meta Data)

- Database Information
- General database info is stored in pg_database and statistics are stored in pg_stat_database.
- `SELECT oid,* FROM pg_database WHERE datname = 'accountdb';`
- `SELECT * FROM pg_stat_database WHERE datname = 'accountdb';`

Understanding And Reading the PostgreSQL System Catalog (Meta Data)

- Database Activity and Locks
- There are two views that show current user activity, `pg_stat_activity` and `pg_locks`.
- When queried, these show information about current connections to the databases, and what kind of locks they have on what relations.
- `SELECT * FROM pg_stat_activity;`
- `SELECT * FROM pg_locks;`

Understanding And Reading the PostgreSQL System Catalog (Meta Data)

- `SELECT * FROM pg_stat_user_tables WHERE schemaname = 'public' AND relname = 'history';`

Understanding And Reading the PostgreSQL System Catalog (Meta Data)

- Table Vacuum Activity
- Table maintenance is done through either VACUUM or AUTOVACUUM, and statistics are gathered through ANALYZE or AUTOANALYZE.
- The next four columns contain the dates for when each of these operations were last run: 'last_vacuum', 'last_autovacuum', 'last_analyze', 'last_autoanalyze'.
- We also have four more convenient columns that simply count how many times the previous actions occur.
- Using these, we can see which tables get the most activity: 'vacuum_count', 'autovacuum_count', 'analyze_count', and 'autoanalyze_count'.

Understanding And Reading the PostgreSQL System Catalog (Meta Data)

- `SELECT * FROM pg_statio_user_tables WHERE schemaname = 'public' AND relname = history;`
- Index Meta data
- `SELECT * FROM pg_stat_user_indexes WHERE indexrelname = 'history_pkey';`

How to use table partitioning to scale PostgreSQL

- 1. Benefits of partitioning
- 2. When to use partitioning
- 3. How to use partitioning
 - a. List partition
 - b. Range partition
 - c. Hash partition
 - d. Multilevel partition
- 4. Limitations

Partitioning

- With huge data being stored in databases, performance and scaling are two main factors that are affected.
- As table size increases with data load, more data scanning, swapping pages to memory, and other table operation costs also increase.
- Partitioning may be a good solution, as It can help divide a large table into smaller tables and thus reduce table scans and memory swap problems, which ultimately increases performance.
- Partitioning helps to scale PostgreSQL by splitting large logical tables into smaller physical tables that can be stored on different storage media based on uses.
- Users can take better advantage of scaling by using declarative partitioning along with foreign tables using `postgres_fdw`.

Benefits of Partitioning

- PostgreSQL declarative partitioning is highly flexible and provides good control to users.
- Users can create any level of partitioning based on need and can modify, use constraints, triggers, and indexes on each partition separately as well as on all partitions together.
- Query performance can be increased significantly compared to selecting from a single large table.
- Partition-wise-join and partition-wise-aggregate features increase complex query computation performance as well.
- Bulk loads and data deletion can be much faster, as based on user requirements these operations can be performed on individual partitions.
- Each partition can contain data based on its frequency of use and so can be stored on media that may be cheaper or slower for low-use data.

When to use partitioning

- Most benefits of partitioning can be enjoyed when a single table is not able to provide them.
- So we can say that if a lot of data is going to be written on a single table at some point, users need partitioning.
- Apart from data, there may be other factors users should consider, like update frequency of the data, use of data over a time period, how small a range data can be divided, etc.
- With good planning and taking all factors into consideration, table partitioning can give a great performance boost and scale your PostgreSQL to larger datasets.

How to use partitioning

- As of PostgreSQL12 release List, Range, Hash and combinations of these partition methods at different levels are supported.

LIST PARTITION

- A list partition is created with predefined values to hold in a partitioned table.
- A default partition (optional) holds all those values that are not part of any specified partition.
- `CREATE TABLE customers (id INTEGER, status TEXT, arr NUMERIC) PARTITION BY LIST(status);`
- `CREATE TABLE cust_active PARTITION OF customers FOR VALUES IN ('ACTIVE');`
- `CREATE TABLE cust_archived PARTITION OF customers FOR VALUES IN ('EXPIRED');`
- `CREATE TABLE cust_others PARTITION OF customers DEFAULT;`

RANGE PARTITION

- A range partition is created to hold values within a range provided on the partition key.
- Both minimum and maximum values of the range need to be specified, where minimum value is inclusive and maximum value is exclusive.
- `CREATE TABLE customers (id INTEGER, status TEXT, arr NUMERIC) PARTITION BY RANGE(arr);`
- `CREATE TABLE cust_arr_small PARTITION OF customers FOR VALUES FROM (MINVALUE) TO (25);`

RANGE PARTITION

- CREATE TABLE cust_arr_medium PARTITION OF customers FOR VALUES FROM (25) TO (75);
- CREATE TABLE cust_arr_large PARTITION OF customers FOR VALUES FROM (75) TO (MAXVALUE);

HASH PARTITION

- A hash partition is created by using modulus and remainder for each partition, where rows are inserted by generating a hash value using these modulus and remainders.
- `CREATE TABLE customers (id INTEGER, status TEXT, arr NUMERIC) PARTITION BY HASH(id);`
- `CREATE TABLE cust_part1 PARTITION OF customers FOR VALUES WITH (modulus 3, remainder 0);`

HASH PARTITION

- CREATE TABLE cust_part2 PARTITION OF customers FOR VALUES WITH (modulus 3, remainder 1);
- CREATE TABLE cust_part3 PARTITION OF customers FOR VALUES WITH (modulus 3, remainder 2);

MULTILEVEL PARTITION

- PostgreSQL multilevel partitions can be created up to N levels.
- Partition methods LIST-LIST, LIST-RANGE, LIST-HASH, RANGE-RANGE, RANGE-LIST, RANGE-HASH, HASH-HASH, HASH-LIST, and HASH-RANGE can be created in PostgreSQL declarative partitioning.

MULTILEVEL PARTITION

- `CREATE TABLE customers (id INTEGER, status TEXT, arr NUMERIC) PARTITION BY LIST(status);`
- `CREATE TABLE cust_active PARTITION OF customers FOR VALUES IN ('ACTIVE','RECURRING','REACTIVATED') PARTITION BY RANGE(arr);`
- `CREATE TABLE cust_arr_small PARTITION OF cust_active FOR VALUES FROM (MINVALUE) TO (101) PARTITION BY HASH(id);`

MULTILEVEL PARTITION

- CREATE TABLE cust_arr_small PARTITION OF cust_active FOR VALUES FROM (MINVALUE) TO (101) PARTITION BY HASH(id);
- CREATE TABLE cust_part11 PARTITION OF cust_arr_small FOR VALUES WITH (modulus 2, remainder 0);
- CREATE TABLE cust_part12 PARTITION OF cust_arr_small FOR VALUES WITH (modulus 2, remainder 1);
- CREATE TABLE cust_other PARTITION OF customers DEFAULT PARTITION BY RANGE(arr);

MULTILEVEL PARTITION

- CREATE TABLE cust_arr_large PARTITION OF cust_other FOR VALUES FROM (101) TO (MAXVALUE) PARTITION BY HASH(id);
- CREATE TABLE cust_part21 PARTITION OF cust_arr_large FOR VALUES WITH (modulus 2, remainder 0);
- postgres=# CREATE TABLE cust_part22 PARTITION OF cust_arr_large FOR VALUES WITH (modulus 2, remainder 1);

Limitations

- Partitioning was introduced in PostgreSQL 10 and continues to be improved and made more stable. Still, there are certain limitations that users may need to consider:
 - 1. Unique constraints on partitioned tables must include all the partition key columns. One work-around is to create unique constraints on each partition instead of a partitioned table.
 - 2. Partition does not support BEFORE ROW triggers on partitioned tables. If necessary, they must be defined on individual partitions, not the partitioned table.
 - 3. Range partition does not allow NULL values.
 - 4. PostgreSQL does not create a system-defined subpartition when not given it explicitly, so if a subpartition is present at least one partition should be present to hold values.
 - 5. In the case of HASH-LIST, HASH-RANGE, and HASH-HASH composite partitions, users need to make sure all partitions are present at the subpartition level as HASH can direct values at any partition based on hash value.

Postgres vs Oracle

	PostgreSQL Community	Oracle Standard Edition	Oracle Enterprise Edition
Single Database Edition Costs	\$0	\$17,500	\$47,500
Clustering / Replication	\$0	Not available	\$23,000
Advanced Security	\$0	Not available	\$15,000
Software Update License & Support (annual)	\$0	Not available	\$18,810
Total Cost	\$0	\$17,500	\$104,310

Postgres vs Oracle

	PostgreSQL	Oracle
Scalability	PostgreSQL offers free scalability, and can scale up to millions of transactions per seconds .	Oracle Enterprise is recommended for high workloads which are highly scalable, but costly.
Updates	Since the last few years, new major PostgreSQL versions are released every year and minor versions with bug fixes are released every 3 months. One of the best things about Postgres is that the PostgreSQL Global Development Group announces the major and minor release dates in advance for the convenience of users and prospects.	New Oracle versions are generally available every 2-4 years.

Postgres vs Oracle

Security	PostgreSQL offers strong security capabilities through different authentication options (Host, LDAP, PAM and certificates authentication) and role-based access control (user-level, table-level and row-level). Data encryption can be achieved with advanced security plugins like pgcrypto which are available for free.	Oracle offers advanced security packages, but as a commercial database, they are available as an expensive add-on.
Replication	PostgreSQL supports native streaming replication and logical replication.	Oracle supported master-slave and master-master replication via Oracle Streams and Oracle multi-master in the older versions of Oracle Enterprise Edition, which has now been replaced by Oracle GoldenGate (a separately licensed application) for all types of data replications.

Postgres vs Oracle

Partitioning	PostgreSQL supports declarative partitioning .	Oracle supports general horizontal partitioning supported by all RDBMS.
Cloud Deployments	Can be deployed on any cloud provider, with a variety of PostgreSQL hosting solutions available.	Can only be deployed on Oracle Cloud and other popular cloud providers, but users must Bring Your Own License (BYOL) or use on-demand licensing .

Postgres vs Oracle

	PostgreSQL	Oracle
Compatibility	PostgreSQL PL/pgsql is compatible with other relational databases like Oracle which makes it relatively easy to move to PostgreSQL.	Oracle infrastructure does not offer strong compatibility with open source RDBMS.
Extensions	PostgreSQL offers many free open-source extensions, including: PostGIS - support for geospatial objects store and query. CitusDB - distributes data and queries horizontally across nodes. pg_repack - reorganizes tables online to reclaim storage.	Oracle offers commercial add-ons that are available for an additional license fee .

Postgres vs Oracle

Extensions	PostgreSQL offers many free open-source extensions, including: PostGIS – support for geospatial objects store and query. CitusDB – distributes data and queries horizontally across nodes. pg_repack – reorganizes tables online to reclaim storage.	Oracle offers commercial add-ons that are available for an additional license fee .
Tuning	PostgreSQL offers more light-weight tuning capabilities, like their Query Optimizer, and DBaaS platforms like ScaleGrid offer advanced slow query analysis.	Oracle requires significantly more effort to install and configure due to the hundreds of tuning variables and complex system requirements. Most of the tuning capabilities provided from Automatic Workload Repository (AWR) and database advisers comes bundled with Oracle Enterprise Manager Database/Grid control that requires the Enterprise Edition.

Postgres vs Oracle

Supported Operating Systems	• AIX • HP-UX • Linux • OS X • Solaris • Windows • z/OS	• FreeBSD • HP-UX • Linux • NetBSD • OpenBSD • OS X • SolarisUnix • Windows
Supported Languages	• C • C# • C++ • Clojure • Cobol • Delphi • Eiffel • Erlang • Fortran • Groovy • Haskell • Java • JavaScript	• .Net • C • C++ • Delphi • Java • JavaScript (Node.js)

Backup and Restore

- PostgreSQL provides the `pg_dump` utility to help you back up databases.
- It generates a database file with SQL commands in a format that can be easily restored in the future.
- To back up, a PostgreSQL database, start by logging into your database server, then switch to the Postgres user account, and run `pg_dump` as follows (replace `tecmintdb` with the name of the database you want to backup).
- By default, the output format is a plain-text SQL script file.

Backup and Restore

- Understand the Basic Syntax of `pg_dump`
- PostgreSQL comes with built-in utilities called `pg_dump` for easily creating and restoring backups.
- The basic syntax of `pg_dump` command is shown below:
- `pg_dump [OPTION]... [DBNAME]`
- A brief explanation of each option is shown below:
 - `-d, --dbname=DATABASENAME` : Used to specify the database that you want to back up.
 - `-h, --host=HOSTNAME` : Used to specify the hostname of your database server.
 - `-U, --username=USERNAME` : Used to specify the PostgreSQL username.
 - `-w, --no-password` : Used to ignore the password prompt.
 - `-p, --port=PORT` : Used to specify the PostgreSQL server port number.
 - `-W, --password` : Used to force password prompt.
 - `--role=ROLENAME` : SET ROLE before the dump.

Backup and Restore

- Backup
 - `pg_dump -d testdb -f testdb_backup.sql`
- Restore
 - `psql -d testdb -f testdb_backup.sql`
- If you want to stop the database restore process in case an error occurs, run the following command:
 - `psql -d testdb --set ON_ERROR_STOP=on -f testdb_backup.sql`

Backup and Restore

Administrator: Command Prompt

```
D:\PostgreSQL\13\bin>pg_dump.exe -U admin -d accountdb -f D:\account_dump.sql
Password:
```

```
D:\PostgreSQL\13\bin>_
```

```
D:\Program Files\edb\as13\bin>psql.exe -U admin -d accountdb -f D:\account_dump.sql
```

```
Password for user admin:
```

```
SET
SET
SET
SET
SET
set_config
-----
```

```
(1 row)
```

```
SET
SET
SET
SET
SET
SET
CREATE TABLE
```

Admin Queries

```
SELECT pg_size_pretty(pg_total_relation_size('table_name')) AS total_size;
```

× script.sql



```
select pg_size_pretty(pg_total_relation_size  
( 'sensors.observations' )) as total_size;
```

Query Results

total_size

2352 kB

Database Schema

Admin Queries

```
SELECT indexname, indexdef  
FROM pg_indexes  
WHERE tablename = 'sensors.observations';
```

Admin Queries - Size of Each Index on a Table

```
SELECT
    i.relname AS index_name,
    pg_size_pretty(pg_relation_size(i.oid)) AS index_size,
    pg_relation_size(i.oid) AS size_bytes
FROM
    pg_class t
JOIN
    pg_index ix ON t.oid = ix.indrelid
JOIN
    pg_class i ON i.oid = ix.indexrelid
WHERE
    t.relname = 'employee'
ORDER BY
    size_bytes DESC;
```

Admin Queries - Query: Table Data, Index, and Total Size

```
SELECT
    relname AS table_name,
    pg_size_pretty(pg_relation_size(relid)) AS data_size,
    pg_size_pretty(pg_indexes_size(relid)) AS index_size,
    pg_size_pretty(pg_total_relation_size(relid)) AS total_size
FROM
    pg_catalog.pg_statio_user_tables
WHERE
    relname = 'sensors.observations';
```

Backup and Restore All Databases

- You can back up all databases in PostgreSQL using `pg_dumpall` utility. The basic syntax to backup all databases as shown below:
 - `pg_dumpall -f [alldatabase_backup.sql]`
- For example, to back up all databases in PostgreSQL and generate a backup file named `alldb_backup.sql`, run the following command:
 - `pg_dumpall -f alldb_backup.sql`
- To restore all databases from a backup file named `alldb_backup.sql`, run the following command:
 - `psql -f alldb_backup.sql`

Backup and Restore Single Table

- PostgreSQL also allows you to back up a single table from the specific database. You can achieve this using the following syntax:
- `pg_dump -d [source-database] -t [table_name]-f [dbtable_backup.sql]`
- For example, to back up a table named mytab from the database named testdb and generate a backup file named testdb_mytab.sql, run the following command:
- `pg_dump -d testdb -t mytab -f testdb_mytab_backup.sql`
- If you want to restore this table from the backup file, run the following command:
- `psql -d testdb -f testdb_mytab_backup.sql`

Backup and Restore Compressed Data Store

- You can also back up the PostgreSQL database and compress it in .gz format to reduce the backup size.
- To take a back up of database named testdb and generate a compressed backup file named testdb_compressed.sql.gz, run the following command:

```
pg_dump -d testdb | gzip > testdb_compressed.sql.gz
```
- You can also restore the backup from the compressed file using the following command:

```
gunzip -c testdb_compressed.sql.gz | psql -d testdb
```


PL/PgSQL

- PL/pgSQL procedural language adds many procedural elements e.g., control structures, loops, and complex computations to extend standard SQL.
- It allows you to develop complex functions and stored procedures in PostgreSQL that may not be possible using plain SQL.
- Since PostgreSQL 9.0, PL/pgSQL is installed by default.

PL/PgSQL

- PL/pgSQL procedural language is similar to the Oracle PL/SQL. The following are reasons to learn PL/pgSQL:
 - PL/pgSQL is easy to learn and simple to use.
 - PL/pgSQL comes with PostgreSQL by default. The user-defined functions and stored procedures developed in PL/pgSQL can be used like any built-in functions and stored procedures.
 - PL/pgSQL inherits all user-defined types, functions, and operators.
 - PL/pgSQL has many features that allow you to develop complex functions and stored procedures.
 - PL/pgSQL can be defined to be trusted by the PostgreSQL database server.

PL/PGSQL

- PL/pgSQL is a procedural programming language for the PostgreSQL database system.
- PL/pgSQL allows you to extend the functionality of the PostgreSQL database server by creating server objects with complex logic.
- PL/pgSQL was designed to :
 - Create user-defined functions, stored procedures, and triggers.
 - Extend standard SQL by adding control structures such as if, case, and loop statements.
 - Inherit all user-defined functions, operators, and types.

Advantages of using PL/pgSQL

- SQL is a query language that allows you to query data from the database easily. However, PostgreSQL only can execute SQL statements individually.
- It means that you have multiple statements, you need to execute them one by one like this:
 - First, send a query to the PostgreSQL database server.
 - Next, wait for it to process.
 - Then, process the result set.
 - After that, do some calculations.
 - Finally, send another query to the PostgreSQL database server and repeat this process.

Advantages of using PL/pgSQL

- This process incurs the interprocess communication and network overheads.
- To resolve this issue, PostgreSQL uses PL/pgSQL.
 - PL/pgSQL wraps multiple statements in an object and store it on the PostgreSQL database server.
 - So instead of sending multiple statements to the server one by one, you can send one statement to execute the object stored in the server. This allows you to:
 - Reduce the number of round trips between the application and the PostgreSQL database server.
 - Avoid transferring the immediate results between the application and the server.

PostgreSQL PL/pgSQL disadvantages

- Besides the advantages of using PL/pgSQL, there are some caveats:
 - Slower in software development because PL/pgSQL requires specialized skills that many developers do not possess.
 - Difficult to manage versions and hard to debug.
 - May not be portable to other database management systems.

Dollar-Quoted String Constants

- In PostgreSQL, you use single quotes for a string constant like this:
 - `select 'String constant';`
- When a string constant contains a single quote ('), you need to escape it by doubling up the single quote. For example:
 - `select 'I''m also a string constant';`

Dollar-Quoted String Constants

- PostgreSQL version 8.0 introduced the dollar quoting feature to make string constants more readable.
- The following shows the syntax of the dollar-quoted string constants:
 - `$tag$<string_constant>$tag$`
- In this syntax, the tag is optional. It may contain zero or many characters.
 - Between the `$tag$`, you can place any string with single quotes (`'`) and backslashes (`\`).
 - For example:
 - `select $$I'm a string constant that contains a backslash \$$;`

Dollar-Quoted String Constants

- In this example, we did not specify the tag between the two dollar signs(\$).
- The following example uses the dollar-quoted string constant syntax with a tag:
 - `SELECT $message$I'm a string constant that contains a backslash \message$;`
 - In this example, we used the string message as a tag between the two dollar signs (\$)

Using dollar-quoted string constant in anonymous blocks

do

'declare

 film_count integer;

begin

 select count(*) into film_count

 from film;

 raise notice "The number of films: %", film_count;

end;;

Using dollar-quoted string constant in anonymous blocks

To avoid escaping every single quotes and backslashes, you can use the dollar-quoted string as follows

do

\$\$

declare

 film_count integer;

begin

 select count(*) into film_count

 from film;

 raise notice 'The number of films: %', film_count;

end;

\$\$

Using dollar-quoted string constant in anonymous blocks

Dashboard Properties SQL Statistics Dependencies Dependents accountdb/enterprisedb@EDB Postgres Advanced Server 13 *

accountdb/enterprisedb@EDB Postgres Advanced Server 13

Query Editor Query History

```
1 do
2 $$
3 declare
4     film_count integer;
5 begin
6     select count(*) into film_count
7     from films;
8     raise notice 'The number of films: %', film_count;
9 end;
10 $$
```

Data Output Explain Messages Notifications

NOTICE: The number of films: 3
DO

Query returned successfully in 1 secs 14 msec.

Using dollar-quoted string constants in functions

```
create function find_film_by_id(  
    id int  
) returns film  
language sql  
as  
$$  
    select * from film  
    where film_id = id;  
$$;
```

Using dollar-quoted string constants in functions

EDB-PSQL - 13

```
accountdb=# create function find_film_by_id(  
accountdb(#   id int  
accountdb(# ) returns films  
accountdb-# language sql  
accountdb-# as  
accountdb-# $$  
accountdb$$  select * from films  
accountdb$$    where film_id = id;  
accountdb$$ $$;  
CREATE FUNCTION
```

No limit

accountdb/enterprisedb@EDB Postgres Advanced Server 13 ▾

Query Editor Query History

1 **select** find_film_by_id(1) **from** dual;

Data Output Explain Messages Notifications

	find_film_by_id films	
1	(1,Joker)	

PL/pgSQL Block Structure

- PL/pgSQL is a block-structured language, therefore, a PL/pgSQL function or stored procedure is organized into blocks.

```
[ <<label>> ]
```

```
[ declare  
    declarations ]
```

```
begin  
    statements;
```

```
    ...
```

```
end [ label ];
```

PL/pgSQL Block Structure

- PL/pgSQL is a block-structured language, therefore, a PL/pgSQL function or stored procedure is organized into blocks.

```
[ <<label>> ]
```

```
[ declare  
    declarations ]
```

```
begin  
    statements;
```

```
    ...
```

```
end [ label ];
```


PL/pgSQL Block Structure

- Each block has two sections: declaration and body.
- The declaration section is optional while the body section is required.
- A block is ended with a semicolon (;) after the END keyword.
- A block may have an optional label located at the beginning and at the end.
- We use the block label when you want to specify it in the EXIT statement of the block body or when you want to qualify the names of variables declared in the block.
- The declaration section is where you declare all variables used within the body section.
- Each statement in the declaration section is terminated with a semicolon (;).
- The body section is where you place the code. Each statement in the body section is also terminated with a semicolon (;).

PL/pgSQL Block Structure

```
do $$
<<first_block>>
declare
    film_count integer := 0;
begin
    -- get the number of films
    select count(*)
    into film_count
    from films;
    -- display a message
    raise notice 'The number of films is %', film_count;
end first_block $$;
```

PL/pgSQL Block Structure

The screenshot displays the EDB PostgreSQL Advanced Server 13 Query Editor interface. The top navigation bar includes tabs for Dashboard, Properties, SQL, Statistics, Dependencies, and Dependents. The current connection is 'accountdb/enterprisedb@EDB Postgres Advanced Server 13 *'. The toolbar contains various icons for file operations, search, and execution. The main area shows the Query Editor with a PL/pgSQL block. The block starts with 'do \$\$', followed by a block comment '<<first_block>>', a 'declare' section for 'film_count integer := 0;', a 'begin' section containing a 'select count(*) into film_count from films;' query and a 'raise notice' statement, and ends with 'end first_block \$\$;'. The bottom section shows the Messages tab with the output: 'NOTICE: The number of films is 3' and 'DO'. A status message at the bottom indicates 'Query returned successfully in 217 msec.'

Dashboard Properties SQL Statistics Dependencies Dependents accountdb/enterprisedb@EDB Postgres Advanced Server 13 *

accountdb/enterprisedb@EDB Postgres Advanced Server 13

Query Editor Query History

```
1 do $$
2 <<first_block>>
3 declare
4     film_count integer := 0;
5 begin
6     -- get the number of films
7     select count(*)
8     into film_count
9     from films;
10    -- display a message
11    raise notice 'The number of films is %', film_count;
12 end first_block $$;
```

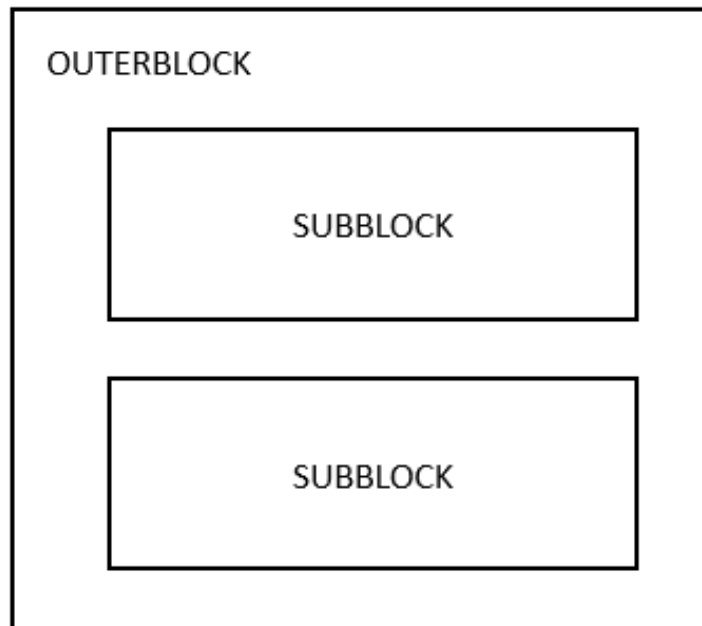
Data Output Explain Messages Notifications

NOTICE: The number of films is 3
DO

Query returned successfully in 217 msec.

PL/pgSQL Subblocks

- PL/pgSQL allows you to place a block inside the body of another block.
- The block nested inside another block is called a subblock. The block that contains the subblock is referred to as an outer block.



Introduction to variables in PL/pgSQL

- A variable is a meaningful name of a memory location.
- A variable holds a value that can be changed through the block.
- A variable is always associated with a particular data type.
- `variable_name data_type [:= expression];`

Introduction to variables in PL/pgSQL

- First, specify the name of the variable.
- It is a good practice to assign a meaningful name to a variable.
- For example, instead of naming a variable `i` you should use `index` or `counter`.
- Second, associate a specific data type with the variable.
- The data type can be any valid data type such as integer, numeric, varchar, and char.
- Third, optionally assign a default value to a variable. If you don't do so, the initial value of the variable is NULL.

Introduction to variables in PL/pgSQL

- First, specify the name of the variable.
- It is a good practice to assign a meaningful name to a variable.
- For example, instead of naming a variable `i` you should use `index` or `counter`.
- Second, associate a specific data type with the variable.
- The data type can be any valid data type such as integer, numeric, varchar, and char.
- Third, optionally assign a default value to a variable. If you don't do so, the initial value of the variable is NULL.
- Note that you can use either `:=` or `=` assignment operator to initialize and assign a value to a variable.

Introduction to variables in PL/pgSQL

```
do $$
declare
    counter    integer := 1;
    first_name varchar(50) := 'John';
    last_name  varchar(50) := 'Doe';
    payment    numeric(11,2) := 20.5;
begin
    raise notice '% % % has been paid % USD',
        counter,
        first_name,
        last_name,
        payment;
end $$;
```


Variable initialization timing

```
do $$  
declare  
    created_at time := now();  
begin  
    raise notice '%', created_at;  
    perform pg_sleep(10);  
    raise notice '%', created_at;  
end $$;
```

Variable initialization timing

accountdb/enterprisedb@EDB Postgres Advanced Server 13 ▾

Query Editor Query History

```
1 do $$
2 declare
3     created_at time := now();
4 begin
5     raise notice '%', created_at;
6     perform pg_sleep(10);
7     raise notice '%', created_at;
8 end $$;
9
```

Data Output Explain Messages Notifications

NOTICE: 20:46:10.686542
NOTICE: 20:46:10.686542
DO

Query returned successfully in 10 secs 636 msec.

Copying data types

- The %type provides the data type of a table column or another variable.
- Typically, you use the %type to declare a variable that holds a value from the database or another variable.
 - variable_name table_name.column_name%type;
- And the following shows how to declare a variable with the data type of another variable:
 - variable_name variable%type;

Copying data types

```
do $$
declare
    film_title films.title%type;
    featured_title film_title%type;
begin
    -- get title of the film id 100
    select title
    from films
    into film_title
    where film_id = 1;

    -- show the film title
    raise notice 'Film title id 100: %s', film_title;
end; $$
```

Variables in block and subblock

- When you declare a variable in a subblock which has the same name as another variable in the outer block, the variable in the outer block is hidden in the subblock.

```

do $$
<<outer_block>>
declare
    counter integer := 0;
begin
    counter := counter + 1;
    raise notice 'The current value of the
counter is %', counter;

    declare
        counter integer := 0;
    begin
        counter := counter + 10;
        raise notice 'Counter in the subblock is
%', counter;
        raise notice 'Counter in the outer block
is %', outer_block.counter;
    end;

    raise notice 'Counter in the outer block is
%', counter;

end outer_block $$;

```

PL/pgSQL row types example

```
do $$
declare
    selected_film films%rowtype;
begin
    -- select film with id 1
    select *
    from films
    into selected_film
    where film_id = 1;

    -- show the number of actor
    raise notice 'The film name is % %',
        selected_film.film_id,
        selected_film.title;
end; $$
```

PL/pgSQL Record Types

```
do
$$
declare
    rec record;
begin
    -- select the film
    select film_id, title
    into rec
    from films
    where film_id = 1;

    raise notice '% %', rec.film_id, rec.title;

end;
$$
```


Using record variables in the for loop statement

```
do
$$
declare
    rec record;
begin
    for rec in select title
                    from films
                    where film_id > 0
                    order by title
    loop
        raise notice '%', rec.title;
    end loop;
end;
$$
```

PL/pgSQL Constants

- do \$\$
- declare
 - vat constant numeric := 0.1;
 - net_price numeric := 20.5;
- begin
 - raise notice 'The selling price is %', net_price * (1 + vat);
- end \$\$;

PL/pgSQL Errors and Messages

- Reporting messages
- To raise a message, you use the raise statement as follows:
 - raise level format;
- Following the raise statement is the level option that specifies the error severity.
- PostgreSQL provides the following levels:
 - debug
 - log
 - notice
 - info
 - warning
 - exception

Reporting Messages

do \$\$

begin

raise info 'information message %', now() ;

raise log 'log message %', now();

raise debug 'debug message %', now();

raise warning 'warning message %', now();

raise notice 'notice message %', now();

end \$\$;

Raising errors

- To raise an error, you use the exception level after the raise statement.
- Note that raise statement uses the exception level by default.

Raising errors

- Besides raising an error, you can add more information by using the following additional clause:
- using option = expression
- The option can be:
 - message: set error message
 - hint: provide the hint message so that the root cause of the error is easier to be discovered.
 - detail: give detailed information about the error.
 - errcode: identify the error code, which can be either by condition name or directly five-character SQLSTATE code. Please refer to the table of error codes and condition names.

Raising Errors

```
do $$  
declare  
    email varchar(255) := 'info@postgresqltutorial.com';  
begin  
    -- check email for duplicate  
    -- ...  
    -- report duplicate email  
    raise exception 'duplicate email: %', email  
        using hint = 'check the email again';  
end $$;
```

Assert Statement

do \$\$

declare

 film_count integer;

begin

 select count(*)

 into film_count

 from films;

 assert film_count > 1000, '1000 Film not found, check the
film table';

end\$\$;

PL/pgSQL IF Statement

- The if statement determines which statements to execute based on the result of a boolean expression.
- PL/pgSQL provides you with three forms of the if statements.
 - if then
 - if then else
 - if then elsif

PL/pgSQL IF Statement

- PL/pgSQL if-then statement
- The following illustrates the simplest form of the if statement:
 - if condition then
 - statements;
 - end if;

PL/pgSQL IF Statement

```
do $$  
declare  
    selected_film films%rowtype;  
    input_film_id films.film_id%type := 0;  
begin  
    select * from films  
    into selected_film  
    where film_id = input_film_id;  
    if not found then  
        raise notice 'The film % could not be found',  
            input_film_id;  
    end if;  
end $$
```

PL/pgSQL if-then-else statement

if condition then

 statements;

else

 alternative-statements;

END if;

PL/pgSQL if-then-else statement

if condition then

 statements;

else

 alternative-statements;

END if;

PL/pgSQL if-then-else statement

```
do $$
```

```
declare
```

```
    selected_film films%rowtype;
```

```
    input_film_id films.film_id%type := 100;
```

```
begin
```

```
    select * from films
```

```
    into selected_film
```

```
    where film_id = input_film_id;
```

```
    if not found then
```

```
        raise notice 'The film % could not be found',
```

```
            input_film_id;
```

```
    else
```

```
        raise notice 'The film title is %', selected_film.title;
```

```
    end if;
```

```
end $$
```

PL/pgSQL if-then-elsif Statement

- if condition_1 then
- statement_1;
- elsif condition_2 then
- statement_2
- ...
- elsif condition_n then
- statement_n;
- else
- else-statement;
- end if;

PL/pgSQL if-then-elsif Statement

```
do $$  
declare  
    v_film films%rowtype;  
    len_description varchar(100);  
begin  
  
    select * from films  
    into v_film  
    where film_id = 100;  
  
    if not found then  
        raise notice 'Film not found';  
    else
```


PL/pgSQL if-then-elsif Statement

```
if v_film.length >0 and v_film.length <= 50 then
    len_description := 'Short';
elsif v_film.length > 50 and v_film.length < 120 then
    len_description := 'Medium';
elsif v_film.length > 120 then
    len_description := 'Long';
else
    len_description := 'N/A';
end if;

raise notice 'The % film is %.',
    v_film.title,
    len_description;

end if;
end $$
```

PL/pgSQL CASE Statement

- The case statement selects a when section to execute from a list of when sections based on a condition.
- The case statement has two forms:
 - Simple case statement
 - Searched case statement

Simple case statement

case search-expression

when expression_1 [, expression_2, ...] then

when-statements

[...]

[else

else-statements]

END case;

Searched case statement

- case
- when boolean-expression-1 then
- statements
- [when boolean-expression-2 then
- statements
- ...]
- [else
- statements]
- end case;

While Loop

- do \$\$
- declare
 - counter integer := 0;
- begin
 - while counter < 5 loop
 - raise notice 'Counter %', counter;
 - counter := counter + 1;
 - end loop;
- end\$\$;

For Loop

- [<<label>>]
- for loop_counter in [reverse] from.. to [by step] loop
- statements
- end loop [label];

For Loop

- do
- \$\$
- declare
- f record;
- begin
- for f in select title
- from films
- order by title
- limit 10
- loop
- raise notice '%', f.title;
- end loop;
- end;
- \$\$

Introduction to PostgreSQL CREATE PROCEDURE statement

- create [or replace] procedure
 procedure_name(parameter_list)
- language plpgsql
- as \$\$
- declare
- -- variable declaration
- begin
- -- stored procedure body
- end; \$\$

Introduction to PostgreSQL CREATE PROCEDURE statement

- First, specify the name of the stored procedure after the create procedure keywords.
- Second, define parameters for the stored procedure. A stored procedure can accept zero or more parameters.
- Third, specify plpgsql as the procedural language for the stored procedure. Note that you can use other procedural languages for the stored procedure such as SQL, C, etc.
- Finally, use the dollar-quoted string constant syntax to define the body of the stored procedure.

Create Function

- create [or replace] function function_name(param_list)
- returns return_type
- language plpgsql
- as
- \$\$
- declare
- -- variable declaration
- begin
- -- logic
- end;
- \$\$

PL/pgSQL Function Parameter Modes: IN, OUT, INOUT

IN	OUT	INOUT
The default	Explicitly specified	Explicitly specified
Pass a value to function	Return a value from a function	Pass a value to a function and return an updated value.
in parameters act like constants	out parameters act like uninitialized variables	inout parameters act like an initialized variables
Cannot be assigned a value	Must assign a value	Should be assigned a value

Introduction to PL/pgSQL Function Overloading

- PostgreSQL allows multiple functions to share the same name as long as they have different arguments.
- If two or more functions share the same name, the function names are overloaded.
- When you can call an overloading function, PostgreSQL select the best candidate function to execute based on the the function argument list.

How to Develop a PL/pgSQL Function That Returns a Table

- create or replace function function_name (
- parameter_list
-)
- returns table (column_list)
- language plpgsql
- as \$\$
- declare
- -- variable declaration
- begin
- -- body
- end; \$\$

Triggers

- A PostgreSQL trigger is a function invoked automatically whenever an event such as insert, update, or delete occurs.

Triggers

- A PostgreSQL trigger is a function invoked automatically whenever an event associated with a table occurs.
- An event could be any of the following: INSERT, UPDATE, DELETE or TRUNCATE.
- A trigger is a special user-defined function associated with a table.
- To create a new trigger, you define a trigger function first, and then bind this trigger function to a table.
- The difference between a trigger and a user-defined function is that a trigger is automatically invoked when a triggering event occurs.
- PostgreSQL provides two main types of triggers: row and statement-level triggers. The differences between the two kinds are how many times the trigger is invoked and at what time.

Triggers

- For example, if you issue an UPDATE statement that affects 20 rows, the row-level trigger will be invoked 20 times, while the statement level trigger will be invoked 1 time.
- You can specify whether the trigger is invoked before or after an event.
- If the trigger is invoked before an event, it can skip the operation for the current row or even change the row being updated or inserted.
- In case the trigger is invoked after the event, all changes are available to the trigger.

Triggers

- Triggers are useful in case the database is accessed by various applications, and you want to keep the cross-functionality within the database that runs automatically whenever the data of the table is modified.
- For example, if you want to keep the history of data without requiring the application to have logic to check for every event such as INSERT or UPDATE.

Triggers

- You can also use triggers to maintain complex data integrity rules which you cannot implement elsewhere except at the database level.
- For example, when a new row is added into the customer table, other rows must be also created in tables of banks and credits.

Triggers

- The main drawback of using a trigger is that you must know the trigger exists and understand its logic to figure out the effects when data changes.
- Even though PostgreSQL implements SQL standard, triggers in PostgreSQL has some specific features:
 - PostgreSQL fires trigger for the TRUNCATE event.
 - PostgreSQL allows you to define the statement-level trigger on views.
 - PostgreSQL requires you to define a user-defined function as the action of the trigger, while the SQL standard allows you to use any SQL commands.

PostgreSQL CREATE TRIGGER

- To create a new trigger in PostgreSQL, you follow these steps:
 - First, create a trigger function using CREATE FUNCTION statement.
 - Second, bind the trigger function to a table by using CREATE TRIGGER statement.

PostgreSQL CREATE TRIGGER

```
CREATE FUNCTION trigger_function()  
    RETURNS TRIGGER  
    LANGUAGE PLPGSQL  
AS $$  
BEGIN  
    -- trigger logic  
END;  
$$
```

Inheritance

```
CREATE TABLE CUSTOMERS (  
    name text,  
    age int,  
    address text  
);
```

```
CREATE TABLE OFFICE_CUSTOMERS(  
    office_address text  
) INHERITS (customers)
```

Inheritance

```
INSERT INTO CUSTOMERS VALUES('ravi','32','32,  
head street');
```

```
INSERT INTO CUSTOMERS VALUES('michael','35','56,  
gotham street');
```

```
INSERT INTO OFFICE_CUSTOMERS  
VALUES('bane','28','56, circadia street','92 homebush');
```

Non Atomic Columns

```
CREATE TABLE customer (  
    name          text,  
    Address       text,  
    payment_schedule integer[],  
);
```


Non Atomic Columns

```
INSERT INTO CUSTOMER_SCHEDULE VALUES( 'jack',  
'Athens, Colarado',  
'{1,2,3,4}'  
)
```

```
INSERT INTO CUSTOMER_SCHEDULE VALUES(  
'jackson',  
'Tennessey, greece',  
'{1,7,3,4}'  
)
```

Non Atomic Columns

```
SELECT * FROM CUSTOMER_SCHEDULE WHERE  
CUSTOMER_SCHEDULE.payment_schedule[1] <>  
CUSTOMER_SCHEDULE.payment_schedule[2];
```

Postgres Cluster

- `.\initdb.exe -D e:/PostgreSQL/14/data`
- Change the server port in data folder config file
- `.\pg_ctl.exe start -D e:\PostgreSQL\14\data`
- Server started
- `psql -h localhost -d postgres -p 5436 -W`

Postgres Replication

- #5432
- `CREATE TABLE LogicalReplicationTest (id int PRIMARY KEY);`
- `INSERT INTO LogicalReplicationTest VALUES (generate_series(1,10000));`
- `SELECT count(*) FROM LogicalReplicationTest;`
- `CREATE PUBLICATION testpub FOR TABLE LogicalReplicationTest;`

- #5436
- `CREATE TABLE LogicalReplicationTest (id int PRIMARY KEY);`
- `CREATE SUBSCRIPTION testsub CONNECTION 'host=localhost port=5432 user=postgres password=vignesh dbname=testdb' PUBLICATION testpub;`

Basic Transaction Syntax:

```
BEGIN;  
-- SQL operations  
COMMIT;    -- Save changes  
-- or  
ROLLBACK;  -- Undo changes
```

```
BEGIN;  
  
UPDATE accounts SET balance = balance - 100 WHERE id = 1;  
UPDATE accounts SET balance = balance + 100 WHERE id = 2;  
  
COMMIT;
```

Basic Transaction Syntax:

```
BEGIN;
```

```
UPDATE products SET stock = stock - 5 WHERE id = 100;
```

```
-- Oops: this fails due to constraint
```

```
INSERT INTO logs VALUES (NULL, 'bad insert');
```

```
ROLLBACK; -- Reverts the stock update too
```

Transaction Control Summary

Command	Purpose
BEGIN	Start transaction
COMMIT	Apply changes
ROLLBACK	Undo changes
SAVEPOINT name	Set save point inside tx
ROLLBACK TO SAVEPOINT name	Partial rollback

Postgres Security

-  1. AuthenticationControls who can connect to the database.
- Configured in `pg_hba.conf` (Host-Based Authentication)
- Common methods:
 - trust: No password (not recommended for production)
 - md5 / scram-sha-256: Password-based (md5 deprecated gradually)
 - peer: OS user must match database user (local Unix)
 - cert: SSL certificate authentication
 - ldap, gss, kerberos, pam, radius: External systems

Postgres Security

- Select name,setting from pg_settings;

name	setting
allow_in_place_tablespaces	off
allow_system_table_mods	off
application_name	psql
archive_cleanup_command	
archive_command	(disabled)
archive_mode	off
archive_timeout	0
array_nulls	on
authentication_timeout	60
autovacuum	on
autovacuum_analyze_scale_factor	0.1
autovacuum_analyze_threshold	50
autovacuum_freeze_max_age	200000000
autovacuum_max_workers	3
autovacuum_multixact_freeze_max_age	400000000
autovacuum_naptime	60
autovacuum_vacuum_cost_delay	2
autovacuum_vacuum_cost_limit	-1
autovacuum_vacuum_scale_factor	0.2
autovacuum_vacuum_threshold	50
autovacuum_work_mem	-1
backend_flush_after	0
backslash_quote	safe_encoding
bgwriter_delay	200
bgwriter_flush_after	64
bgwriter_lru_maxpages	100
bgwriter_lru_multiplier	2
block_size	8192
bonjour	off
bonjour_name	
bytea_output	hex
check_function_bodies	on
checkpoint_completion_target	0.5
checkpoint_flush_after	32
checkpoint_timeout	300

Postgres Security

-  **2. Authorization (Roles & Privileges)**
- Controls what a user can do.
-  **Roles**
 - Superusers (CREATE ROLE myuser SUPERUSER)
 - Normal users (LOGIN privilege)
 - Can be granted or revoked:

```
CREATE ROLE readonly;  
GRANT CONNECT ON DATABASE mydb TO readonly;  
GRANT USAGE ON SCHEMA public TO readonly;  
GRANT SELECT ON ALL TABLES IN SCHEMA public TO readonly;
```

Postgres Security

```
CREATE ROLE appuser LOGIN PASSWORD 'secret'  
NOSUPERUSER NOCREATEDB NOCREATEROLE  
NOINHERIT NOREPLICATION;
```

Object	Privileges
Table	SELECT, INSERT, UPDATE, DELETE, TRUNCATE, REFERENCES, TRIGGER
Database	CONNECT, TEMP
Schema	USAGE, CREATE
Function	EXECUTE

Postgres Security Object Level Permission

```
GRANT SELECT ON employees TO readonly;  
REVOKE DELETE ON employees FROM appuser;
```

Object	Privileges
Table	SELECT, INSERT, UPDATE, DELETE, TRUNCATE, REFERENCES, TRIGGER
Database	CONNECT, TEMP
Schema	USAGE, CREATE
Function	EXECUTE

Postgres Security Row-Level Security (RLS)

```
ALTER TABLE employees ENABLE ROW LEVEL SECURITY;  
CREATE POLICY hr_policy  
  ON employees  
  USING (department = current_setting('app.current_department'));  
ALTER TABLE employees ENABLE POLICY hr_policy;
```

Use SET app.current_department = 'HR'; at session level.

Questions



Module Summary

- In this module we discussed
 - Installing Postgres
 - Querying Postgres
 - Data Management
 - PL/SQL

